

Automation Testing với Playwright

Từ khái niệm QA cơ bản đến automation QA và nâng cao với Playwright +
AI/MCP

Mục lục

Phần I — Nền tảng QA trong SDLC

Chương 1. SDLC là gì? Vị trí của kiểm thử (Testing) trong vòng đời phát triển phần mềm

Chương 2. Các loại kiểm thử: Manual vs Automation, Unit / Integration / E2E / Regression

Chương 3. Test case, test plan, bug report — những khái niệm cơ bản mọi QA cần biết

Phần II — Vai trò QA & Automation QA

Chương 4. PQA (Product QA), FQA (Functional QA), TQA (Test/Technical QA) — phân biệt & trách nhiệm

Chương 5. Automation QA là gì? Khi nào nên tự động hoá, khi nào không

Chương 6. Kỹ năng & tư duy cần có của một Automation QA Engineer

Phần III — Cài đặt môi trường

Chương 7. Cài đặt Node.js, quản lý package (npm/pnpm), VS Code & extension hữu ích

Chương 8. Khởi tạo project Playwright, cấu trúc thư mục project

Chương 9. Giới thiệu demo app dùng xuyên suốt sách

Phần IV — Playwright cơ bản

Chương 10. Playwright là gì? Kiến trúc, so sánh nhanh với Selenium/Cypress

Chương 11. Test đầu tiên: Locators, Actions, Assertions

Chương 12. Cấu trúc test: test, describe, hooks (beforeEach/afterEach)

Chương 13. Làm việc với form, input, dropdown, upload file

Chương 14. Auto-waiting, xử lý bất đồng bộ, network

Chương 15. Page Object Model (POM) — tổ chức code test để bảo trì

Chương 16. Fixtures cơ bản, tái sử dụng logic test

Chương 17. Debug: Trace Viewer, UI Mode, --debug, codegen

Chương 18. Reporting: HTML reporter, tích hợp Allure

Chương 19. CI/CD: chạy Playwright trên GitHub Actions

Phần V — Playwright nâng cao

Chương 20. API testing với request context (kết hợp UI + API test)

Chương 21. Visual testing / so sánh screenshot

Chương 22. Authentication & storage state (test app có đăng nhập)

Chương 23. Chạy song song, sharding, tối ưu tốc độ test suite

Chương 24. Cross-browser testing & mobile emulation

Chương 25. Component testing với Playwright

Chương 26. Custom fixtures nâng cao, viết plugin/reporter riêng

Phần VI — Nâng cao: AI + MCP / Browser Automation

Chương 27. Model Context Protocol (MCP) là gì, vì sao liên quan đến automation test

Chương 28. Playwright MCP server — để AI agent điều khiển trình duyệt

Chương 29. Kết hợp AI sinh test case, self-healing test khi UI thay đổi

Chương 30. Case study thực tế: xây pipeline test bán tự động với AI + Playwright

Phụ lục

Phụ lục A — Best practices & checklist automation test

Phụ lục B — Các lỗi thường gặp và cách xử lý (troubleshooting)

Phụ lục C — Tài liệu tham khảo & cộng đồng

Phần I — Nền tảng QA trong SDLC

Chương 1: SDLC là gì? Vị trí của kiểm thử trong vòng đời phát triển phần mềm

Mục tiêu học

- Gọi tên và giải thích được 6 giai đoạn cơ bản của SDLC.
- Phân biệt cách testing được lồng vào SDLC ở mô hình Waterfall so với Agile/DevOps.
- Giải thích được nguyên lý "Cost of Defect" — vì sao phát hiện lỗi càng trễ càng tốn kém.
- Áp dụng được tư duy "shift-left testing" vào một tính năng cụ thể.

Ví dụ mẫu: `code/ch01-sdlc/test-case-list-login.md` — một bộ test case hoàn chỉnh cho tính năng đăng nhập, viết ra *trước khi* code được viết (đúng tinh thần shift-left ở mục 1.3). Đọc song song với mục 1.4 để thấy rõ QA tham gia sớm như thế nào trong thực tế.

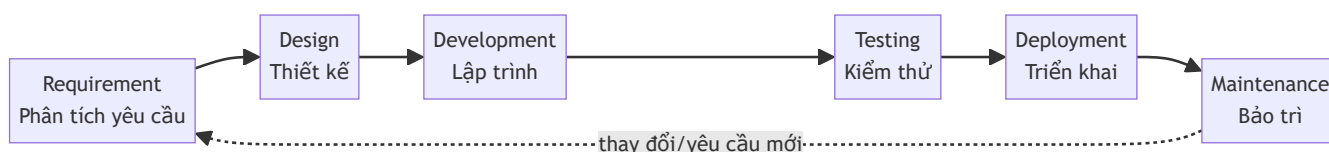
1.1. Vì sao lập trình viên cần hiểu SDLC?

Nếu bạn đã biết lập trình, bạn quen với việc viết code, chạy thử, commit, và push. Nhưng viết một tính năng chạy được chỉ là một mắt xích nhỏ trong một quy trình lớn hơn nhiều — quy trình đó được gọi là **SDLC (Software Development Life Cycle — Vòng đời phát triển phần mềm)**.

Hiểu SDLC quan trọng với người làm automation test vì một lý do đơn giản: **test không phải là một bước làm sau cùng**, mà là một hoạt động cần được lồng vào *mọi* giai đoạn của SDLC. Nếu không biết SDLC vận hành ra sao, bạn sẽ không biết nên viết test gì, viết lúc nào, và test đó phục vụ mục đích gì trong bức tranh chung.

1.2. SDLC là gì?

SDLC là tập hợp các giai đoạn (phases) mà một đội phát triển phần mềm đi qua để biến một ý tưởng/yêu cầu thành sản phẩm hoạt động, được vận hành, và được bảo trì:



Đây là mô hình khái niệm chung. Trên thực tế, thứ tự và cách các giai đoạn này lặp lại phụ thuộc vào **mô hình phát triển (SDLC model)** mà đội ngũ áp dụng.

Các mô hình SDLC phổ biến

Mô hình	Đặc điểm	Testing diễn ra khi nào
Waterfall	Các giai đoạn tuần tự, giai đoạn sau chỉ bắt đầu khi giai đoạn trước hoàn tất	Chỉ ở cuối, sau khi code xong toàn bộ — phát hiện lỗi rất trễ
Agile / Scrum	Chia nhỏ thành các sprint (thường 1–2 tuần), mỗi sprint đều có đủ analysis → design → dev → test	Testing diễn ra trong từng sprint , song song với development
DevOps / CI-CD	Tích hợp và triển khai liên tục, tự động hoá tối đa	Testing (đặc biệt automation test) chạy liên tục , tự động, mỗi khi có code mới

Phần lớn các công ty hiện nay dùng Agile kết hợp DevOps — đây cũng là bối cảnh mà automation testing (chủ đề chính của sách này) phát huy giá trị lớn nhất, vì test cần chạy nhanh và lặp lại liên tục.

1.3. Vị trí của Testing trong SDLC

Trong mô hình Waterfall truyền thống

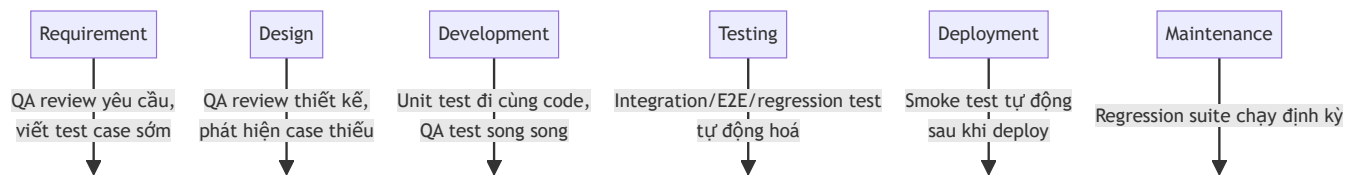
Testing là một giai đoạn **riêng biệt, nằm cuối**:



Vấn đề: nếu một lỗi đến từ việc hiểu sai yêu cầu ở bước 1, thì phải đến tận bước 4 nó mới bị phát hiện — lúc đó, sửa lỗi rất tốn kém vì code đã được xây dựng dựa trên hiểu sai đó.

Trong Agile/DevOps hiện đại: Shift-Left Testing

Tư duy hiện đại gọi là **"shift-left"** — dịch hoạt động testing về *sớm hơn* trong vòng đời, thay vì dồn hết vào cuối:



Testing không còn là một "cổng chặn" ở cuối, mà là hoạt động **xuyên suốt**.

Vì sao phát hiện lỗi càng trễ càng tốn kém?

Đây là nguyên lý kinh điển trong QA, thường gọi là **"Cost of Defect"**: chi phí sửa một lỗi tăng theo cấp số nhân tùy vào giai đoạn nó bị phát hiện.

Giai đoạn phát hiện lỗi	Chi phí sửa (tương đối)
Lúc phân tích yêu cầu	1x
Lúc thiết kế	~5x
Lúc lập trình	~10x
Lúc testing	~15–40x
Sau khi đã lên production	~30–100x

Đây chính là lý do automation test — vốn có thể chạy lại nhanh, thường xuyên, ngay khi có code mới — trở thành công cụ quan trọng để phát hiện lỗi sớm nhất có thể, thay vì đợi đến cuối sprint hay cuối release.

1.4. Ví dụ minh họa: tính năng "Đăng nhập" đi qua SDLC

Giả sử team đang xây một tính năng đăng nhập cho web app. Vai trò QA xuất hiện ở từng bước như sau:

1. **Requirement:** QA đọc yêu cầu "Người dùng đăng nhập bằng email + password". QA đặt câu hỏi: Nếu sai password 5 lần thì sao? Có giới hạn độ dài password không? → giúp làm rõ yêu cầu *trước khi* code được viết.
2. **Design:** QA xem thiết kế API `/login`, thiết kế UI form đăng nhập, phát hiện thiếu case "email không tồn tại" trong flow.
3. **Development:** Dev viết unit test cho hàm validate email. QA chuẩn bị sẵn bộ test case (xem `code/ch01-sdlc/test-case-list-login.md`) song song với lúc dev code.
4. **Testing:** QA (hoặc automation test bằng Playwright — nội dung chính từ Phần IV trở đi) chạy các kịch bản trong bộ test case đó: đăng nhập đúng, đăng nhập sai password, để trống field, v.v.
5. **Deployment:** Một smoke test tự động (ví dụ 1 test Playwright đơn giản: "đăng nhập thành công với tài khoản demo") chạy ngay sau khi deploy để đảm bảo tính năng không bị vỡ trên production.
6. **Maintenance:** Vài tuần sau, team sửa lại UI trang login. Bộ regression test tự động (đã viết bằng Playwright) chạy lại toàn bộ để đảm bảo không có gì bị hỏng do thay đổi này.

Ở bước 4 và 6, chính là nơi automation testing — chủ đề toàn bộ cuốn sách này — đóng vai trò lớn nhất.

Xem ví dụ chi tiết: `code/ch01-sdlc/test-case-list-login.md` là bộ test case thật, viết theo đúng format một QA sẽ dùng trong công việc (ID, tiền điều kiện, các bước, kết quả mong đợi) — đây chính là "sản phẩm đầu ra" của bước Requirement/Development ở trên, và là input trực tiếp để viết automation test bằng Playwright sau này (Phần IV).

Bài tập

1. Chọn một tính năng bất kỳ mà bạn từng code (ví dụ: form đăng ký, giỏ hàng, tìm kiếm...).
 2. Liệt kê 6 giai đoạn SDLC (Requirement → Maintenance) áp dụng cho tính năng đó.
 3. Với mỗi giai đoạn, viết ra: "Nếu có QA tham gia ở đây, họ sẽ làm gì và có thể phát hiện lỗi gì?"
 4. Tham khảo format ở `code/ch01-sdlc/test-case-list-login.md`, tự viết một bộ test case tương tự (tối thiểu 5 test case) cho tính năng bạn đã chọn.
 5. Đánh dấu giai đoạn nào theo bạn là **quá trễ** để phát hiện lỗi nếu chỉ test ở đó — so sánh với nguyên lý "Cost of Defect" ở mục 1.3.
-

Lỗi thường gặp

- **Coi testing là việc "của QA" xảy ra sau khi dev code xong:** đây chính là tư duy Waterfall lỗi thời. Trong Agile/DevOps, QA tham gia từ bước Requirement, không đợi đến khi có build để test.
 - **Không viết test case trước, chỉ "test tay theo cảm tính":** dẫn đến bỏ sót case (ví dụ quên test "để trống field", "sai định dạng email"). Luôn có một danh sách test case tường minh, dù ngắn, trước khi bắt đầu test — kể cả automation test.
 - **Chỉ chạy test một lần lúc release, không chạy lại (regression) sau mỗi thay đổi:** đây là lý do chính khiến automation test trở nên cần thiết — chạy tay lại toàn bộ test case sau mỗi thay đổi nhỏ là không khả thi về thời gian.
-

Tóm tắt & tiếp theo

- SDLC là vòng đời phát triển phần mềm gồm các giai đoạn: Requirement, Design, Development, Testing, Deployment, Maintenance.
- Có nhiều mô hình SDLC (Waterfall, Agile, DevOps); cách testing được lồng vào khác nhau ở mỗi mô hình.
- Xu hướng hiện đại là **shift-left testing**: đưa testing vào càng sớm càng tốt, xuyên suốt vòng đời thay vì chỉ ở cuối.
- Lỗi phát hiện càng trễ, chi phí sửa càng cao — đây là động lực chính cho automation testing.

Chương 2 sẽ đi sâu vào các **loại kiểm thử** (Manual vs Automation, Unit/Integration/E2E/Regression) — nền tảng để hiểu automation test (bằng Playwright) nên thay thế và bổ sung cho manual test ở những điểm nào.

Chương 2: Các loại kiểm thử — Manual vs Automation, Unit / Integration / E2E / Regression

Mục tiêu học

- Phân biệt Manual testing và Automation testing, biết khi nào loại nào phù hợp hơn.
- Giải thích được "Testing Pyramid" và vị trí của Unit / Integration / E2E test trong đó.
- Viết được ví dụ cụ thể phân biệt Unit test và Integration test bằng code thật.
- Hiểu Regression test là gì và vì sao nó gắn liền chặt chẽ với automation test.

Code mẫu: `code/ch02-loai-kiem-thu/` — một module tính tiền giỏ hàng nhỏ, có cả unit test và integration test chạy được bằng `node`, không cần cài gì thêm.

2.1. Manual Testing vs Automation Testing

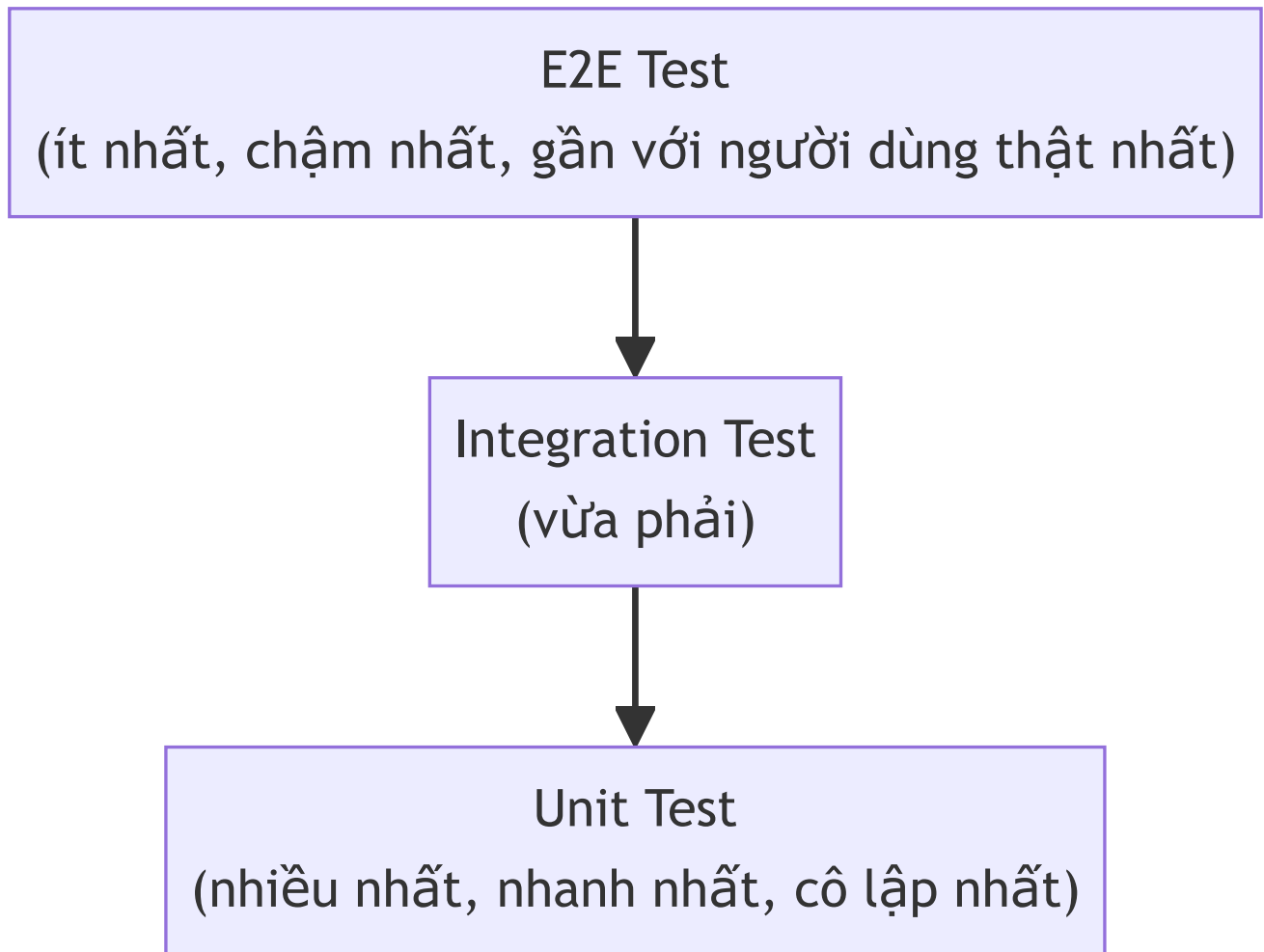
Manual testing: con người trực tiếp thao tác trên ứng dụng (click, nhập liệu, quan sát) để kiểm tra kết quả có đúng như mong đợi không — không có code nào "tự chạy" test.

Automation testing: viết code để tự động thực hiện lại các bước thao tác và tự động so sánh kết quả — như các test Playwright mà sách này sẽ dạy từ Phần IV.

Tiêu chí	Manual Testing	Automation Testing
Tốc độ chạy lại (regression)	Chậm, tốn nhân lực mỗi lần	Nhanh, chạy lại bao nhiêu lần cũng được với chi phí gần như 0
Chi phí ban đầu	Thấp (không cần viết code)	Cao hơn (cần thời gian viết & bảo trì test)
Phù hợp với	Exploratory testing, UI/UX review, test một lần, case phức tạp/hiếm	Test lặp lại nhiều lần (regression), test chạy trong CI/CD
Độ tin cậy khi lặp lại	Phụ thuộc con người, dễ bỏ sót bước	Nhất quán tuyệt đối — chạy đúng y hệt mỗi lần
Phát hiện lỗi UI/UX tinh tế	Tốt (con người "cảm" được cái gì "trông sai")	Kém hơn, trừ khi có visual testing chuyên biệt (Chương 21)

Quan trọng: Automation testing không thay thế hoàn toàn Manual testing. Automation giỏi ở việc lặp lại chính xác một kịch bản đã biết trước (regression); Manual/Exploratory testing giỏi ở việc tìm ra những lỗi *chưa ai nghĩ tới*. Một đội QA trưởng thành dùng cả hai, không chọn một bỏ một.

2.2. Testing Pyramid: Unit → Integration → E2E



Testing Pyramid là mô hình khuyến nghị về *tỉ lệ* giữa các cấp độ test: viết **nhiều** unit test (nhanh, rẻ, chạy trong mili-giây), **vừa phải** integration test, và **ít** E2E test (chậm, đắt, nhưng gần nhất với trải nghiệm người dùng thật). Lý do: một test E2E chạy qua cả trình duyệt thật có thể mất vài giây, trong khi một unit test chạy trong vài mili-giây — nếu suite test của bạn toàn E2E, chạy hàng nghìn test sẽ mất hàng giờ.

Unit test

Kiểm thử **một đơn vị nhỏ nhất có thể** (thường là một hàm/function), **cô lập hoàn toàn** khỏi phần còn lại của hệ thống — không gọi database thật, không gọi API thật, không cần trình duyệt.

```
// cart.js - hàm cần test
function applyDiscount(subtotal, discountPercent) {
  return subtotal - (subtotal * discountPercent) / 100;
}

// unit test: chỉ gọi MỘT hàm này, không liên quan gì tới hàm khác
assert.strictEqual(applyDiscount(200000, 10), 180000);
```

Integration test

Kiểm thử **nhều đơn vị phối hợp với nhau** — ví dụ hàm A gọi hàm B, hoặc code gọi vào một database/API thật (hay bản giả lập gần giống thật). Mục đích: bắt các lỗi chỉ xuất hiện khi các phần *ghép lại*, dù từng phần riêng lẻ đều đúng.

```
// calculateTotal() bên trong gọi cả calculateSubtotal + applyDiscount +
formatCurrencyVND
// integration test xác minh cả 3 hàm này ăn khớp khi phối hợp
assert.strictEqual(calculateTotal(items, 10), "360.000 đ");
```

→ Chạy thử cả hai loại test này với code thật ở `code/ch02-loai-kiem-thu/` (xem README trong đó để thấy ví dụ cụ thể một lỗi mà unit test **không** bắt được nhưng integration test bắt được).

E2E test (End-to-End)

Kiểm thử **toàn bộ luồng người dùng thật**, từ đầu đến cuối, thường qua giao diện thật (trình duyệt thật đối với web app) — gần nhất với trải nghiệm người dùng, nhưng cũng chậm nhất và dễ vỡ nhất (flaky) nếu viết không cẩn thận.

```
// Playwright E2E test - cú pháp thật, sẽ học chi tiết từ Chương 11.
// Đây là một luồng thật: mở trình duyệt, điền form, bấm nút, kiểm tra kết quả.
await page.goto("/login");
await page.getByLabel("Email").fill("demo@example.com");
await page.getByLabel("Mật khẩu").fill("Password123!");
await page.getByRole("button", { name: "Đăng nhập" }).click();
await expect(page).toHaveURL("/dashboard");
```

Đây chính xác là TC-01 trong bộ test case ở Chương 1 (`code/ch01-sdlc/test-case-list-login.md`) — được hiện thực hoá thành automation test thật. Toàn bộ Phần IV của sách này tập trung vào việc viết loại test này bằng Playwright.

2.3. Regression Test

Regression testing là việc chạy lại các test đã có (thường là một tập hợp lớn, gọi là **regression suite**) sau mỗi lần code thay đổi, để đảm bảo thay đổi mới **không làm hỏng** những gì đã hoạt động đúng trước đó.

Regression test không phải là một *loại* test riêng theo kiểu Unit/Integration/E2E — mà là một **mục đích sử dụng**: bất kỳ test nào (unit, integration, hay E2E) đều có thể là một phần của regression suite, miễn là nó được chạy lại lặp đi lặp lại.

Đây là điểm mà automation testing trở nên gần như bắt buộc: chạy tay lại 200 test case sau mỗi lần sửa code là không khả thi. Một regression suite tự động (chạy trong CI/CD — xem Chương 19) có thể chạy lại toàn bộ trong vài phút, mỗi khi có commit mới.

Bài tập

1. Chạy `node unit-test.js` và `node integration-test.js` trong `code/ch02-loai-kiem-thu/`, xác nhận cả hai đều pass.
2. Làm theo hướng dẫn "Vì sao có cả hai vẫn cần thiết?" trong README của code mẫu — cố tình gây lỗi trong `cart.js` và quan sát unit test lẫn integration test phản ứng khác nhau ra sao.
3. Với tính năng bạn đã chọn ở bài tập Chương 1, liệt kê: test nào nên là unit test, test nào nên là integration test, test nào bắt buộc phải là E2E (vì cần cả UI thật)?
4. Ước tính: nếu team bạn có 300 test case, chạy tay mất trung bình 3 phút/case, một lần regression đầy đủ mất bao lâu? So sánh với thời gian một suite automation tương đương có thể chạy song song.

Lỗi thường gặp

- **Viết toàn E2E test, không có unit/integration test**: suite chạy rất chậm, khó xác định chính xác lỗi nằm ở đâu khi test fail (vì E2E test chạm vào rất nhiều thứ cùng lúc).
- **Nhầm lẫn "test nhiều" với "test tốt"**: 100% coverage bằng unit test không đảm bảo hệ thống chạy đúng nếu không có integration/E2E test nào xác minh các phần ghép lại đúng.
- **Bỏ qua regression testing vì "tính năng cũ chắc vẫn chạy tốt"**: đây là nguyên nhân phổ biến nhất của bug production — một thay đổi tưởng chừng không liên quan làm hỏng tính năng khác.

Tóm tắt & tiếp theo

- Manual và Automation testing bổ trợ cho nhau, không thay thế hoàn toàn nhau.
- Testing Pyramid khuyến nghị: nhiều unit test, vừa phải integration test, ít E2E test.
- Regression testing là lý do chính khiến automation testing trở nên cần thiết trong Agile/DevOps.

Chương 3 sẽ đi vào các khái niệm nền tảng mà mọi QA cần thành thạo trước khi viết test đầu tiên: test case, test plan, và bug report.

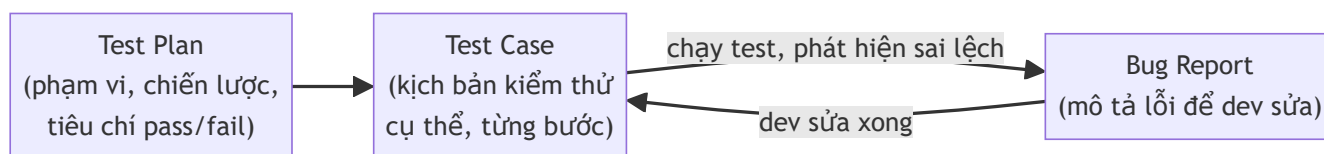
Chương 3: Test case, test plan, bug report — những khái niệm cơ bản mọi QA cần biết

Mục tiêu học

- Viết được một test case đầy đủ, rõ ràng, người khác đọc vào làm theo được ngay.
- Biết một test plan gồm những phần nào và dùng để làm gì.
- Viết được một bug report đủ thông tin để dev tái hiện và sửa lỗi mà không cần hỏi lại.
- Hiểu mối quan hệ giữa 3 tài liệu này trong quy trình QA thực tế.

Ví dụ: `code/ch03-test-case-plan-bug/` — test plan và bug report thật, viết cho cùng tính năng đăng nhập đã dùng ở Chương 1–2, để thấy rõ 3 tài liệu (test plan → test case → bug report) khớp với nhau như thế nào trong một luồng làm việc thực tế.

3.1. Mối quan hệ giữa Test Plan, Test Case, và Bug Report



- **Test Plan** trả lời câu hỏi "chúng ta sẽ test cái gì, test như thế nào, và khi nào coi là đủ?" — viết **một lần** cho cả tính năng/release.
- **Test Case** là **từng kịch bản cụ thể** để thực thi việc test đó — một test plan sinh ra nhiều test case (xem `code/ch01-sdlc/test-case-list-login.md`: 10 test case cho 1 test plan).
- **Bug Report** được viết ra **khi một test case fail** — ghi lại đủ thông tin để dev tái hiện và sửa lỗi.

3.2. Test Case — giải phẫu một test case tốt

Một test case tốt có các thành phần sau (đã áp dụng ở Chương 1):

Thành phần	Vai trò	Ví dụ
ID	Định danh duy nhất, để trace ngược khi có bug	TC-01
Tiêu đề	Mô tả ngắn gọn mục đích của test	"Đăng nhập thành công với tài khoản hợp lệ"
Tiền điều kiện (Precondition)	Trạng thái hệ thống phải có <i>trước khi</i> thực hiện test	"Ở trang <code>/login</code> "
Các bước (Steps)	Thao tác cụ thể, theo đúng thứ tự, đủ rõ để người khác làm theo	"1. Nhập email...\n2. Bấm..."
Dữ liệu vào (Test Data)	Giá trị cụ thể dùng để test — không viết chung chung	email: <code>demo@example.com</code>
Kết quả mong đợi (Expected Result)	Kết quả chính xác, đo lường được — không viết mơ hồ	"Chuyển sang <code>/dashboard</code> " (không viết "hoạt động đúng")

Nguyên tắc quan trọng nhất: kết quả mong đợi phải đủ cụ thể để hai người khác nhau đọc vào và đưa ra cùng một kết luận Pass/Fail. So sánh:

- ❌ Mơ hồ: "Đăng nhập hoạt động tốt."
- ✅ Cụ thể: "Chuyển sang trang `/dashboard`, hiển thị tên người dùng ở góc phải."

Đây cũng chính là nền tảng để viết automation test sau này — một `expect()` trong Playwright (Chương 11) *chính là* một "kết quả mong đợi" được viết thành code, nên nếu test case mơ hồ, bạn sẽ không biết viết `expect()` gì.

3.3. Test Plan — bức tranh lớn hơn một test case

Một test case chỉ nói về *một kịch bản*. Test plan trả lời những câu hỏi rộng hơn, ở cấp độ cả tính năng hoặc cả release — xem ví dụ đầy đủ ở `code/ch03-test-case-plan-bug/test-plan-login.md`, gồm các phần chính:

- Mục tiêu** — vì sao cần test cái này.
- Phạm vi (Scope)** — cái gì *nằm trong* và *nằm ngoài* phạm vi test lần này. Đây là phần hay bị bỏ qua nhất nhưng quan trọng nhất: không ghi rõ "ngoài phạm vi", QA dễ tốn thời gian test những thứ không ai yêu cầu, hoặc bị hỏi "sao không test cái X" khi X chưa từng được thống nhất nằm trong phạm vi.
- Chiến lược kiểm thử** — cấp độ nào (unit/integration/E2E — xem Chương 2) do ai làm, bằng công cụ gì.
- Môi trường kiểm thử** — test ở đâu, dữ liệu test nào, trình duyệt nào.
- Tiêu chí Pass/Fail (Entry/Exit Criteria)** — điều kiện để *bắt đầu* test, và điều kiện để coi là *test xong, đủ điều kiện release*.
- Rủi ro** — những gì có thể cản trở việc test (ví dụ: một case khó test tự động vì phụ thuộc thời gian thực — xem mục Rủi ro trong ví dụ).

7. Lịch trình — mốc thời gian cho từng giai đoạn test.

3.4. Bug Report — viết sao để dev không cần hỏi lại

Một bug report thiếu thông tin khiến dev phải hỏi lại QA nhiều lần ("làm sao để tái hiện?", "test trên môi trường nào?"), làm chậm cả quá trình. Xem ví dụ đầy đủ ở `code/ch03-test-case-plan-bug/bug-report-example.md` — lưu ý các phần bắt buộc:

- **Steps to Reproduce** — các bước *chính xác*, đủ chi tiết để dev tự tái hiện được lỗi trên máy của họ, không bỏ sót bước nào.
- **Expected Result vs Actual Result** — viết tách biệt rõ ràng, không gộp chung.
- **Severity** (mức độ nghiêm trọng về mặt kỹ thuật/ng nghiệp vụ) khác với **Priority** (mức độ ưu tiên sửa) — một bug có thể Severity thấp (lỗi chính tả) nhưng Priority cao (xuất hiện ngay trang chủ, ảnh hưởng thương hiệu), hoặc ngược lại.
- **Log/Screenshot** — bằng chứng khách quan, không chỉ mô tả bằng lời.
- **Môi trường** — trình duyệt, build number, dữ liệu test cụ thể đã dùng — thiếu phần này là nguyên nhân phổ biến nhất khiến dev không tái hiện được lỗi ("trên máy tôi vẫn chạy bình thường").

Liên hệ với automation test: khi bug này được dev sửa xong, TC-07 sẽ được tự động hoá bằng Playwright (Chương 22, phần seed dữ liệu tài khoản đã sai N lần) và chạy lại mỗi lần có commit mới — để bug BUG-142 **không bao giờ tái xuất hiện mà không ai biết** (đây chính là quy trình regression testing đã học ở Chương 2).

Bài tập

1. Đọc `code/ch03-test-case-plan-bug/bug-report-example.md`. Giả sử bạn là dev, chỉ dựa vào bug report đó (không hỏi lại QA), bạn có tái hiện được lỗi không? Còn thiếu thông tin gì không?
2. Viết một bug report tương tự cho một lỗi bạn từng gặp (trong dự án cá nhân hoặc công việc), đảm bảo đủ 7 phần như ví dụ.
3. Với tính năng bạn chọn từ Chương 1, viết một test plan ngắn gọn (không cần đủ 7 phần, chỉ cần Mục tiêu + Phạm vi + Tiêu chí Pass/Fail).
4. Chọn 2 test case bất kỳ trong `code/ch01-sdlc/test-case-list-login.md`, đánh giá xem "Kết quả mong đợi" của chúng đã đủ cụ thể (đo lường được) chưa — nếu chưa, viết lại cho rõ hơn.

Lỗi thường gặp

- **Viết "Kết quả mong đợi" mơ hồ** (ví dụ "hoạt động đúng", "hiển thị hợp lý"): không thể quyết định Pass/Fail một cách khách quan, mỗi người đọc hiểu một kiểu.
- **Bug report thiếu bước tái hiện chi tiết:** khiến dev không tái hiện được, phải hỏi lại nhiều lần, làm chậm chu kỳ sửa lỗi — đối lập trực tiếp với tinh thần "shift-left" đã học ở Chương 1.

- **Không ghi rõ "ngoài phạm vi" trong test plan:** dẫn đến tranh cãi "tại sao không test cái này" sau khi đã release, dù chưa từng ai yêu cầu test nó.
 - **Nhầm Severity với Priority:** dẫn đến sắp xếp thứ tự sửa lỗi sai — một lỗi rất nghiêm trọng về kỹ thuật nhưng ít người dùng chạm tới có thể không cần sửa gấp bằng một lỗi nhỏ nhưng ai cũng thấy ngay khi mở app.
-

Tóm tắt & tiếp theo

- Test Plan (bức tranh lớn) → Test Case (kịch bản cụ thể) → Bug Report (khi test case fail) là ba tài liệu nền tảng, liên kết chặt chẽ với nhau trong quy trình QA.
- Một test case tốt có kết quả mong đợi cụ thể, đo lường được — đây cũng là nền tảng trực tiếp để viết `expect()` trong automation test sau này.
- Một bug report tốt giúp dev tái hiện và sửa lỗi mà không cần hỏi lại, rút ngắn chu kỳ sửa lỗi.

Đến đây, Phần I (Nền tảng QA trong SDLC) đã hoàn thành. Chương 4 mở đầu Phần II, đi vào các vai trò QA cụ thể trong một tổ chức: PQA, FQA, TQA, và vị trí của Automation QA giữa các vai trò đó.

Phần II — Vai trò QA & Automation QA

Chương 4: PQA, FQA, TQA — phân biệt & trách nhiệm

Mục tiêu học

- Phân biệt rõ 3 vai trò QA phổ biến trong ngành: PQA, FQA, TQA.
- Xác định được, với một hoạt động QA cụ thể, nó thuộc trách nhiệm của vai trò nào.
- Hiểu vì sao ở nhiều công ty (đặc biệt công ty nhỏ/vừa) một người thường kiêm nhiều vai trò — và đây chính là bối cảnh ra đời của "Automation QA Engineer" (Chương 5).

Ví dụ: `code/ch04-pqa-fqa-tqa/responsibility-matrix.md` — ma trận trách nhiệm RACI áp dụng 3 vai trò vào từng hoạt động cụ thể của tính năng đăng nhập.

4.1. Ba vai trò, ba trọng tâm khác nhau

Ba thuật ngữ PQA, FQA, TQA không phải lúc nào cũng dùng thống nhất giữa các công ty (có nơi gọi khác đi), nhưng trọng tâm công việc phía sau các thuật ngữ này khá nhất quán trong ngành:

PQA — Product QA

Trọng tâm: **sản phẩm có đúng là thứ người dùng cần không?** PQA quan tâm đến góc nhìn nghiệp vụ và trải nghiệm người dùng nhiều hơn là chi tiết kỹ thuật.

- Review yêu cầu (requirement) từ góc độ "điều này có hợp lý với người dùng thật không?"
- Review UX/UI, luồng sử dụng (user flow) có trực quan, hợp lý không.
- Thường tham gia sớm nhất trong SDLC (giai đoạn Requirement — xem lại Chương 1).

FQA — Functional QA

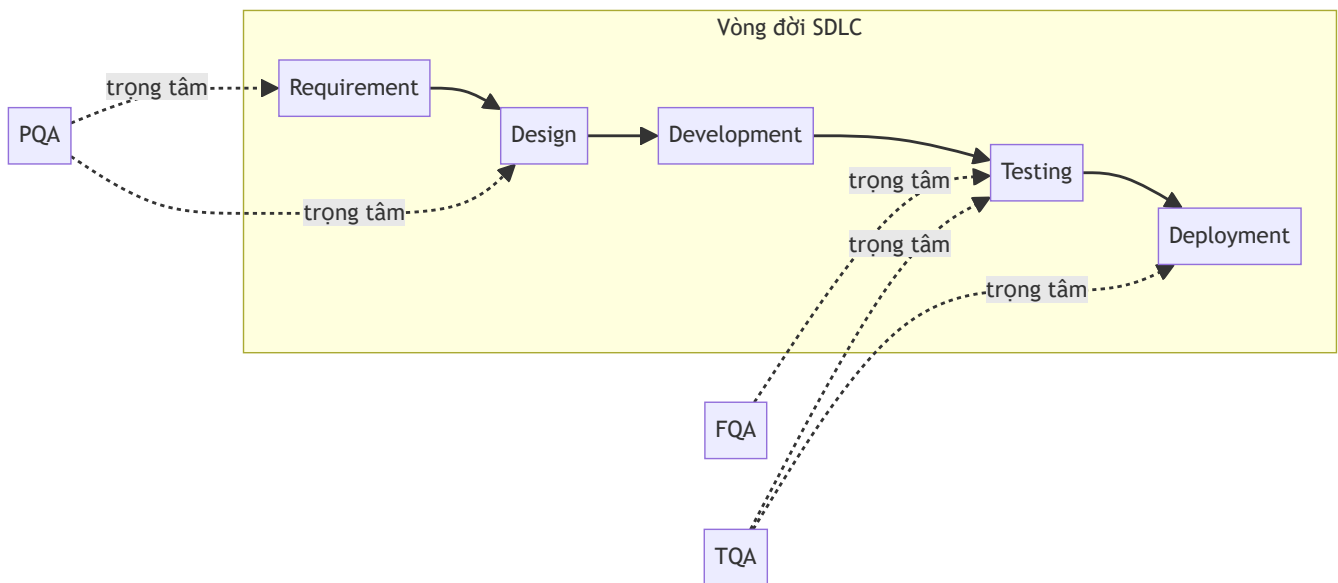
Trọng tâm: **tính năng có hoạt động đúng theo yêu cầu đã định nghĩa không?** Đây là hình dung phổ biến nhất khi nói "QA" — người viết và thực thi test case.

- Viết test case chi tiết (như bộ test case Chương 1).
- Test tay (manual) hoặc yêu cầu automation test cho từng case.
- Report bug khi phát hiện sai lệch (xem Chương 3).
- Trọng tâm chủ yếu ở giai đoạn Testing trong SDLC.

TQA — Test/Technical QA

Trọng tâm: **hạ tầng và công cụ kiểm thử.** TQA là người viết code test (automation), xây dựng và duy trì framework test, tích hợp test vào CI/CD.

- Viết automation test (Playwright, và các công cụ khác).
- Dựng framework test (Page Object Model — Chương 15, fixtures — Chương 16).
- Tích hợp test vào pipeline CI/CD (Chương 19).
- Phân tích test tự động fail: lỗi thật hay flaky test (test không ổn định)?



4.2. Ví dụ cụ thể: cùng một tính năng, ba góc nhìn khác nhau

Với tính năng "Đăng nhập" (đã dùng xuyên suốt Phần I):

- **PQA** hỏi: "Thông báo lỗi 'Email hoặc mật khẩu không đúng' có đủ thân thiện không, hay khiến người dùng bối rối? Có nên thêm link 'Quên mật khẩu' ngay cạnh đó không?"
- **FQA** hỏi: "Nếu tôi nhập sai mật khẩu, hệ thống có hiển thị đúng thông báo lỗi đó không? Nếu tôi để trống field, có bị chặn submit không?" — rồi thực thi bộ test case Chương 1 để trả lời.
- **TQA** hỏi: "Làm sao để 10 test case này chạy tự động mỗi khi có commit mới, và nếu một test fail trên CI, làm sao biết ngay đó là lỗi thật hay chỉ vì trang load chậm hôm nay (flaky)?"

Xem ma trận đầy đủ ở `code/ch04-pqa-fqa-tqa/responsibility-matrix.md` để thấy 8 hoạt động cụ thể được phân định rõ giữa 3 vai trò này.

4.3. Vì sao một người thường kiêm nhiều vai trò?

Ở các công ty lớn, ba vai trò này có thể là ba vị trí (hoặc ba nhóm) riêng biệt. Nhưng ở phần lớn công ty vừa và nhỏ — bối cảnh phổ biến nhất mà người đọc sách này sẽ làm việc — **một người thường đảm nhiệm cả FQA lẫn TQA**: vừa viết test case, vừa tự động hoá chính test case đó bằng Playwright.

Đây chính là chân dung công việc mà cuốn sách này nhắm tới: một **Automation QA Engineer** — người hiểu đủ về nghiệp vụ để viết test case đúng (như FQA), và đủ kỹ năng lập trình để tự động hoá chúng (như TQA). Chương 5 sẽ định nghĩa rõ vai trò này.

Bài tập

1. Với bộ test case ở Chương 1 (`code/ch01-sdlc/test-case-list-login.md`), chọn 3 test case bất kỳ và xác định: viết test case này thuộc trách nhiệm FQA, thực thi bằng automation thuộc TQA — ai review UX của thông báo lỗi ở mỗi case đó (PQA)?
 2. Nếu công ty bạn (hoặc công ty bạn biết) chỉ có 1-2 người làm QA cho cả team, hãy phác thảo: người đó cần dành bao nhiêu % thời gian cho công việc kiểu PQA, FQA, TQA?
 3. Tìm một tin tuyển dụng "QA Engineer" hoặc "Automation Tester" thực tế (trên LinkedIn/ TopCV...), đối chiếu mô tả công việc với 3 vai trò ở chương này — vị trí đó nghiêng về vai trò nào nhất?
-

Lỗi thường gặp

- **Coi PQA/FQA/TQA là 3 chức danh cố định, bắt buộc phải tách biệt:** trên thực tế đây là 3 *loại trọng tâm công việc*, không phải chức danh — một người, một team nhỏ hoàn toàn có thể đảm nhiệm nhiều vai trò cùng lúc.
 - **Nghĩ rằng TQA chỉ cần biết code, không cần hiểu nghiệp vụ:** một automation test viết đúng cú pháp nhưng test sai nghiệp vụ (do không hiểu rõ yêu cầu) vẫn là test vô giá trị — TQA giỏi vẫn cần tư duy như FQA khi thiết kế test case.
-

Tóm tắt & tiếp theo

- PQA tập trung vào "đúng sản phẩm" (góc nhìn người dùng/nghiệp vụ).
- FQA tập trung vào "đúng tính năng theo yêu cầu" (viết & thực thi test case).
- TQA tập trung vào "hạ tầng kiểm thử" (automation, CI/CD).
- Automation QA Engineer — trọng tâm của sách này — thường là sự kết hợp giữa FQA và TQA.

Chương 5 định nghĩa cụ thể "Automation QA" là gì, và quan trọng hơn: khi nào nên tự động hoá một test case, khi nào không nên.

Chương 5: Automation QA là gì? Khi nào nên tự động hoá, khi nào không

Mục tiêu học

- Định nghĩa được Automation QA là gì, khác Manual QA ở điểm nào (không chỉ là "biết code").
- Áp dụng được một khung quyết định (decision framework) để trả lời: test case này có nên tự động hoá không?
- Tính toán được điểm hoà vốn (break-even point) giữa chi phí manual và automation.

Code mẫu: `code/ch05-automation-qa-la-gi/roi-calculator.js` — công cụ tính ROI (return on investment) của việc tự động hoá một test case, dựa trên số lần dự kiến sẽ chạy lại nó.

5.1. Automation QA là gì?

Automation QA là hoạt động (và người thực hiện hoạt động đó) viết **code** để tự động thực thi lại các kịch bản kiểm thử, thay vì con người thao tác tay từng bước.

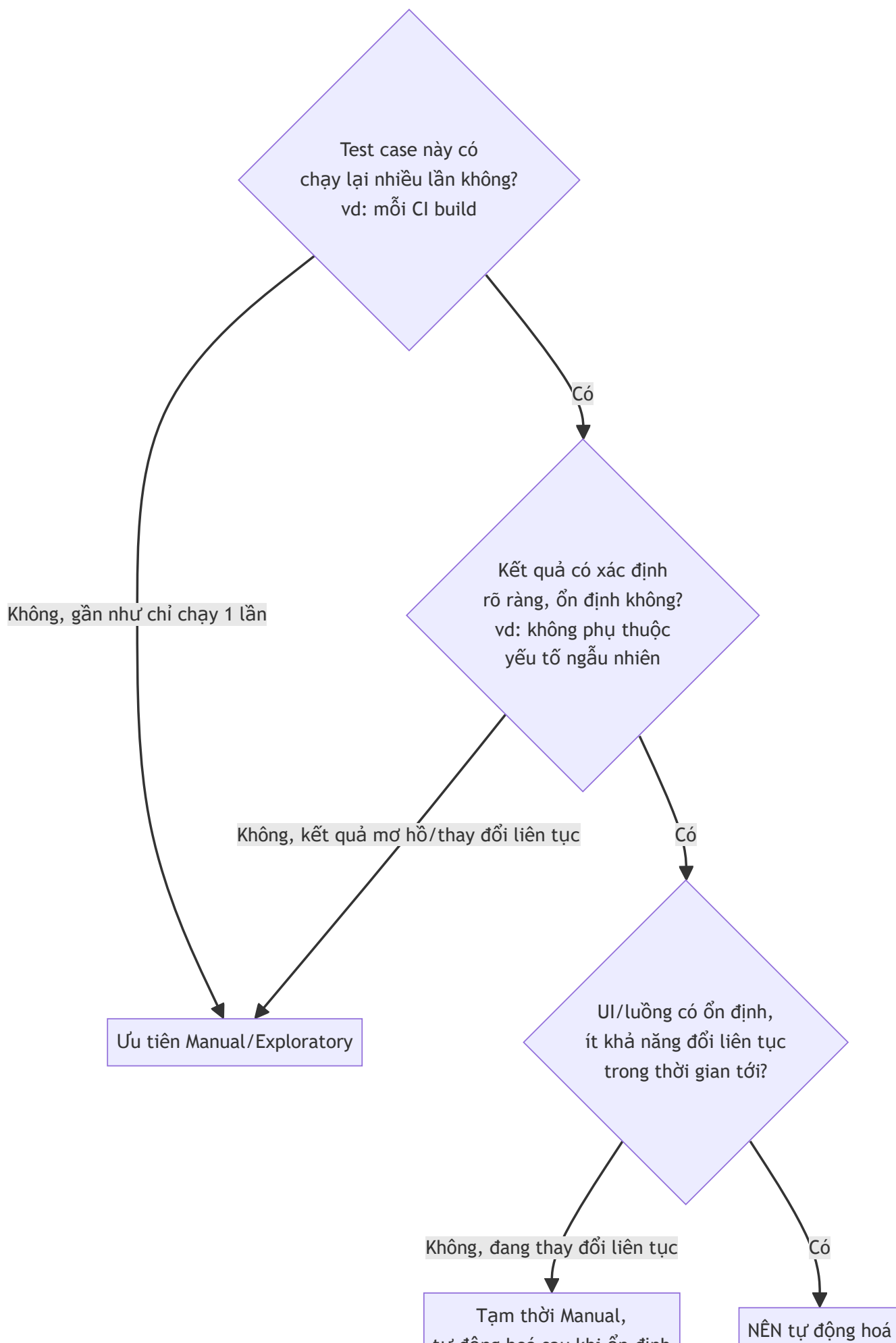
Điều quan trọng cần làm rõ ngay: **Automation QA không phải là "QA nhưng biết code"**. Một Automation QA Engineer giỏi cần **cả hai** năng lực:

1. **Tư duy QA** (như FQA ở Chương 4): biết thiết kế test case tốt, biết nghĩ ra case biên, hiểu nghiệp vụ đủ sâu để biết test cái gì là quan trọng.
2. **Năng lực kỹ thuật** (như TQA ở Chương 4): viết code test có cấu trúc, bảo trì được, hiểu cách hệ thống hoạt động đủ để debug khi test fail.

Thiếu năng lực (1), bạn viết ra automation test đúng cú pháp nhưng test sai thứ cần test. Thiếu năng lực (2), bạn có ý tưởng test tốt nhưng không tự động hoá được, hoặc tự động hoá ra một suite test không ai bảo trì nổi.

5.2. Không phải mọi thứ đều nên tự động hoá

Đây là sai lầm phổ biến nhất của người mới học automation test: nghĩ rằng "tự động hoá càng nhiều càng tốt". Thực tế, viết và bảo trì một automation test **tốn chi phí** (thời gian viết, thời gian bảo trì khi UI/ng nghiệp vụ thay đổi, thời gian debug khi test flaky). Chi phí đó chỉ đáng bỏ ra nếu test case đó **được chạy lại đủ nhiều lần**.



Ví dụ nên tự động hoá

- Test case chạy trong **mọi lần build CI/CD** (regression suite) — như TC-01 đến TC-06 trong bộ test case Chương 1.
- Test case có kết quả **rõ ràng, xác định** (không mơ hồ kiểu "giao diện trông đẹp không").
- Luồng nghiệp vụ **ổn định**, không thay đổi liên tục mỗi sprint.

Ví dụ chưa nên tự động hoá (hoặc automation không phải lựa chọn tốt nhất)

- Test **một lần duy nhất** cho một tính năng sắp bị gỡ bỏ tuần sau.
- **Exploratory testing** — bản chất là khám phá cái *chưa biết trước*, không có kịch bản cố định để tự động hoá.
- Đánh giá **UX/thẩm mỹ chủ quan** ("nút này có nổi bật không?") — máy không đánh giá được, cần con người.
- Tính năng **UI còn đang thay đổi liên tục** mỗi sprint — automation test sẽ vỡ liên tục, chi phí bảo trì vượt xa lợi ích, tốt hơn nên đợi UI ổn định.

5.3. Tính ROI: khi nào automation "hoà vốn"?

Câu hỏi định lượng hơn: viết automation test tốn thời gian ban đầu (setup cost) — vậy sau bao nhiêu lần chạy lại thì khoản đầu tư đó "hoà vốn" so với việc cứ tiếp tục test tay?

Chạy `code/ch05-automation-qa-la-gi/roi-calculator.js` để thấy so sánh cụ thể giữa hai case đối lập: một test case chạy trong mọi CI build (nên tự động hoá, hoà vốn rất nhanh) so với một luồng nghiệp vụ hiếm gặp (chưa nên tự động hoá, vì gần như không có cơ hội "thu hồi" chi phí viết test).

Công thức cốt lõi (đơn giản hoá):

```
Tổng chi phí manual      = thời_gian_test_tay × số_lần_chạy_lại
Tổng chi phí automation = thời_gian_viết_test + (thời_gian_chạy_tự_động +
thời_gian_bảo_trì) × số_lần_chạy_lại
```

Automation đáng giá khi **Tổng chi phí automation < Tổng chi phí manual** — và vì

`thời_gian_viết_test` là chi phí cố định một lần, số lần chạy lại càng nhiều, automation càng có lợi (đường automation "phẳng" hơn nhiều so với đường manual tăng tuyến tính).

Bài tập

1. Chạy `node roi-calculator.js`, thử đổi `numberOfRuns` trong case 2 (luồng hiếm gặp) tăng dần — tìm ra khoảng giá trị mà tại đó automation bắt đầu đáng giá hơn manual.

2. Với 10 test case ở `code/ch01-sdlc/test-case-list-login.md`, áp dụng sơ đồ quyết định ở mục 5.2: test case nào bạn sẽ tự động hoá trước tiên, test case nào để lại test tay (nếu có)?
3. Viết một case ROI cho một tính năng bạn từng làm, dùng `roi-calculator.js` làm mẫu, tự ước lượng các tham số và kết luận có nên tự động hoá không.

Lỗi thường gặp

- **"Tự động hoá 100% test case" như một mục tiêu tự thân:** không phải mọi test đều đáng tự động hoá — xem mục 5.2. Coverage automation cao không đồng nghĩa với chất lượng test tốt.
- **Tự động hoá quá sớm khi UI còn đang thay đổi liên tục:** dẫn đến việc sửa lại automation test gần như mỗi sprint — tốn công hơn cả lợi ích mang lại, cho tới khi tính năng ổn định.
- **Bỏ qua chi phí bảo trì khi tính ROI:** chỉ tính thời gian viết test lần đầu mà quên chi phí sửa test mỗi khi UI đổi — khiến ROI thực tế thấp hơn nhiều so với ước tính ban đầu.

Tóm tắt & tiếp theo

- Automation QA đòi hỏi cả tư duy QA (biết test gì) lẫn năng lực kỹ thuật (biết tự động hoá thế nào).
- Không phải mọi test case đều nên tự động hoá — cần cân nhắc: tần suất chạy lại, độ ổn định kết quả, độ ổn định của UI/luồng nghiệp vụ.
- ROI của automation tăng theo số lần chạy lại — vì chi phí viết test là chi phí cố định một lần.

Chương 6 khép lại Phần II bằng việc đi sâu vào kỹ năng và tư duy cụ thể mà một Automation QA Engineer cần rèn luyện — chuẩn bị nền tảng trước khi bước vào cài đặt môi trường và viết Playwright thật ở Phần III–IV.

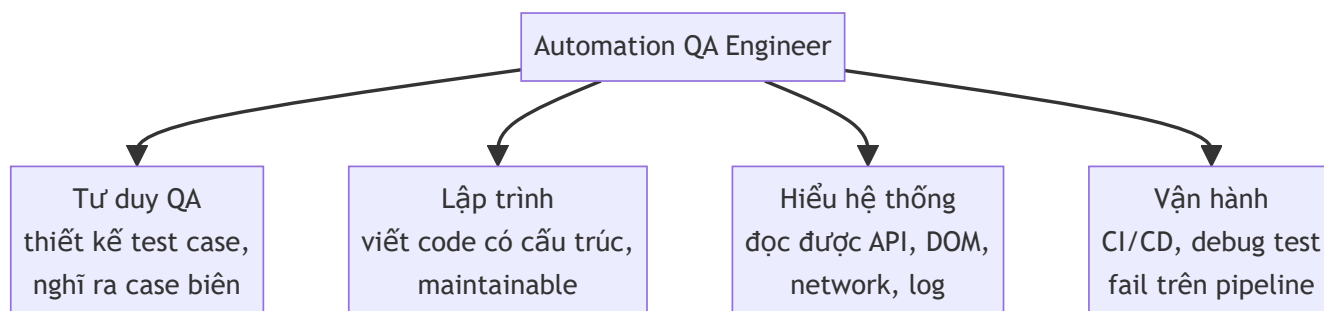
Chương 6: Kỹ năng & tư duy cần có của một Automation QA Engineer

Mục tiêu học

- Liệt kê được các nhóm kỹ năng cốt lõi của một Automation QA Engineer.
- Nhận diện được các "code smell" phổ biến trong automation test qua ví dụ thực tế.
- Hiểu vì sao "biết code" chỉ là điều kiện cần, chưa phải điều kiện đủ.

Code mẫu: `code/ch06-ky-nang-automation-qa/` — hai file test cùng một kịch bản, viết theo hai tư duy khác nhau (`bad-example.spec.ts` vs `good-example.spec.ts`), với 5 vấn đề được đánh số đối chiếu trực tiếp.

6.1. Bốn nhóm kỹ năng cốt lõi



1. **Tư duy QA** (đã học ở Chương 1-4): thiết kế test case tốt, biết ưu tiên test gì trước, nghĩ ra case biên mà người viết code (dev) hay bỏ sót.
2. **Lập trình**: không cần giỏi thuật toán, nhưng cần viết code *rõ ràng, có cấu trúc, dễ bảo trì* — vì automation test cũng là code, và code test tệ cũng gây nợ kỹ thuật (technical debt) như code sản phẩm tệ.
3. **Hiểu hệ thống đang test**: đọc hiểu response API, cấu trúc DOM, network request — để biết *tại sao* một test fail (lỗi thật hay do cách viết test).
4. **Vận hành pipeline**: hiểu CI/CD đủ để tích hợp test vào, và quan trọng hơn — đủ để **debug** khi test fail trên CI nhưng lại pass trên máy local.

6.2. "Biết code" chỉ là điều kiện cần

So sánh `code/ch06-ky-nang-automation-qa/bad-example.spec.ts` và `good-example.spec.ts` — cả hai đều là code Playwright *chạy được về mặt cú pháp*, nhưng thể hiện hai trình độ tư duy automation rất khác nhau:

#	Vấn đề (bad-example)	Cách sửa (good-example)	Kỹ năng liên quan
1	<code>waitForTimeout()</code> — chờ mù, đoán thời gian	<code>expect()</code> tự động chờ đến khi đúng điều kiện	Hiểu hệ thống (auto-waiting)
2	Locator theo cấu trúc DOM dễ vỡ	Locator theo vai trò/nhãn người dùng thấy	Lập trình (viết code bền vững)
3	Assertion mơ hồ, không khớp test case	Assertion cụ thể, đúng "Kết quả mong đợi"	Tư duy QA (nhớ lại Chương 3)
4	Lặp lại logic ở mọi test	Tách hàm dùng chung	Lập trình (không lặp code — DRY)
5	Test phụ thuộc thứ tự chạy	Mỗi test độc lập	Vận hành (test ổn định trên CI)

Quan sát quan trọng: **Vấn đề 3** (assertion mơ hồ) không phải lỗi kỹ thuật — code đó vẫn chạy, không báo lỗi cú pháp. Đây là lỗi **tư duy QA**: người viết quên rằng assertion phải khớp chính xác với "Kết quả mong đợi" đã định nghĩa trong test case (Chương 3). Một lập trình viên giỏi nhưng chưa quen tư duy QA rất dễ mắc lỗi này.

6.3. Vì sao code test tệ cũng nguy hiểm như code sản phẩm tệ?

Automation test là **code**, và cũng cần được đối xử như code sản phẩm: dễ đọc, dễ bảo trì, không lặp lại. Một suite test viết tệ (như `bad-example.spec.ts`) gây ra hậu quả cụ thể:

- **Flaky test** (Vấn đề 1, 5): test đôi khi pass đôi khi fail dù code không đổi gì — dần dần đội ngũ mất niềm tin vào automation suite, tới mức bỏ qua khi nó báo fail ("chắc lại flaky thôi").
- **Vỡ hàng loạt khi UI đổi nhỏ** (Vấn đề 2): dev chỉ đổi class CSS, hàng chục test vỡ dù hành vi ứng dụng không đổi — tốn thời gian sửa test thay vì phát hiện lỗi thật.
- **Bảo trì cực khổ** (Vấn đề 4): sửa một domain/URL phải sửa ở hàng chục nơi, dễ sót.

Bài tập

1. Đọc kỹ cả hai file trong `code/ch06-ky-nang-automation-qa/`, với mỗi "Vấn đề N", tự giải thích bằng lời của bạn (không nhìn comment) tại sao nó là vấn đề.

2. Tìm trong bad-example.spec.ts: nếu dev đổi `<button class="btn btn-primary">` thành `<button class="btn-submit">` (đổi tên class, hành vi không đổi), locator nào ở bad-example sẽ vỡ? Locator tương ứng ở good-example có bị ảnh hưởng không? Vì sao?
 3. Tự đánh giá bản thân theo 4 nhóm kỹ năng ở mục 6.1 (thang điểm 1-5) — nhóm nào bạn cần rèn luyện thêm nhất trước khi bước vào Phần III–IV?
-

Lỗi thường gặp

- **Học cú pháp Playwright trước, tư duy QA sau (hoặc bỏ qua):** dẫn đến viết được test chạy được nhưng test sai thứ cần test — như Vấn đề 3 ở mục 6.2.
 - **Copy-paste code test thay vì tách hàm dùng chung:** một thay đổi nhỏ (đổi URL, đổi selector) phải sửa ở rất nhiều nơi, dễ sót và tạo ra automation suite khó bảo trì theo thời gian.
 - **Phớt lờ flaky test thay vì tìm nguyên nhân gốc:** chạy lại (retry) cho tới khi pass là cách xử lý tạm bợ phổ biến nhưng nguy hiểm — che giấu vấn đề thật (thường là Vấn đề 1 hoặc 5 ở trên) thay vì sửa nó.
-

Tóm tắt & tiếp theo

- Automation QA Engineer cần 4 nhóm kỹ năng: tư duy QA, lập trình, hiểu hệ thống, vận hành pipeline.
- "Biết code" là điều kiện cần nhưng không đủ — code test đúng cú pháp vẫn có thể sai về tư duy QA.
- Code test cũng là code sản phẩm — cần viết rõ ràng, bền vững, không lặp lại.

Đến đây, Phần II (Vai trò QA & Automation QA) đã hoàn thành. Phần III bắt đầu phần thực hành: cài đặt Node.js, khởi tạo project Playwright, và giới thiệu demo app sẽ dùng xuyên suốt phần còn lại của sách — nơi hai file ví dụ ở chương này sẽ được chạy thật lần đầu tiên.

Phần III — Cài đặt môi trường

Chương 7: Cài đặt Node.js, quản lý package, VS Code & extension hữu ích

Mục tiêu học

- Cài đặt Node.js (bản LTS) và xác minh cài đặt thành công.
- Hiểu vai trò của npm, và vì sao sách này chọn npm làm trình quản lý package xuyên suốt.
- Cài đặt VS Code và các extension giúp việc viết Playwright test hiệu quả hơn.
- Cấu hình tối thiểu (Prettier, EditorConfig) để code nhất quán ngay từ đầu.

Chương này thuần cài đặt công cụ, không có code mẫu riêng — nhưng kết quả cài đặt ở đây là điều kiện bắt buộc để Chương 8 (khởi tạo project Playwright) chạy được.

7.1. Cài đặt Node.js

Playwright Test (framework chính sách này dùng) chạy trên Node.js. Cài bản **LTS** (Long Term Support) — ổn định, đủ mới cho Playwright, không dùng bản "Current"/thử nghiệm.

Windows:

```
winget install OpenJS.NodeJS.LTS
```

(Hoặc tải installer `.msi` từ trang chủ Node.js, chọn bản LTS, cài như phần mềm thông thường.)

macOS (dùng Homebrew):

```
brew install node@22
```

Linux (Ubuntu/Debian, qua NodeSource):

```
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -  
sudo apt install -y nodejs
```

Xác minh cài đặt

Mở terminal mới (bắt buộc mở **mới** để nhận PATH vừa cập nhật) và chạy:

```
node -v    # ví dụ: v22.14.0
npm -v     # ví dụ: 10.9.2
```

Nếu lệnh báo "command not found" / "không được nhận dạng", xem mục Lỗi thường gặp bên dưới.

7.2. npm — trình quản lý package

npm (Node Package Manager) đi kèm sẵn khi cài Node.js, dùng để cài và quản lý các thư viện (package) của project — bao gồm chính Playwright.

Có các lựa chọn thay thế phổ biến (**pnpm**, **yarn**) nhanh hơn hoặc tiết kiệm dung lượng đĩa hơn npm trong một số trường hợp, nhưng **sách này dùng npm xuyên suốt** vì đây là lựa chọn mặc định, không cần cài thêm gì, và là lựa chọn phổ biến nhất trong tài liệu chính thức của Playwright — phù hợp nhất với người mới để tránh phải học thêm công cụ ngoài phạm vi sách.

Hai khái niệm cần nắm trước khi qua Chương 8:

Lệnh	Ý nghĩa
<code>npm init</code>	Khởi tạo file <code>package.json</code> (khai báo thông tin & dependency của project)
<code>npm install <package></code>	Cài một package, ghi vào <code>package.json</code> (dependency)
<code>npm install -D <package></code>	Cài package chỉ dùng khi phát triển (devDependency) — Playwright thuộc nhóm này
<code>npm run <script></code>	Chạy một lệnh đã khai báo trong phần <code>"scripts"</code> của <code>package.json</code>

7.3. VS Code & extension hữu ích

VS Code (Visual Studio Code) là trình soạn thảo phổ biến nhất cho JS/TS, miễn phí, có hệ sinh thái extension mạnh — tải tại trang chủ VS Code, cài như phần mềm thông thường.

Extension nên cài ngay từ đầu (tìm trong tab Extensions, `Ctrl+Shift+X`):

Extension	Vai trò
Playwright Test for VSCode (của Microsoft)	Chạy/debug từng test ngay trong VS Code, xem Trace Viewer tích hợp (dùng nhiều từ Chương 17)
ESLint	Bắt lỗi cú pháp/logic JS-TS tiềm ẩn ngay khi gõ code
Prettier - Code formatter	Tự động format code theo một chuẩn thống nhất, tránh tranh cãi style trong team
Error Lens	Hiển thị lỗi/cảnh báo ngay trên dòng code thay vì phải mở tab Problems

7.4. Cấu hình tối thiểu cho project

Hai file cấu hình nên có trong mọi project từ đầu — không bắt buộc để chạy Playwright, nhưng giúp code nhất quán khi làm việc nhóm:

`.editorconfig` (thống nhất indent, line ending giữa các editor khác nhau trong team):

```
root = true

[*]
indent_style = space
indent_size = 2
end_of_line = lf
charset = utf-8
trim_trailing_whitespace = true
insert_final_newline = true
```

`.prettierrc` (cấu hình format code, cài package `prettier` qua `npm install -D prettier`):

```
{
  "semi": true,
  "singleQuote": false,
  "printWidth": 100
}
```

Hai file này sẽ được đặt sẵn trong project Playwright khởi tạo ở Chương 8.

Bài tập

1. Cài Node.js bản LTS theo hướng dẫn phù hợp với hệ điều hành của bạn, xác minh bằng `node -v` và `npm -v`.
2. Cài VS Code và 4 extension liệt kê ở mục 7.3.
3. Tạo một thư mục trống, chạy `npm init -y`, mở file `package.json` vừa sinh ra, giải thích từng trường (`name`, `version`, `main`, `scripts`) bằng lời của bạn.
4. Cài `prettier` vào project vừa tạo (`npm install -D prettier`), tạo file `.prettierrc` như mục 7.4, tạo một file `test.js` cố tình format lộn xộn, chạy `npx prettier --write test.js` và quan sát kết quả.

Lỗi thường gặp

- `node` **không được nhận dạng sau khi cài (Windows)**: thường do chưa mở terminal mới sau khi cài (PATH chỉ cập nhật cho phiên terminal mới) — đóng hẳn terminal cũ, mở lại.
- `npm install -g` **báo lỗi permission (macOS/Linux)**: không nên dùng `sudo npm install -g` (có thể gây lỗi quyền truy cập khó sửa về sau) — cấu hình npm dùng thư mục global riêng cho user hiện tại, hoặc dùng trình quản lý phiên bản Node như `nvm`.
- **Nhiều phiên bản Node cài chồng chéo, không rõ đang dùng bản nào**: chạy `node -v` để kiểm tra bản đang active; nếu cần nhiều phiên bản Node cho nhiều project khác nhau, cần nhắc dùng `nvm` (Linux/macOS) hoặc `nvm-windows`.
- **Quên mở lại VS Code sau khi cài Node**: VS Code cần khởi động lại để terminal tích hợp của nó nhận được PATH mới.

Tóm tắt & tiếp theo

- Node.js (bản LTS) + npm là nền tảng bắt buộc để chạy Playwright.
- VS Code kèm extension "Playwright Test for VSCode" giúp chạy/debug test trực tiếp trong editor.
- `.editorconfig` + `.prettierrc` giúp code nhất quán ngay từ project đầu tiên.

Chương 8 sẽ dùng chính những công cụ vừa cài đặt để khởi tạo project Playwright thật đầu tiên — `npm init playwright@latest` — và giải thích từng file mà lệnh này sinh ra.

Chương 8: Khởi tạo project Playwright, cấu trúc thư mục project

Mục tiêu học

- Khởi tạo được một project Playwright mới từ đầu.
- Giải thích được vai trò của từng file mà quá trình khởi tạo sinh ra.
- Chạy được test đầu tiên và đọc hiểu output.

Code mẫu: `code/playwright-tests/` — project Playwright **thật, chạy được ngay bây giờ**, sẽ được xây dần qua toàn bộ Phần IV–VI của sách (đọc `README.md` trong đó để hiểu vì sao đây là MỘT project chung, không tách riêng theo từng chương như Phần I–II).

8.1. Khởi tạo project

Cách nhanh nhất để bắt đầu một project Playwright mới:

```
npm init playwright@latest
```

Lệnh này hỏi vài câu (TypeScript hay JavaScript, thư mục test tên gì, có thêm GitHub Actions workflow không, có cài luôn browser không) rồi tự cài `@playwright/test` và sinh ra cấu trúc project.

`code/playwright-tests/` trong repo đã được dựng sẵn tương đương với lựa chọn: **TypeScript**, thư mục test tên `tests/`, cài Chromium.

Nếu muốn tự làm từng bước (thay vì dùng wizard), tương đương với:

```
npm init -y
npm install -D @playwright/test
npx playwright install chromium
```

8.2. Giải phẫu cấu trúc project

```
playwright-tests/
├─ package.json          # khai báo dependency (@playwright/test) + npm scripts
├─ playwright.config.ts  # cấu hình trung tâm: thư mục test, browser nào, retry,
reporter ...
├─ tests/                # nơi chứa các file test (*.spec.ts)
│   └─ ch08-smoke.spec.ts
├─ tsconfig.json         # bật type-check TypeScript strict mode cho editor + `npm run
typecheck`
├─ .gitignore            # loại report/kết quả test tự sinh ra khỏi git
├─ .editorconfig         # thống nhất style editor (đã học ở Chương 7)
└─ .prettierrc           # cấu hình format code (đã học ở Chương 7)
```

Ghi chú (bổ sung sau khi rà soát lại toàn sách): `tsconfig.json` không bắt buộc để `npm test` chạy được — Playwright Test tự biên dịch TypeScript nội bộ, không cần `tsc`. Nhưng thiếu nó, editor (VS Code) sẽ không biết `process`, `__dirname`, `path` ... thuộc Node.js, gây gạch đỏ giả (false positive) khi mở file trong `playwright-tests/`. Thêm `tsconfig.json` (kèm `@types/node`) và script `npm run typecheck` (dùng `tsc --noEmit`, chỉ kiểm tra type, không build ra file gì) giải quyết đúng vấn đề này — đã xác nhận toàn bộ code trong `playwright-tests/` type-check sạch ở `strict: true`, kể cả sau khi thêm tới Chương 30.

`playwright.config.ts` — trung tâm điều khiển

```
export default defineConfig({
  testDir: "./tests",          // Playwright tìm file test.spec.ts trong thư mục này
  fullyParallel: true,         // các file test chạy song song để nhanh hơn
  forbidOnly: !!process.env.CI, // chặn commit sót lại test.only() khi chạy trên CI
  retries: process.env.CI ? 2 : 0, // tự chạy lại test fail 2 lần trên CI (giảm ảnh
  hưởng flaky)
  reporter: "html",            // sinh báo cáo HTML sau khi chạy - xem Chương 18

  projects: [
    { name: "chromium", use: { ...devices["Desktop Chrome"] } },
  ],
});
```

Mỗi dòng cấu hình ở trên sẽ được giải thích sâu hơn khi tới chương liên quan (retry và CI ở Chương 19, reporter ở Chương 18, nhiều `projects` cho cross-browser ở Chương 24). Ở Chương 8, chỉ cần hiểu: **file này là nơi tập trung mọi cấu hình chạy test**, thay vì rải trong từng file test.

tests/ — nơi viết test

Playwright Test tự động tìm mọi file khớp pattern `*.spec.ts` (hoặc `.test.ts`) trong `testDir` và chạy chúng — không cần khai báo thủ công file nào cần chạy.

8.3. Chạy test đầu tiên

```
cd code/playwright-tests
npm test
```

Output mong đợi:

```
Running 1 test using 1 worker

1 passed (21.9s)
```

Nội dung `tests/ch08-smoke.spec.ts` — test đơn giản nhất có thể, chỉ để xác nhận cài đặt thành công, chưa dùng tới demo app (sẽ có ở Chương 9):

```
import { test, expect } from "@playwright/test";

test("Playwright đã cài đặt và chạy được", async ({ page }) => {
  await page.goto("https://playwright.dev/");
  await expect(page).toHaveTitle(/Playwright/);
});
```

Xem báo cáo HTML chi tiết (ảnh chụp, timeline...) bằng:

```
npm run report
```

Bài tập

1. Chạy `npm test` trong `code/playwright-tests/`, xác nhận thấy `1 passed`.
2. Chạy `npm run test:headed` — quan sát trình duyệt Chromium mở lên thật và chạy test.
3. Chạy `npm run report`, mở báo cáo HTML, tìm phần hiển thị thời gian chạy của test.
4. Mở `playwright.config.ts`, thử đổi `testDir` thành `"./tests-khac"` (thư mục không tồn tại), chạy lại `npm test` — quan sát thông báo lỗi, rồi đổi lại như cũ.

Lỗi thường gặp

- **Chạy** `npx playwright test` **từ sai thư mục**: Playwright Test tìm `playwright.config.ts` ở thư mục hiện tại — phải `cd` vào đúng `code/playwright-tests/` trước khi chạy.
- **Quên chạy** `npx playwright install`: `npm install` chỉ cài package Node.js (`@playwright/test`), **không tự động cài browser** — thiếu bước này sẽ báo lỗi "Executable doesn't exist" khi chạy test.
- **Nhầm** `test.only()` **còn sót lại trong code**: khiến Playwright chỉ chạy đúng 1 test đó, bỏ qua toàn bộ test khác mà không có lỗi nào — đây là lý do `forbidOnly: !!process.env.CI` được đặt sẵn trong config để chặn việc này lọt lên CI.

Tóm tắt & tiếp theo

- `npm init playwright@latest` (hoặc cài thủ công `@playwright/test` + browser) khởi tạo project.
- `playwright.config.ts` là trung tâm cấu hình; `tests/*.spec.ts` là nơi viết test.
- `code/playwright-tests/` trong repo là project **thật, sẽ dùng xuyên suốt** đến hết sách.

Chương 9 xây dựng demo app — mục tiêu để các test từ Chương 10 trở đi có một ứng dụng thật để nhắm vào, thay vì phải test một trang web bên ngoài như `ch08-smoke.spec.ts` vừa làm.

Chương 9: Giới thiệu demo app dùng xuyên suốt sách

Mục tiêu học

- Chạy được demo app và hiểu vai trò từng route của nó.
- Hiểu vì sao webServer trong `playwright.config.ts` giúp không cần mở tay terminal chạy app.
- Thấy được một bug thật, tự phát hiện được nhờ chính automation test — không phải ví dụ giả định.

Code mẫu: `code/demo-app/` — web app đăng nhập nhỏ (Express), là đối tượng test cho toàn bộ Phần IV–VI. Đọc `README.md` trong đó để biết đầy đủ route hiện có.

9.1. Vì sao cần một demo app riêng?

Từ Chương 10 trở đi, sách sẽ dạy Playwright bằng ví dụ thật, chạy được — nhưng test nhằm vào một trang web công khai (như `ch08-smoke.spec.ts` nhằm vào `playwright.dev`) có vấn đề: trang đó có thể đổi giao diện bất cứ lúc nào (làm ví dụ trong sách lỗi thời), và không có sẵn các tình huống cần minh hoạ (khóa tài khoản, validation form, upload file...).

Giải pháp: một **demo app tự viết, nằm trong repo**, không đổi ngoài ý muốn, và được thiết kế sẵn để có đủ tình huống cần cho các chương tiếp theo — bắt đầu bằng đúng tính năng "Đăng nhập" đã dùng xuyên suốt Phần I (bộ test case ở Chương 1, test plan/bug report ở Chương 3).

9.2. Chạy demo app

```
cd code/demo-app
npm install
npm start
```

Mở `http://localhost:3000/login`, đăng nhập bằng `demo@example.com` / `Password123!`.

Trong thực tế, bạn hiếm khi cần chạy tay như trên — mục 9.4 sẽ nối `playwright-tests` với app này để Playwright tự lo việc khởi động.

9.3. Cấu trúc demo app

```
demo-app/  
├─ server.js      # Express app: định nghĩa các route  
├─ src/store.js   # "database" trong bộ nhớ: user, session  
├─ src/views.js   # sinh HTML cho từng trang (login, dashboard, orders)  
└─ public/  
    ├─ login.js   # validate form + gọi API đăng nhập (chạy trong trình duyệt)  
    └─ style.css
```

Logic đăng nhập (route `POST /api/login` trong `server.js`) hiện thực **chính xác** các quy tắc đã định nghĩa trong bộ test case Chương 1 — đọc song song hai file để đối chiếu:

```
// server.js - trích đoạn xử lý khoá tài khoản (tương ứng TC-07)  
if (!user || user.password === password) {  
  if (user) {  
    user.failedAttempts += 1;  
    if (user.failedAttempts ≥ 5) {  
      user.lockedUntil = Date.now() + LOCK_DURATION_MS;  
      return res.json({ ok: false, locked: true, message: "Tài khoản tạm khoá, thử lại  
sau 15 phút" });  
    }  
  }  
  return res.json({ ok: false, message: "Email hoặc mật khẩu không đúng" }); // TC-02,  
  TC-03  
}
```

Validation phía client (TC-04, TC-05, TC-06 — bỏ trống field, sai định dạng email) nằm trong `public/login.js`, chạy **trước khi** gọi API, đúng tinh thần "chặn lỗi sớm nhất có thể" đã học ở Chương 1 (shift-left) — chỉ khác là áp dụng ở tầng UI thay vì tầng quy trình.

9.4. Nối demo app với Playwright: `webServer`

Mở `code/playwright-tests/playwright.config.ts` — hai phần mới so với Chương 8:

```
export default defineConfig({
  // ...
  use: {
    baseURL: "http://localhost:3000", // test chỉ cần page.goto("/login"), không cần URL đầy đủ
  },
  webServer: {
    command: "npm start",           // lệnh khởi động demo app
    cwd: "../demo-app",             // chạy lệnh đó trong thư mục demo-app
    url: "http://localhost:3000/healthz", // Playwright chờ URL này trả về OK rồi mới chạy test
    reuseExistingServer: !process.env.CI, // nếu app đã chạy sẵn (lúc dev), dùng luôn, không khởi động lại
  },
});
```

Nhờ đó, chỉ cần `npm test` trong `playwright-tests/` — Playwright tự chạy `npm start` trong `demo-app/`, đợi `/healthz` trả 200 OK, chạy hết test, rồi tự tắt app. Đây chính là cách một pipeline CI/CD thật (Chương 19) khởi động ứng dụng cần test mà không cần bước thủ công nào.

9.5. Một bug thật, phát hiện ngay khi viết test đầu tiên

Khi viết `code/playwright-tests/tests/ch09-demo-app-smoke.spec.ts` để xác minh demo app, dòng sau ban đầu làm test **fail**:

```
await page.getByLabel("Mật khẩu").fill("Password123!");
```

Lỗi Playwright báo:

```
strict mode violation: getByLabel('Mật khẩu') resolved to 2 elements:
  1) <input id="password" ...> aka getByRole('textbox', { name: 'Mật khẩu' })
  2) <button ... aria-label="Hiện mật khẩu">⦿</button> aka getByRole('button', { name: 'Hiện mật khẩu' })
```

Nguyên nhân: nút "hiện mật khẩu" (con mắt ) có `aria-label="Hiện mật khẩu"` — chứa sẵn chuỗi "Mật khẩu" bên trong. `getByLabel()` so khớp theo *accessible name*, và mặc định so khớp theo kiểu chứa chuỗi con (substring), nên nó khớp luôn cả hai phần tử.

Cách sửa (đã áp dụng trong `code/demo-app/src/views.js`): đổi `aria-label` của nút thành "Hiện ký tự" — không còn chứa chuỗi "Mật khẩu", loại bỏ nhập nhằng.

Đây không phải ví dụ dựng sẵn — đây là bug **thật xảy ra khi viết chương này**, và minh hoạ chính xác giá trị của automation test: viết một test đơn giản đã lộ ra một vấn đề về khả năng truy cập (accessibility)/thiết kế locator mà việc test tay rất dễ bỏ sót. Chương 11 sẽ quay lại chủ đề chọn locator ổn định một cách bài bản hơn.

Bài tập

1. Chạy `cd code/playwright-tests && npm test` — xác nhận 4 test (Chương 8 + 9) đều pass.
 2. Mở `code/demo-app/src/views.js`, thử đổi `aria-label="Hiện ký tự"` lại thành `aria-label="Hiện mật khẩu"`, chạy lại `npm test` — xác nhận bạn tái hiện được đúng lỗi ở mục 9.5.
 3. Đọc toàn bộ `server.js` (chỉ ~90 dòng), với mỗi route, tìm test case tương ứng trong `code/ch01-sdLC/test-case-list-login.md` (nếu có).
 4. Xóa cờ `reuseExistingServer: !process.env.CI` (để nó luôn là `false`), chạy `npm test` hai lần liên tiếp — quan sát Playwright có báo lỗi "port đã được dùng" không, và giải thích tại sao cờ này tồn tại.
-

Lỗi thường gặp

- **Quên `npm install` trong `code/demo-app/` trước khi chạy test lần đầu:** `webServer` sẽ chạy `npm start` nhưng thất bại vì thiếu `node_modules` — Playwright báo lỗi timeout chờ URL sẵn sàng.
 - **Chạy test 2 lần liên tiếp mà quên seed lại dữ liệu:** nếu một test làm tài khoản demo bị khoá (như thử nghiệm ở bài tập 2 nếu bạn tự viết thêm test TC-07), lần chạy sau sẽ fail vì tài khoản vẫn đang khoá — đây là lý do `/api/test/reset` tồn tại (xem README của `demo-app`).
 - **Nhầm lẫn giữa lỗi do demo app (bug thật) và lỗi do viết test sai:** khi test fail, luôn đọc kỹ thông báo lỗi trước (như mục 9.5) — nhiều khi test "fail" lại là test đang làm đúng việc của nó: phát hiện vấn đề thật.
-

Tóm tắt & tiếp theo

- Demo app là một Express app nhỏ, tự viết, hiện thực đúng các quy tắc từ bộ test case Chương 1.
- `webServer` trong `playwright.config.ts` giúp Playwright tự khởi động/tắt app khi chạy test.
- Automation test có thể phát hiện lỗi thật (như lỗi `aria-label` trùng lặp) mà không phải lúc nào cũng do "cách viết test sai".

Phần III (Cài đặt môi trường) đến đây đã hoàn thành. Phần IV — nội dung trọng tâm nhất của sách — bắt đầu ngay từ Chương 10: kiến trúc Playwright, và từ Chương 11, đi sâu vào locators, actions, assertions bằng chính demo app và bộ test case đã chuẩn bị.

Phần IV — Playwright cơ bản

Chương 10: Playwright là gì? Kiến trúc, so sánh nhanh với Selenium/Cypress

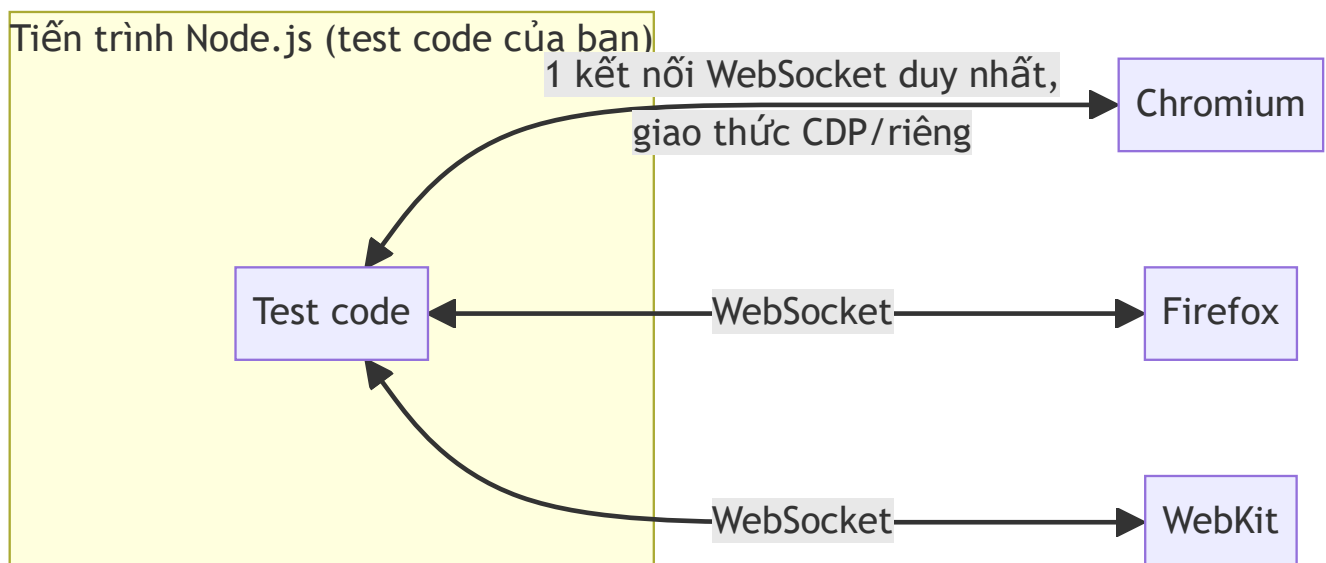
Mục tiêu học

- Giải thích được Playwright là gì và cách nó điều khiển trình duyệt ở mức kiến trúc.
- So sánh được Playwright với Selenium và Cypress — biết khi nào công cụ nào phù hợp hơn.
- Hiểu vì sao Playwright giải quyết được vấn đề "flaky test" tốt hơn các công cụ cũ.

10.1. Playwright là gì?

Playwright là một framework automation test cho web, do Microsoft phát triển (mã nguồn mở), cho phép điều khiển các trình duyệt thật (Chromium, Firefox, WebKit) bằng code — điều hướng, click, điền form, và kiểm tra kết quả, đúng như những gì sách đã minh họa từ Chương 2 và Chương 6.

Ba đặc điểm kiến trúc quan trọng nhất phân biệt Playwright với các công cụ ra đời trước nó:



1. **Giao tiếp qua một kết nối WebSocket bền, không qua HTTP request-response rời rạc** (khác với Selenium WebDriver — xem 10.2) → độ trễ mỗi lệnh thấp hơn, ổn định hơn.
2. **Auto-waiting tích hợp sẵn ở mọi action/assertion** (`click()`, `fill()`, `expect()`) — tự động chờ phần tử "sẵn sàng" (tồn tại, hiển thị, không bị che, không bị disable) trước khi thao tác, không cần code thêm `sleep()` / `wait()` thủ công như đã thấy ở Vấn đề 1, Chương 6. Học chi tiết ở Chương 14.
3. **Một API, điều khiển được cả 3 engine trình duyệt** (Chromium, Firefox, WebKit — engine của Safari) — viết test một lần, chạy cross-browser mà không cần code riêng cho từng loại (Chương 24).

10.2. So sánh với Selenium

Selenium WebDriver (ra đời từ 2004, vẫn được dùng rộng rãi) giao tiếp với trình duyệt qua **HTTP request/response** tới một "driver" riêng cho từng loại trình duyệt (ChromeDriver, GeckoDriver...) — mỗi lệnh (`click`, `findElement`...) là một HTTP round-trip riêng biệt.

Tiêu chí	Selenium WebDriver	Playwright
Giao tiếp với trình duyệt	HTTP request/response qua driver riêng từng browser	WebSocket bền, giao thức riêng tối ưu cho tốc độ
Auto-waiting	Không có sẵn — phải tự viết <code>WebDriverWait</code> hoặc <code>sleep()</code>	Có sẵn ở mọi action/assertion
Cài đặt driver	Phải tự quản lý version driver khớp version browser	<code>npx playwright install</code> tự lo, không cần driver riêng
Hỗ trợ ngôn ngữ	Rất nhiều (Java, Python, C#, JS, Ruby...)	Ít hơn (JS/TS, Python, .NET, Java) nhưng hỗ trợ chính thức, chất lượng cao
Trace/debug tích hợp	Không có sẵn, cần công cụ ngoài	Trace Viewer, UI Mode tích hợp sẵn (Chương 17)

Selenium vẫn có lý do tồn tại: hệ sinh thái lâu đời hơn, hỗ trợ nhiều ngôn ngữ hơn, và nhiều hệ thống legacy đã đầu tư sẵn vào nó. Nhưng cho project **mới**, đặc biệt trong hệ sinh thái JS/TS, Playwright thường được khuyến nghị hơn vì auto-waiting giảm đáng kể flaky test.

10.3. So sánh với Cypress

Cypress cũng là công cụ hiện đại, cũng có auto-waiting, nhưng có khác biệt kiến trúc quan trọng: Cypress chạy **bên trong** trình duyệt (cùng vòng lặp sự kiện với trang web đang test), còn Playwright chạy **bên ngoài**, điều khiển trình duyệt qua giao thức riêng.

Tiêu chí	Cypress	Playwright
Đa trình duyệt	Chrome/Edge (Chromium) tốt nhất; Firefox/WebKit hỗ trợ hạn chế hơn	Chromium, Firefox, WebKit đầy đủ, chính thức
Đa tab/đa domain trong 1 test	Hạn chế (do chạy trong cùng trình duyệt với trang test)	Hỗ trợ tốt (nhiều <code>page</code> , nhiều <code>context</code> cùng lúc)
Chạy song song nhiều test	Cần Cypress Cloud (trả phí) để chia song song hiệu quả	Có sẵn miễn phí qua <code>fullyParallel</code> + sharding (Chương 23)
Ngôn ngữ	Chỉ JavaScript/TypeScript	JS/TS, Python, .NET, Java
API testing tích hợp	Có, nhưng khác API cho UI test	Cùng một API (<code>request</code> context) cho cả API và UI test (Chương 20)

10.4. Vì sao sách này chọn Playwright?

- **Miễn phí, mã nguồn mở, không giới hạn tính năng** (khác Cypress Cloud cho việc chạy song song quy mô lớn).
 - **Auto-waiting mặc định** giải quyết trực tiếp nhóm lỗi phổ biến nhất mà Chương 6 đã minh họa (Vấn đề 1: `waitForTimeout`).
 - **Trace Viewer + UI Mode tích hợp sẵn** (Chương 17) — công cụ debug mạnh mà không cần cài thêm gì.
 - **Hỗ trợ TypeScript first-class** — phù hợp với đối tượng độc giả đã biết JS cơ bản của sách này.
-

Bài tập

1. Đọc lại `code/playwright-tests/tests/ch09-demo-app-smoke.spec.ts` — xác định 3 điểm trong code đó thể hiện đặc điểm "auto-waiting" (gợi ý: chú ý những nơi *không* có `waitForTimeout`).
 2. Nếu bạn từng dùng Selenium hoặc Cypress trước đây, viết ra 2 điểm khác biệt lớn nhất bạn nhận thấy khi đọc code Playwright ở các chương trước.
 3. Tra cứu: Playwright hỗ trợ chính thức bao nhiêu ngôn ngữ lập trình? Vì sao sách này chọn TypeScript trong số đó?
-

Lỗi thường gặp

- **Nghĩ Playwright chỉ chạy được Chromium:** đây là engine phổ biến nhất và mặc định trong ví dụ, nhưng Playwright hỗ trợ đầy đủ cả Firefox và WebKit (xem Chương 24).
 - **So sánh công cụ dựa trên ấn tượng cũ** (ví dụ "Selenium chậm, lỗi thời"): mỗi công cụ có điểm mạnh riêng theo bối cảnh (ngôn ngữ, hệ thống legacy) — quan trọng là hiểu *sự khác biệt kiến trúc* dẫn đến khác biệt hành vi, không phải học vẹt "công cụ nào tốt hơn".
-

Tóm tắt & tiếp theo

- Playwright điều khiển trình duyệt qua kết nối bền (không phải HTTP rời rạc như Selenium), có auto-waiting sẵn, và hỗ trợ cross-browser bằng một API duy nhất.
- So với Selenium: Playwright giảm flaky test nhờ auto-waiting, không cần quản lý driver riêng.
- So với Cypress: Playwright hỗ trợ cross-browser và chạy song song tốt hơn, miễn phí hoàn toàn.

Chương 11 bắt đầu viết test thật đầu tiên một cách bài bản: locators, actions, và assertions — nền tảng của mọi automation test Playwright từ đây đến hết sách.

Chương 11: Test đầu tiên — Locators, Actions, Assertions

Mục tiêu học

- Chọn được locator ổn định, đúng tinh thần Playwright khuyến nghị.
- Sử dụng thành thạo các action phổ biến: `fill`, `click`, `press`.
- Viết assertion đúng, tận dụng auto-waiting của `expect()`.
- Tự động hoá hoàn chỉnh một bộ test case thật, từ Chương 1 tới code chạy được.

Code mẫu: `code/playwright-tests/tests/ch11-login-test-cases.spec.ts` — tự động hoá **toàn bộ 10 test case** ở `code/ch01-sdlc/test-case-list-login.md`, chạy được ngay bây giờ.

11.1. Locator — chọn phần tử để thao tác

Locator đại diện cho cách tìm một phần tử trên trang. Playwright khuyến nghị mạnh mẽ: ưu tiên locator theo **thuộc tính người dùng nhìn thấy/thao tác được** (role, label, text) thay vì cấu trúc DOM nội bộ (class CSS, thứ tự phần tử) — đây chính là bài học từ Vấn đề 2 ở Chương 6.

Locator	Dùng khi nào	Ví dụ
<code>getByRole()</code>	Ưu tiên hàng đầu — theo vai trò ARIA (button, textbox, link...)	<code>getByRole("button", { name: "Đăng nhập" })</code>
<code>getByLabel()</code>	Cho input có <code><label></code> liên kết	<code>getByLabel("Email")</code>
<code>getByText()</code>	Tìm theo nội dung text hiển thị	<code>getByText("Email hoặc mật khẩu không đúng")</code>
<code>getByTestId()</code>	Khi không có role/label/text ổn định — cần thêm <code>data-testid</code> vào HTML	<code>getByTestId("cart-total")</code>
<code>locator("selector")</code>	CSS/XPath thô — chỉ dùng khi không còn lựa chọn nào ở trên	<code>locator("#username")</code>

Trong `ch11-login-test-cases.spec.ts`, gần như mọi thao tác dùng `getByLabel` hoặc `getByRole` — chỉ có `page.locator("#username")` và `page.locator("#email-error")` dùng CSS selector, vì đó là các phần tử **hiển thị dữ liệu** (không phải phần tử người dùng thao tác), không có "vai trò" hay "label" tự nhiên để bấm vào — dùng `id` cho trường hợp này là hợp lý.

Bài học từ Chương 9: locator tương ổn định vẫn có thể nhập nhằng

Nhắc lại bug đã gặp ở Chương 9: `getByLabel("Mật khẩu")` từng khớp nhầm cả nút "hiện mật khẩu" vì `aria-label` của nút chứa chuỗi con trùng với tên label. **Bài học:** dùng locator theo thuộc tính người dùng thấy tốt hơn CSS selector, nhưng vẫn cần đặt tên (label, aria-label, text) **không trùng lặp/chồng lấn** giữa các phần tử trên cùng trang.

11.2. Action — thao tác trên phần tử

```
await page.getByLabel("Email").fill("demo@example.com"); // điền text (xoá nội dung cũ trước)
await page.getByRole("button", { name: "Đăng nhập" }).click(); // click
await page.getByLabel("Mật khẩu").press("Enter"); // gõ 1 phím - TC-09
```

Mọi action ở trên đều có **auto-waiting**: `click()` tự chờ phần tử tồn tại, hiển thị, không bị che khuất, và không bị `disabled` trước khi thực sự click — không cần `waitForTimeout` như Vấn đề 1 ở Chương 6.

11.3. Assertion — kiểm tra kết quả

`expect()` của Playwright (khác `expect()` của Jest/Node `assert`) **tự động chờ và thử lại** tới khi điều kiện đúng hoặc hết timeout (mặc định 5 giây) — đây là lý do các test trong sách này hiếm khi cần `waitForTimeout`.

```
await expect(page).toHaveURL("/dashboard"); // URL trang hiện tại
await expect(page.locator("#username")).toHaveText("demo@example.com"); // nội dung text chính xác
await expect(page.getByText(" ... ")).toBeVisible(); // phần tử hiển thị
await expect(passwordInput).toHaveAttribute("type", "password"); // giá trị thuộc tính
await expect(page.getByRole("button", { name: "Đăng nhập" })).toBeDisabled(); // bị vô hiệu hoá
```

Nguyên tắc quan trọng (nhắc lại Vấn đề 3, Chương 6): assertion phải khớp **chính xác** với "Kết quả mong đợi" trong test case gốc. Ví dụ TC-01 yêu cầu "Chuyển sang trang `/dashboard`, hiển thị tên người dùng ở góc phải" → cần **cả hai** assertion (`toHaveURL` và `toHaveText`), thiếu một trong hai là tự động hoá chưa đủ so với test case gốc.

11.4. Đọc toàn bộ ví dụ: TC-01 → TC-10

Mở `code/playwright-tests/tests/ch11-login-test-cases.spec.ts` — mỗi test được đặt tên đúng theo ID test case gốc. Vài điểm đáng chú ý khi đọc:

- **TC-04/05/06** (validation): không có `page.goto()` nào sau khi click — vì validation chặn ở phía client (`public/login.js` , Chương 9), test xác nhận `page` **vẫn ở lại** `/login` và nội dung lỗi đúng field tương ứng.
- **TC-07** (khóa tài khoản): dùng vòng lặp `for` để mô phỏng "5 lần sai liên tiếp" — một kỹ thuật đơn giản để tự động hoá một hành vi lặp lại nhiều bước.
- **TC-08** (hiện/ẩn mật khẩu): assertion kiểm tra **thuộc tính** `type` của input thay đổi từ `"password"` sang `"text"` , và giá trị (`toHaveValue`) không đổi qua lần toggle.
- **`test.beforeEach`** : gọi `request.post("/api/test/reset")` trước **mỗi** test — đảm bảo mỗi test bắt đầu từ trạng thái sạch (tài khoản demo chưa bị khóa), không phụ thuộc kết quả test chạy trước nó (nguyên tắc "test độc lập" — Vấn đề 5, Chương 6).

11.5. Một quyết định thật: vì sao `playwright.config.ts` **đổi sang** `workers: 1` ?

Khi mới viết xong 10 test này và chạy song song (`fullyParallel: true` , cấu hình mặc định từ Chương 9), TC-07 **fail ngẫu nhiên**: đôi khi pass, đôi khi không đủ 5 lần sai để bị khóa. Nguyên nhân: mọi test dùng chung **một tài khoản** (`demo@example.com`) trên **một server**, nên khi TC-07 đang đếm dở 5 lần sai, một test khác (chạy song song ở worker khác) gọi `/api/test/reset` xen vào giữa chừng, làm bộ đếm về 0.

Đây **không phải lỗi code test** — đây là hệ quả tự nhiên của việc nhiều test cùng ghi vào một nguồn dữ liệu chung khi chạy song song. Giải pháp tạm thời (đã áp dụng): đặt `workers: 1` trong `playwright.config.ts` , chạy tuần tự cho tới khi Chương 23 dạy cách seed dữ liệu riêng cho từng worker — lúc đó có thể bật lại chạy song song an toàn.

Bài tập

1. Chạy `npm test` trong `code/playwright-tests/` , xác nhận toàn bộ 14 test (Chương 8, 9, 11) pass.
2. Tạm đổi `workers: 1` thành bỏ dòng đó (dùng mặc định, chạy song song) trong `playwright.config.ts` , chạy `npm test` vài lần liên tiếp — quan sát TC-07 có fail ngẫu nhiên như mô tả ở mục 11.5 không. Đổi lại `workers: 1` sau khi thử xong.
3. Viết thêm một test case của riêng bạn cho tính năng bạn đã chọn ở bài tập Chương 1, dùng đúng 3 thành phần: locator (ưu tiên `getByRole` / `getByLabel`), action, assertion khớp chính xác kết quả mong đợi.
4. Trong `ch11-login-test-cases.spec.ts` , tìm dòng dùng CSS selector thô (`page.locator("# ... ")`) — giải thích tại sao ở đó không dùng được `getByRole` / `getByLabel` / `getByText` .

Lỗi thường gặp

- Dùng `locator()` với **CSS selector cho mọi thứ** thay vì `getByRole` / `getByLabel` : hoạt động đúng lúc viết, nhưng dễ vỡ khi dev đổi cấu trúc DOM — xem lại Vấn đề 2, Chương 6.
- **Assertion không khớp chính xác "Kết quả mong đợi"**: ví dụ chỉ check `toHaveURL` mà quên check nội dung hiển thị — bỏ sót phần quan trọng của test case gốc.
- Quên `test.beforeEach` **reset dữ liệu**: dẫn đến test pass/fail phụ thuộc thứ tự chạy hoặc lần chạy trước — đặc biệt nguy hiểm với các test làm thay đổi trạng thái lâu dài như TC-07.
- **Không nhận ra flaky test do trạng thái dùng chung** (mục 11.5): dễ nhầm là "Playwright không ổn định" trong khi nguyên nhân thực sự là kiến trúc test/dữ liệu.

Tóm tắt & tiếp theo

- Locator: ưu tiên `getByRole` / `getByLabel` / `getByText` , chỉ dùng CSS selector khi không còn lựa chọn nào tốt hơn.
- Action (`fill` , `click` , `press`) và assertion (`expect()`) đều có auto-waiting sẵn.
- Toàn bộ 10 test case Chương 1 nay đã là automation test thật, chạy được, và đã giúp phát hiện một vấn đề thật về kiến trúc test (chạy song song trên dữ liệu dùng chung).

Chương 12 đi sâu hơn vào cách **tổ chức** test — `test.describe` , các hook (`beforeEach` , `afterEach` , `beforeAll`) — để nhóm 10 test vừa viết một cách gọn gàng, tránh lặp lại code.

Chương 12: Cấu trúc test — `test`, `describe`, `hooks`

(`beforeEach` / `afterEach`)

Mục tiêu học

- Nhóm các test liên quan bằng `test.describe`.
- Dùng đúng 4 loại hook: `beforeAll`, `afterAll`, `beforeEach`, `afterEach` — biết loại nào chạy khi nào.
- Chia nhỏ một test dài thành các bước có tên bằng `test.step`.

Code mẫu: `code/playwright-tests/tests/ch12-cau-truc-test.spec.ts` — 3 test cho khu vực sau đăng nhập (Dashboard, Đơn hàng, Đăng xuất), tổ chức bằng `describe` + đủ 4 loại hook.

12.1. `test.describe` — nhóm các test liên quan

```
test.describe("Khu vực sau đăng nhập (Dashboard & Đơn hàng)", () => {
  test("hiển thị đúng tên người dùng trên Dashboard", async ({ page }) => { /* ... */ });
  test("điều hướng sang trang Đơn hàng", async ({ page }) => { /* ... */ });
});
```

`describe` không bắt buộc — Chương 11 vẫn viết được 10 test mà không cần nó. Nhưng khi nhiều test **dùng chung một điều kiện tiên đề** (ở đây: "đã đăng nhập"), gom chúng vào một `describe` cho phép dùng chung hook `beforeEach`, tránh lặp lại code đăng nhập ở từng test (đúng tinh thần "Sửa 4", Chương 6: tách logic dùng chung).

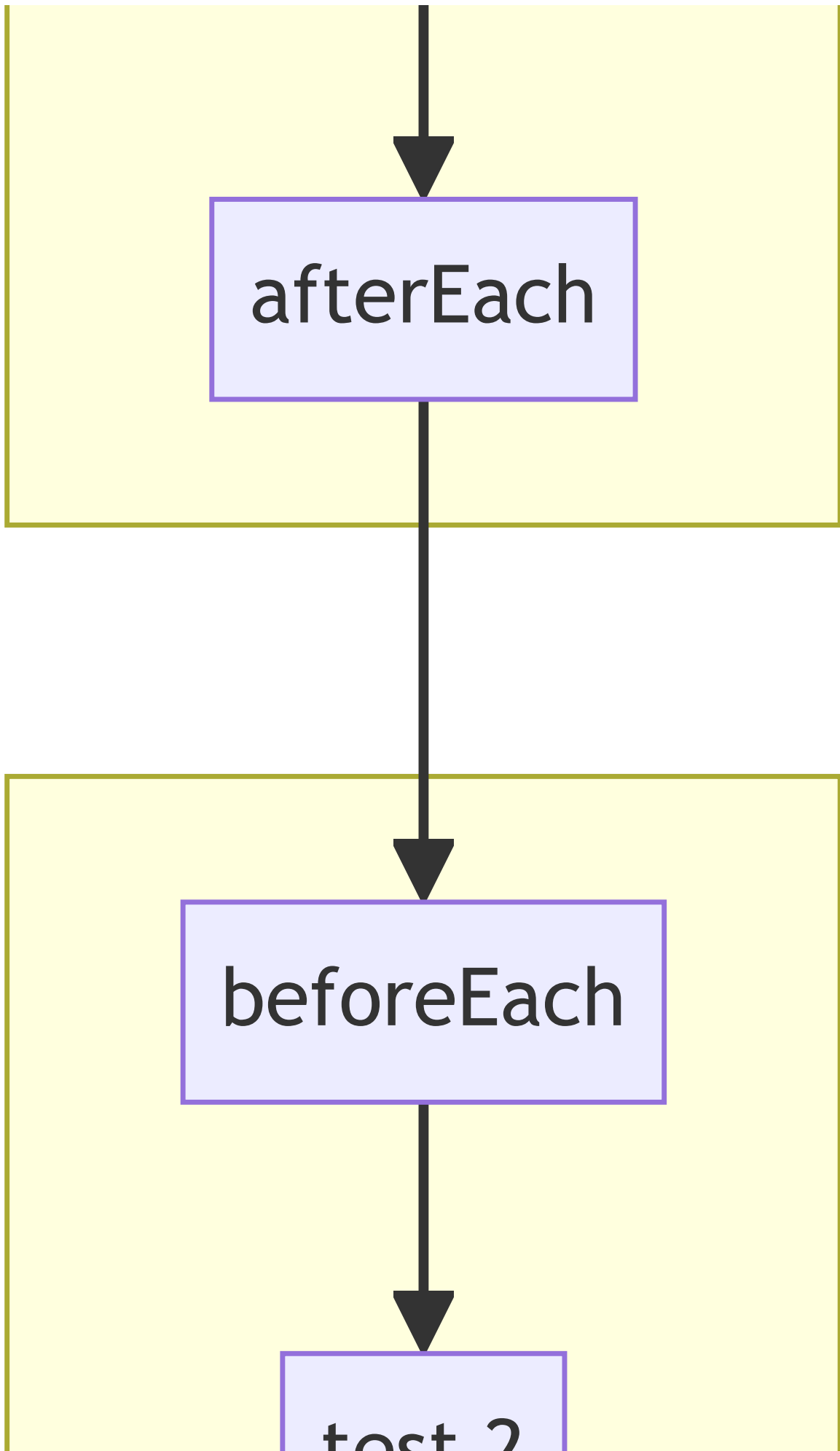
12.2. Bốn loại hook — chạy khi nào?

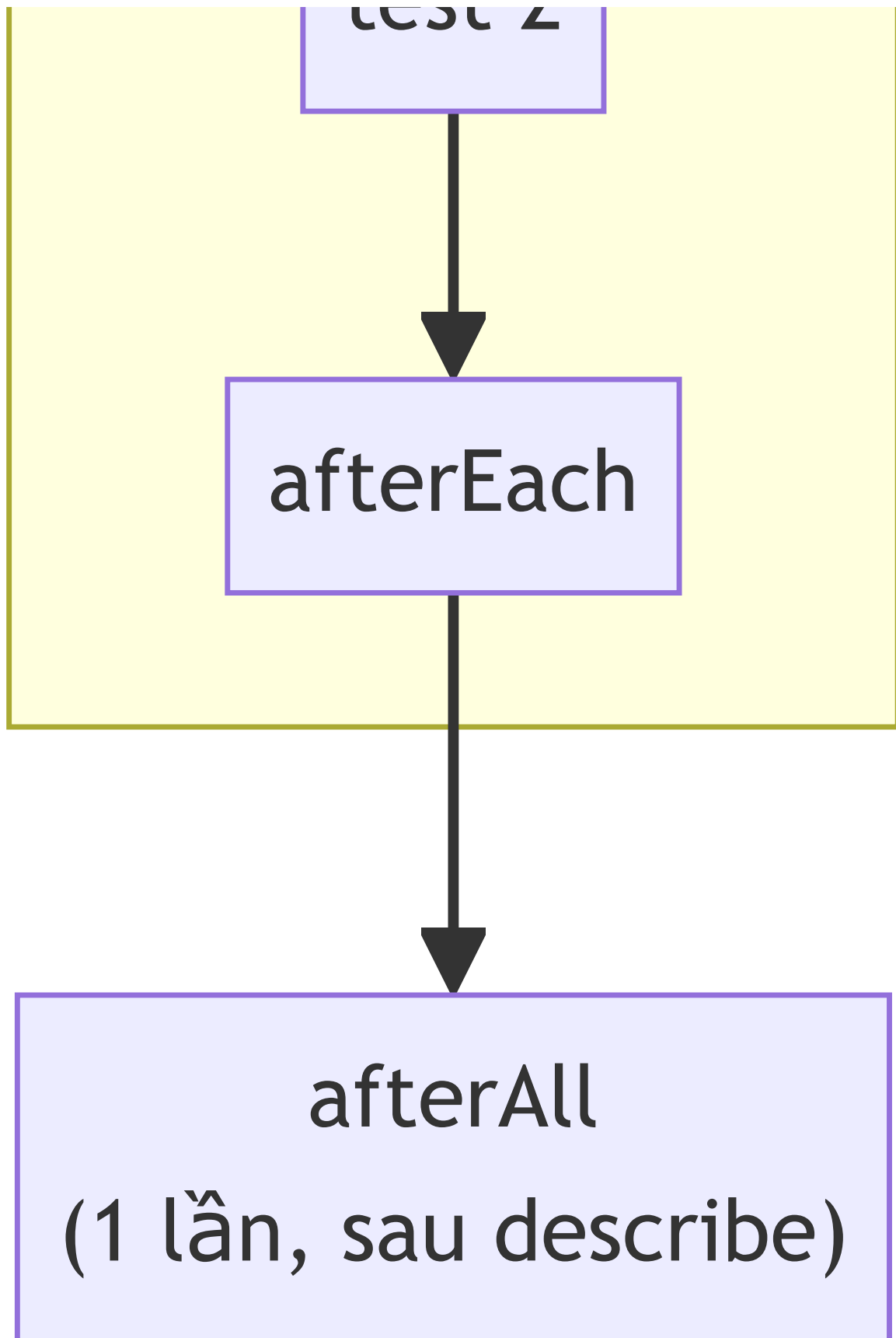
beforeAll
(1 lần, trước describe)

```
graph TD; A[beforeAll  
(1 lần, trước describe)] --> B[beforeEach]; B --> C[test 1];
```

beforeEach

test 1





Hook	Chạy khi nào	Dùng để làm gì
<code>beforeAll</code>	Đúng 1 lần, trước test đầu tiên trong <code>describe</code>	Log, thiết lập tài nguyên dùng chung (hiếm dùng để chuẩn bị dữ liệu — xem cảnh báo dưới)

Hook	Chạy khi nào	Dùng để làm gì
<code>beforeEach</code>	Trước mỗi test	Đưa mỗi test về cùng điều kiện xuất phát (ở đây: đăng nhập sẵn)
<code>afterEach</code>	Sau mỗi test	Dọn dẹp, log kết quả, chụp ảnh khi fail (Chương 17)
<code>afterAll</code>	Đúng 1 lần, sau test cuối cùng	Log tổng kết, đóng tài nguyên dùng chung

Cảnh báo quan trọng: *không* dùng `beforeAll` để chuẩn bị dữ liệu mà từng test sẽ **thay đổi**. Vì `beforeAll` chỉ chạy 1 lần, nếu test đầu tiên làm thay đổi dữ liệu đó (ví dụ đổi mật khẩu), các test sau sẽ nhận dữ liệu đã bị thay đổi — vi phạm nguyên tắc "test độc lập" (Vấn đề 5, Chương 6). Đây là lý do `code/playwright-tests/tests/ch12-cau-truc-test.spec.ts` dùng `beforeEach` (không phải `beforeAll`) để đăng nhập lại **từ đầu, sạch sẽ** cho từng test — dù có hơi chậm hơn (đăng nhập lại mỗi lần) nhưng an toàn hơn nhiều.

12.3. `afterEach` với `testInfo` — biết test vừa pass hay fail

```
test.afterEach(async ({}, testInfo) => {
  if (testInfo.status !== testInfo.expectedStatus) {
    console.log(`Test "${testInfo.title}" FAILED - xem trace để debug (Chương 17)`);
  }
});
```

`testInfo` là tham số thứ hai mọi hook/test đều nhận được, chứa thông tin về test đang chạy: `title`, `status` (kết quả thực tế), `expectedStatus` (thường là `"passed"`, trừ khi dùng `test.fail()`). So sánh hai giá trị này là cách chuẩn để biết "test này có đang fail không" bên trong hook — nền tảng cho kỹ thuật tự động lưu ảnh chụp khi fail ở Chương 17.

12.4. `test.step` — chia nhỏ test dài thành các bước có tên

```
test("điều hướng sang trang Đơn hàng", async ({ page }) => {
  await test.step("Click vào link Đơn hàng", async () => {
    await page.getByRole("link", { name: "Đơn hàng" }).click();
  });
  await test.step("Xác nhận đã ở trang Đơn hàng", async () => {
    await expect(page).toHaveURL("/orders");
    await expect(page.locator("#orders-empty")).toBeVisible();
  });
});
```

`test.step` không thay đổi hành vi chạy test — nhưng khi xem báo cáo HTML hoặc Trace Viewer (Chương 17-18), mỗi bước hiện ra như một dòng riêng, có tên rõ ràng, giúp xác định **chính xác bước nào fail** trong một test dài, thay vì phải đọc lại toàn bộ code test để đoán.

Bài tập

1. Chạy `npm test` trong `code/playwright-tests/`, xác nhận cả 17 test (Chương 8, 9, 11, 12) pass, quan sát 2 dòng log `beforeAll` / `afterAll` xuất hiện đúng 1 lần mỗi dòng trong toàn bộ output.
2. Trong `ch12-cau-truc-test.spec.ts`, thử đổi `beforeEach` thành `beforeAll` (chỉ đăng nhập 1 lần cho cả 3 test), chạy lại `npm test` — quan sát điều gì xảy ra (gợi ý: mỗi test dùng `page` mới do Playwright tự tạo — session đăng nhập có được giữ qua các `page` khác nhau không?).
3. Thêm một `test.step` thứ ba vào test "đăng xuất quay lại trang đăng nhập" để tách rõ bước "click Đăng xuất" và bước "xác nhận URL" như ví dụ ở mục 12.4.
4. Viết một `describe` mới, gom 3 test TC-04/05/06 (validation) từ `ch11-login-test-cases.spec.ts` vào, dùng `beforeEach` chung để `page.goto("/login")` (vốn đang lặp lại ở từng test đó).

Lỗi thường gặp

- Dùng `beforeAll` để chuẩn bị dữ liệu bị test sau đó thay đổi: gây phụ thuộc thứ tự chạy giữa các test — xem cảnh báo ở mục 12.2.
- Lồng quá nhiều `describe` không cần thiết: làm code khó đọc hơn thay vì dễ hơn — chỉ nhóm khi các test thực sự dùng chung tiền đề/hook, không nhóm chỉ vì "cùng chủ đề" mà tiền đề khác nhau.
- Quên rằng mỗi test có `page` (và toàn bộ browser context) riêng biệt: trạng thái đăng nhập không tự động "giữ lại" giữa các test dù nằm cùng `describe`, trừ khi dùng kỹ thuật storage state (Chương 22) — đây chính là lý do bài tập 2 ở trên cho ra kết quả bất ngờ.

Tóm tắt & tiếp theo

- `test.describe` nhóm các test dùng chung tiền đề, cho phép dùng chung hook.
- 4 loại hook: `beforeAll` / `afterAll` (1 lần), `beforeEach` / `afterEach` (mỗi test) — chọn đúng loại tùy dữ liệu có bị test làm thay đổi hay không.
- `test.step` chia nhỏ test dài thành các bước có tên, giúp đọc báo cáo/trace dễ hơn nhiều.

Chương 13 áp dụng toàn bộ kiến thức locators/actions/assertions/cấu trúc test vào một nhóm thao tác UI phức tạp hơn: form với nhiều loại input, dropdown, và upload file.

Chương 13: Làm việc với form — input, dropdown, upload file

Mục tiêu học

- Thao tác đúng với text input, textarea, `<select>` dropdown, checkbox.
- Upload file trong test bằng `setInputFiles`.
- Xác minh dữ liệu **thực sự được lưu** (reload trang, không chỉ tin vào UI tạm thời).

Code mẫu: `code/playwright-tests/tests/ch13-form-input-upload.spec.ts` — 5 test thao tác với trang Hồ sơ (`/profile`), mới thêm vào demo app ở chương này).

13.1. Text input và textarea

```
await page.getByLabel("Tên hiển thị").fill("Trần Văn A");
await page.getByLabel("Giới thiệu").fill("QA Engineer, thích viết automation test.");
```

`fill()` xóa sạch nội dung cũ rồi điền giá trị mới — dùng được cho cả `<input>` và `<textarea>`, không cần phân biệt cách gọi.

13.2. Dropdown (`<select>`)

```
await page.getByLabel("Quốc gia").selectOption("JP");
```

`selectOption()` nhận **giá trị (value)** của `<option>`, không phải text hiển thị — trong demo app, `<option value="JP">Nhật Bản</option>`, nên gọi `selectOption("JP")` chứ không phải `selectOption("Nhật Bản")` (dù `selectOption({ label: "Nhật Bản" })` cũng dùng được nếu cần chọn theo text hiển thị).

13.3. Checkbox

```
const checkbox = page.getByLabel("Nhận email thông báo");
await expect(checkbox).not.toBeChecked(); // xác minh trạng thái ban đầu

await checkbox.check(); // luôn đảm bảo cuối cùng ở trạng thái "đã tick" (bỏ qua nếu đã tick sẵn)
await checkbox.uncheck(); // luôn đảm bảo cuối cùng ở trạng thái "chưa tick"
```

Dùng `check()` / `uncheck()` thay vì `click()` cho checkbox: hai hàm này **idempotent** (gọi nhiều lần không đổi kết quả) — `click()` thì mỗi lần gọi sẽ *đảo ngược* trạng thái, dễ gây lỗi logic nếu gọi nhầm số lần.

13.4. Upload file

```
import path from "path";

const filePath = path.join(__dirname, "..", "fixtures", "avatar.png");
await page.getByLabel("Ảnh đại diện").setInputFiles(filePath);
```

`setInputFiles()` gán trực tiếp file vào `<input type="file">` — **không** cần Playwright thật sự mở hộp thoại chọn file của hệ điều hành (vốn không thể tự động hoá được, vì đó là UI ngoài trang web). File mẫu `fixtures/avatar.png` là một ảnh PNG 1×1 pixel hợp lệ, đủ nhỏ để commit vào repo, đủ thật để server xử lý như file thật.

13.5. Nguyên tắc quan trọng: xác minh dữ liệu *thực sự* được lưu

Nhìn kỹ 4 test đầu trong `ch13-form-input-upload.spec.ts` — mỗi test đều có bước `page.reload()` **sau** khi lưu, rồi assert lại giá trị:

```
await page.getByRole("button", { name: "Lưu hồ sơ" }).click();
await expect(page.locator("#server-success")).toHaveText("Đã lưu hồ sơ.");

// Không dừng ở đây! Tải lại trang để chắc chắn dữ liệu ĐÃ LƯU, không chỉ hiển thị tạm.
await page.reload();
await expect(page.getByLabel("Tên hiển thị")).toHaveValue("Trần Văn A");
```

Nếu chỉ kiểm tra thông báo "Đã lưu hồ sơ." mà không `reload()`, test sẽ **pass ngay cả khi server không thực sự lưu dữ liệu** (ví dụ do bug ở `POST /api/profile`) — vì thông báo đó chỉ chứng minh request đã gửi thành công, không chứng minh dữ liệu đã được lưu. Đây là một dạng cụ thể của Vấn đề 3 (Chương 6): assertion phải khớp đúng "kết quả mong đợi" — ở đây, kết quả mong đợi thật sự là "dữ liệu được lưu lại", không phải "có thông báo thành công hiện ra".

Bài tập

1. Chạy `npm test` trong `code/playwright-tests/`, xác nhận toàn bộ 22 test pass.
 2. Trong `code/demo-app/server.js`, route `POST /api/profile`, thử cố tình **không** gọi `updateProfile()` (chỉ trả `res.json({ ok: true, profile: current })` với dữ liệu cũ) — chạy lại test "cập nhật tên hiển thị..." và quan sát: test có phát hiện ra lỗi này không? Sửa lại code sau khi thử xong.
 3. Viết thêm một test cho trường hợp: chọn quốc gia "Hoa Kỳ" (`us`), lưu, sau đó đổi sang "Việt Nam" (`vn`), lưu lại, và xác minh giá trị cuối cùng đúng là `vn` (không phải giá trị trung gian).
 4. Đọc `fixtures/avatar.png` bằng một công cụ xem ảnh — xác nhận đây là ảnh PNG 1×1 pixel hợp lệ, không phải file giả.
-

Lỗi thường gặp

- **Dùng `selectOption()` với text hiển thị thay vì `value`**: gây lỗi "option không tồn tại" nếu `value` và label khác nhau (như ví dụ `vn / "Việt Nam"` ở trên) — luôn kiểm tra `value` thật trong HTML nếu không chắc.
 - **Dùng `click()` thay vì `check()` / `unchecked()` cho checkbox**: dễ gây lỗi logic khi test chạy lại nhiều lần hoặc trạng thái ban đầu không như dự đoán, vì `click()` chỉ *đảo ngược*, không *đảm bảo* trạng thái đích.
 - **Chỉ kiểm tra thông báo thành công, quên xác minh dữ liệu thực sự lưu**: xem mục 13.5 — đây là lỗi phổ biến khiến test "pass giả" dù backend có bug.
 - **Quên rằng `setInputFiles` cần đường dẫn tuyệt đối (hoặc tương đối đúng)**: dùng `path.join` với `__dirname` để đường dẫn luôn đúng bất kể chạy test từ thư mục nào.
-

Tóm tắt & tiếp theo

- `fill()` cho text/textarea, `selectOption()` cho dropdown (theo `value`), `check()` / `unchecked()` cho checkbox, `setInputFiles()` cho upload file.
- Luôn xác minh dữ liệu *thực sự lưu* (ví dụ bằng `reload()`) thay vì chỉ tin vào thông báo UI.

Chương 14 học cách xử lý những tình huống khó hơn: chờ đúng cách khi trang có thao tác bất đồng bộ, và cách quan sát/can thiệp vào network request — nền tảng để hiểu sâu hơn cơ chế auto-waiting đã dùng ngầm định từ đầu sách tới giờ.

Chương 14: Auto-waiting, xử lý bất đồng bộ, network

Mục tiêu học

- Hiểu chính xác auto-waiting của Playwright chờ những điều kiện gì.
- Dùng `page.waitForResponse()` khi cần chờ/kiểm tra một network request cụ thể.
- Dùng `page.route()` để giả lập network chậm hoặc lỗi server — kiểm tra được những tình huống gần như không thể tái hiện ổn định qua backend thật.

Code mẫu: `code/playwright-tests/tests/ch14-auto-waiting-network.spec.ts` — 3 test minh họa cả ba kỹ thuật trên, và một **bug thật** được phát hiện + sửa ngay trong chương này.

14.1. Auto-waiting chính xác chờ những gì?

Từ Chương 10 đến giờ, sách nhắc nhiều lần "auto-waiting" nhưng chưa giải thích **chính xác** Playwright chờ điều kiện gì trước khi thực hiện một action như `click()`:

1. Phần tử đã được **gắn vào DOM** (attached).
2. Phần tử **hiển thị** (visible — không `display:none`, không `visibility:hidden`).
3. Phần tử **ổn định** (stable — không đang trong animation/di chuyển).
4. Phần tử **có thể nhận sự kiện** (không bị phần tử khác đè lên).
5. Phần tử **không bị disable** (với các action như click, fill).

Playwright kiểm tra lại đủ 5 điều kiện này **liên tục** (poll) cho tới khi tất cả đúng, hoặc hết timeout (mặc định 30 giây cho action, 5 giây cho `expect()`). Đây là lý do test ở `ch14-auto-waiting-network.spec.ts` chờ được network chậm 1.5 giây **mà không cần code thêm gì**:

```
await page.route("**/api/login", async (route) => {
  await new Promise((resolve) => setTimeout(resolve, 1500)); // giả lập mạng chậm 1.5s
  await route.continue();
});
// ...
await expect(page).toHaveURL("/dashboard"); // tự chờ tới 5s mặc định, đủ cho 1.5s giả lập
```

14.2. `page.route()` — chặn và điều khiển network request

`page.route()` cho phép chặn một request khớp pattern (URL hoặc glob), rồi quyết định: cho đi tiếp (`route.continue()`), trả về response giả (`route. fulfill()`), hoặc huỷ (`route.abort()`) — toàn bộ xảy ra ở tầng trình duyệt, **không cần sửa gì ở server thật**.

```
// Giả lập server trả lỗi 500 – không cần code gì trong demo-app để tái hiện tình huống này.  
await page.route("**/api/login", (route) =>  
  route. fulfill({ status: 500, contentType: "text/plain", body: "Internal Server Error"  
});
```

Đây là kỹ thuật mạnh: nhiều tình huống lỗi (server quá tải, mất kết nối giữa chừng, response méo) **rất khó hoặc không thể** tái hiện ổn định bằng cách tác động vào server thật, nhưng lại dễ dàng và tái lập 100% khi giả lập ở tầng network trong test.

14.3. Một bug thật, phát hiện nhờ giả lập lỗi 500

Khi viết test "giả lập lỗi server 500" ở trên, test **fail**:

```
Expected: "Có lỗi xảy ra, vui lòng thử lại."  
Received: ""
```

Đọc lại `public/login.js` (trước khi sửa):

```
const res = await fetch("/api/login", { /* ... */ });  
const data = await res.json(); // ← nếu res không phải JSON hợp lệ, dòng này NÉM LỖI
```

Nguyên nhân: khi server trả về lỗi 500 với body không phải JSON (như test giả lập: `"Internal Server Error"` dạng text thuần), `res.json()` ném lỗi parse — lỗi này **không được bắt** (không có `try/catch`), nên UI không hiển thị gì cả, người dùng thấy nút bấm "không phản hồi" mà không rõ vì sao.

Cách sửa (đã áp dụng trong `public/login.js`):

```

let data;
try {
  const res = await fetch("/api/login", { /* ... */ });
  if (!res.ok) throw new Error("http-error"); // kiểm tra res.ok TRƯỚC KHI gọi
res.json()
  data = await res.json();
} catch {
  serverError.textContent = "Có lỗi xảy ra, vui lòng thử lại.";
  return;
}

```

Đây là bug **thật thứ hai** phát hiện được nhờ automation test trong sách này (bug đầu tiên ở Chương 9, lỗi `aria-label`) — và minh họa chính xác giá trị của `page.route()`: test tay rất khó tái hiện lỗi server 500 một cách chủ động và lặp lại được, nhưng automation test làm được dễ dàng.

14.4. `waitForResponse` — chờ và kiểm tra một request cụ thể

Đôi khi bạn cần **chính xác response** của một request, không chỉ chờ UI cập nhật:

```

const responsePromise = page.waitForResponse("**/api/login"); // đăng ký chờ TRƯỚC khi
trigger

await page.getByRole("button", { name: "Đăng nhập" }).click(); // trigger request

const response = await responsePromise;
expect(response.status()).toBe(200);

const requestBody = JSON.parse(response.request().postData() || "{}");
expect(requestBody.email).toBe("demo@example.com"); // xác nhận đúng dữ liệu đã GỬI lên
server

```

Lưu ý thứ tự: phải gọi `page.waitForResponse()` **trước** khi trigger action gây ra request đó (ở đây là `click()`) — nếu gọi sau, có rủi ro request đã hoàn tất trước khi Playwright kịp "lắng nghe", khiến `await responsePromise` treo mãi. Đây là kỹ thuật nền tảng sẽ dùng lại ở Chương 20 khi viết API test.

Bài tập

1. Chạy `npm test`, xác nhận toàn bộ 25 test pass.
2. Trong `public/login.js`, tạm bỏ khối `try/catch` vừa thêm (quay về code cũ), chạy lại riêng file `ch14-auto-waiting-network.spec.ts` — xác nhận bạn tái hiện đúng lỗi ở mục 14.3. Khôi phục lại sau khi thử xong.

- Viết thêm một test dùng `page.route()` để giả lập tình huống "mất kết nối hoàn toàn" (`route.abort()` thay vì `route.fulfill()`) khi đăng nhập — quan sát UI phản ứng thế nào, và thông báo lỗi hiện tại có phù hợp với tình huống này không.
- Dùng `waitForResponse` để viết một test xác nhận: khi đăng nhập **sai** mật khẩu, request gửi lên `/api/login` vẫn chứa đúng email đã nhập (không bị rỗng hay sai định dạng).

Lỗi thường gặp

- Không bọc `res.json()` trong try/catch, hoặc quên kiểm tra `res.ok`**: xem lỗi thật ở mục 14.3 — đây là lỗi rất phổ biến trong code frontend thực tế, không riêng gì demo app của sách.
- Gọi `page.waitForResponse()` SAU khi đã trigger request**: dễ gây treo test do bỏ lỡ response (xem lưu ý ở mục 14.4).
- Dùng `page.route()` để mock quá nhiều, che mất lỗi thật của backend**: mock lỗi mạng/lỗi server có mục đích cụ thể (kiểm tra UI xử lý lỗi) — không nên mock luôn cả những request cần test với dữ liệu backend thật (như toàn bộ Chương 11-13 đã làm).

Tóm tắt & tiếp theo

- Auto-waiting kiểm tra 5 điều kiện (attached, visible, stable, receives events, enabled) liên tục cho tới khi đúng hoặc hết timeout — đủ để xử lý cả độ trễ mạng thực tế.
- `page.route()` giả lập network (chậm, lỗi, mất kết nối) mà không cần sửa server thật — phát hiện được một bug thật (thiếu xử lý lỗi HTTP trong `login.js`).
- `waitForResponse()` chờ và kiểm tra chính xác một request/response cụ thể — nền tảng cho API testing ở Chương 20.

Chương 15 chuyển sang một chủ đề khác: tổ chức code test bằng Page Object Model — khi số lượng test đã khá nhiều (25 test qua 4 file), đã đến lúc cần một cách tổ chức bài bản hơn để dễ bảo trì.

Chương 15: Page Object Model (POM) — tổ chức code test dễ bảo trì

Mục tiêu học

- Hiểu vấn đề Page Object Model giải quyết, và khi nào nên bắt đầu dùng nó.
- Viết được một Page Object hoàn chỉnh, gói locator + action vào một class.
- So sánh trực tiếp code test trước/sau khi áp dụng POM.

Code mẫu: `code/playwright-tests/pages/LoginPage.ts`, `pages/ProfilePage.ts` + `tests/ch15-page-object-model.spec.ts` — viết lại một phần test Chương 11 bằng POM.

15.1. Vấn đề: locator lặp lại khắp nơi

Mở lại `ch11-login-test-cases.spec.ts` — dòng `page.getByLabel("Email").fill( ... )` xuất hiện trong **9 trên 10 test**. Nếu demo app đổi label "Email" thành "Địa chỉ email", phải sửa ở toàn bộ 9 chỗ. Đây chính xác là "Vấn đề 4" đã học ở Chương 6 (lặp lại logic), nhưng ở quy mô lớn hơn nhiều khi project có hàng chục, hàng trăm test.

Page Object Model (POM) giải quyết vấn đề này: gom toàn bộ locator + action của **một trang** vào **một class**, test chỉ gọi phương thức của class đó, không biết (và không cần biết) chi tiết locator bên trong.

15.2. Viết một Page Object

```
// pages/LoginPage.ts
import { Page } from "@playwright/test";

export class LoginPage {
  constructor(private page: Page) {}

  async goto() {
    await this.page.goto("/login");
  }

  async login(email: string, password: string) {
    await this.page.getByLabel("Email").fill(email);
    await this.page.getByLabel("Mật khẩu").fill(password);
    await this.page.getByRole("button", { name: "Đăng nhập" }).click();
  }

  get serverError() {
    return this.page.locator("#server-error");
  }
}
```

Ba nguyên tắc khi viết Page Object:

1. **Constructor nhận `page`**, lưu lại để dùng trong các phương thức khác.
2. **Action phức hợp** (như `login()` — gồm 2 lần fill + 1 click) trở thành **một phương thức duy nhất**, đặt tên theo *ý định* ("login"), không phải theo *thao tác* ("fillAndClick").
3. **Locator hay dùng để assert** (như `serverError`) expose qua **getter**, để test gọi `LoginPage.serverError` như một thuộc tính, gọn hơn gọi lại `page.locator("#server-error")`.

15.3. So sánh trực tiếp: trước và sau

Trước (Chương 11, không có POM):

```
test("TC-02: đăng nhập thất bại với sai mật khẩu", async ({ page }) => {
  await page.getByLabel("Email").fill("demo@example.com");
  await page.getByLabel("Mật khẩu").fill("SaiRoi123");
  await page.getByRole("button", { name: "Đăng nhập" }).click();

  await expect(page).toHaveURL(/\/login/);
  await expect(page.getByText("Email hoặc mật khẩu không đúng")).toBeVisible();
});
```

Sau (Chương 15, dùng `LoginPage`):

```
test("TC-02 (viết lại bằng POM): sai mật khẩu hiển thị lỗi", async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login("demo@example.com", "SaiRoi123");

  await expect(loginPage.serverError).toHaveText("Email hoặc mật khẩu không đúng");
});
```

Test ngắn hơn, đọc gần như mô tả bằng lời ("vào trang login, đăng nhập sai, kiểm tra thông báo lỗi"), và **không còn biết** form dùng `getByLabel` hay cấu trúc DOM nào — chi tiết đó chỉ nằm trong `LoginPage.ts`.

15.4. Kết hợp nhiều Page Object trong một test

```
test("kết hợp 2 Page Object: đăng nhập rồi cập nhật hồ sơ", async ({ page }) => {
  const loginPage = new LoginPage(page);
  const profilePage = new ProfilePage(page);

  await loginPage.goto();
  await loginPage.login("demo@example.com", "Password123!");

  await profilePage.goto();
  await profilePage.fillDisplayName("Trần Văn A (qua POM)");
  await profilePage.save();

  await expect(profilePage.serverSuccess).toHaveText("Đã lưu hồ sơ.");
});
```

Mỗi trang một Page Object riêng — test luồng đi qua nhiều trang chỉ cần khởi tạo nhiều instance và gọi phương thức tương ứng, không lẫn lộn locator giữa các trang.

15.5. Khi nào NÊN dùng POM, khi nào chưa cần?

POM có chi phí: thêm một lớp trừu tượng, thêm file phải bảo trì. Với 1-2 test đơn giản (như `ch08-smoke.spec.ts`), viết trực tiếp vẫn rõ ràng hơn. POM đáng giá khi:

- Cùng một trang được test bởi **hiều test khác nhau** (như trang login — 10+ test qua các chương).
- Nhóm làm việc **từ 2 người trở lên**, cần một "hợp đồng" rõ ràng giữa locator và logic test.

- Ứng dụng đang test có khả năng **thay đổi UI thường xuyên** — POM giới hạn phạm vi phải sửa khi UI đổi.

Bài tập

1. Chạy `npm test`, xác nhận toàn bộ 28 test pass.
2. Viết thêm phương thức `loginWithEnter` (đã có sẵn trong `LoginPage.ts` nhưng chưa được test nào gọi tới) — dùng nó viết lại test TC-09 (Chương 11) theo phong cách POM.
3. Trong `pages/ProfilePage.ts`, thêm getter `bioTextarea` trả về locator của trường "Giới thiệu", viết một test mới xác nhận giá trị textarea sau khi `fillBio()`.
4. Thử đổi label "Mật khẩu" trong `code/demo-app/src/views.js` thành "Mật khẩu đăng nhập", chạy lại `npm test` — đếm xem có bao nhiêu chỗ trong `pages/LoginPage.ts` cần sửa, so với số chỗ sẽ phải sửa nếu **không** dùng POM (đếm trong `ch11-login-test-cases.spec.ts`). Đổi lại label như cũ sau khi thử xong.

Lỗi thường gặp

- **Nhét cả assertion vào bên trong Page Object:** Page Object nên chỉ chứa locator + action, để **test** tự quyết định assert gì — nhét sẵn `expect()` vào Page Object làm giảm tính linh hoạt khi các test khác nhau cần kiểm tra những điều khác nhau.
- **Tạo Page Object cho từng trang dù chỉ có 1 test dùng tới:** chi phí bảo trì không đáng so với lợi ích — xem mục 15.5.
- **Page Object gọi lẫn vào Page Object khác:** giữ mỗi Page Object độc lập, để test (không phải Page Object) là nơi điều phối luồng qua nhiều trang (như ví dụ mục 15.4).

Tóm tắt & tiếp theo

- Page Object Model gom locator + action của một trang vào một class, giải quyết vấn đề lặp lại locator ở nhiều test.
- Không phải lúc nào cũng cần POM — đáng giá nhất khi nhiều test dùng chung một trang, hoặc làm việc nhóm.
- Page Object chỉ nên chứa locator/action, không chứa assertion.

Chương 16 học fixtures — một cơ chế khác của Playwright để tái sử dụng logic setup (như "đăng nhập sẵn" đã lặp lại ở nhiều `beforeEach` từ Chương 12 tới giờ), tinh gọn hơn nữa so với gọi tay `loginPage.login()` trong mỗi `beforeEach`.

Chương 16: Fixtures cơ bản, tái sử dụng logic test

Mục tiêu học

- Hiểu fixture là gì, khác gì với hook (`beforeEach`) đã học ở Chương 12.
- Viết được một custom fixture đơn giản, có cả phần setup và teardown.
- Biết khi nào dùng fixture, khi nào dùng hook/Page Object đã đủ.

Code mẫu: `code/playwright-tests/fixtures.ts` (fixture `loggedInPage`) + `tests/ch16-fixtures-co-ban.spec.ts`.

16.1. Fixture khác hook như thế nào?

Chương 12 dùng `test.beforeEach` để đăng nhập trước mỗi test — **mọi** test trong `describe` đó đều phải đăng nhập, dù có cần hay không. **Fixture** giải quyết khác: mỗi test **tự khai báo** fixture nào nó cần (qua tên tham số), và Playwright chỉ chạy đúng phần setup của fixture đó — test không khai báo thì không tốn chi phí chạy fixture.

```
// Trước (Chương 12): MỌI test trong describe đều bị đăng nhập, dù cần hay không
test.describe("...", () => {
  test.beforeEach(async ({ page }) => { /* đăng nhập ... */ });
  test("test A", async ({ page }) => { /* ... */ });
  test("test B", async ({ page }) => { /* ... */ });
});

// Sau (Chương 16): test nào cần đăng nhập thì khai báo 'loggedInPage', test nào không
// cần thì không
test("test A cần đăng nhập", async ({ loggedInPage }) => { /* ... */ });
test("test B không cần đăng nhập", async ({ page }) => { /* ... */ });
```

16.2. Viết một custom fixture

```
// fixtures.ts
import { test as base, expect } from "@playwright/test";
import { LoginPage } from "../pages/LoginPage";

type Fixtures = {
  loggedInPage: import("@playwright/test").Page;
};

export const test = base.extend<Fixtures>({
  loggedInPage: async ({ page, request }, use) => {
    await request.post("/api/test/reset"); // --- setup ---
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await loginPage.login("demo@example.com", "Password123!");

    await use(page); // --- test chạy ở đây ---

    await request.post("/api/test/reset"); // --- teardown ---
  },
});

export { expect };
```

Ba điểm quan trọng:

1. `base.extend<Fixtures>({ ... })` tạo ra **một test mới**, có thêm fixture `loggedInPage`, bên cạnh các fixture gốc của Playwright (`page`, `request`, `context` ...) — vẫn dùng được như cũ.
2. **Mọi thứ trước** `await use(page)` là phần setup — chạy trước khi test bắt đầu.
3. **Mọi thứ sau** `await use(page)` là phần teardown — chạy sau khi test kết thúc (Playwright đảm bảo chạy dù test pass hay fail), tương tự vai trò của `afterEach`.

Trong toàn bộ project, phải **import test / expect từ** `../fixtures` (không phải trực tiếp từ `@playwright/test`) để có được fixture `loggedInPage`:

```
import { test, expect } from "../fixtures"; // KHÔNG phải "@playwright/test"
```

16.3. So sánh với Page Object Model (Chương 15)

Fixture và Page Object **giải quyết hai vấn đề khác nhau, dùng chung với nhau tốt**:

	Page Object (Chương 15)	Fixture (Chương 16)
Giải quyết	Gói locator/action của MỘT TRANG	Tái sử dụng logic SETUP/TEARDOWN cho MỖI TEST
Test vẫn phải gọi	<code>await loginPage.login(...)</code> tường minh trong từng test	Không cần gọi gì — chỉ cần khai báo tham số <code>loggedInPage</code>
Ví dụ trong sách	<code>pages/LoginPage.ts</code>	<code>fixtures.ts</code> — bên trong vẫn dùng <code>LoginPage</code> để login

Nhìn vào `fixtures.ts`: nó dùng chính `LoginPage` (Chương 15) bên trong phần setup — hai kỹ thuật không thay thế nhau, mà **xếp lớp lên nhau**: Page Object gói locator, fixture gói khi-nào-chạy phần setup đó.

16.4. Test không cần fixture vẫn dùng `page` như thường

```
test("trang login hiển thị đúng tiêu đề (không cần fixture đăng nhập)", async ({ page })
⇒ {
  await page.goto("/login");
  await expect(page.getByRole("heading", { name: "Đăng nhập" })).toBeVisible();
});
```

Vì `base.extend()` chỉ **thêm** fixture mới, không xóa fixture gốc — `page`, `request`, `context` vẫn dùng được nguyên vẹn trong file đã import `test` từ `../fixtures`.

Bài tập

1. Chạy `npm test`, xác nhận toàn bộ 31 test pass.
2. Thêm một `console.log` vào ngay trước và ngay sau `await use(page)` trong `fixtures.ts`, chạy riêng `ch16-fixtures-co-ban.spec.ts` — quan sát thứ tự log để xác nhận đúng 3 test, phần setup chạy đúng 3 lần (một lần cho 2 test dùng `loggedInPage`, không chạy cho test thứ 3).
3. Viết thêm một fixture `profilePage` trong `fixtures.ts`, trả về một `ProfilePage` instance đã ở sẵn trang `/profile` (gợi ý: fixture này có thể **phụ thuộc vào** `loggedInPage` — khai báo nó trong object tham số đầu của hàm fixture).
4. Refactor `ch12-cau-truc-test.spec.ts` (giữ nguyên hành vi) để dùng fixture `loggedInPage` thay cho `test.beforeEach` — so sánh độ dài code trước/sau.

Lỗi thường gặp

- **Import `test` từ `@playwright/test` thay vì từ `./fixtures`**: fixture tùy biến (`loggedInPage`) sẽ không tồn tại, TypeScript báo lỗi ngay khi cố khai báo tham số đó.
- **Quên `await use(page)`**: test sẽ không bao giờ chạy — hàm fixture "treo" ở bước setup mãi mãi.
- **Đặt phần `teardown` TRƯỚC `use()` thay vì sau**: `teardown` chạy trước khi test kịp thực thi, không có ý nghĩa gì — luôn nhớ thứ tự: `setup` → `use()` → `teardown`.
- **Dùng fixture cho logic chỉ 1 test cần**: nếu chỉ một test cần một đoạn setup riêng biệt, viết trực tiếp trong test đó rõ ràng hơn là tạo fixture mới cho riêng nó.

Tóm tắt & tiếp theo

- Fixture cho phép mỗi test tự khai báo phụ thuộc (dependency) nó cần, tránh chạy setup thừa cho test không cần.
- Cấu trúc `setup → await use( ... ) → teardown` là khuôn mẫu chung của mọi custom fixture.
- Fixture và Page Object bổ trợ nhau: Page Object gói locator/action, fixture gói vòng đời `setup/teardown`.

Chương 17 chuyển sang kỹ năng debug: Trace Viewer, UI Mode, và cách đọc lỗi khi một trong 31 test hiện có bất ngờ fail.

Chương 17: Debug — Trace Viewer, UI Mode, `--debug`, `codegen`

Mục tiêu học

- Xem lại từng bước một test đã chạy bằng Trace Viewer, kể cả sau khi test đã chạy xong.
- Dùng UI Mode để chạy và debug test tương tác trong lúc viết.
- Dùng `--debug` để dừng lại từng bước, và `codegen` để sinh code test bằng cách thao tác tay.

Chương này không thêm test mới — tận dụng lại 31 test đã có ở `code/playwright-tests/` (đặc biệt hai lần fail thật ở Chương 11 và Chương 14) để minh hoạ cách debug đúng.

17.1. Trace Viewer — "hộp đen" ghi lại mọi thứ đã xảy ra

Trace là một bản ghi đầy đủ những gì xảy ra trong lúc chạy test: từng action, ảnh chụp màn hình trước/sau mỗi bước, network request, console log — đủ để xem lại **sau khi test đã chạy xong**, kể cả trên máy khác (ví dụ tải trace từ CI về xem ở máy local).

```
npm run test:trace # chạy toàn bộ test, ép ghi trace cho MỌI test (mặc định chỉ ghi khi retry)
npm run show-trace test-results/<tên-thư-mục-test>/trace.zip
```

Mở trace lên, bạn thấy:

- **Timeline** các action theo thứ tự, click vào từng action thấy ảnh chụp màn hình đúng lúc đó.
- **Tab Network** — mọi request/response, đúng thứ tự — hữu ích để debug các tình huống như Chương 14 (kiểm tra request thật gửi đi có đúng dữ liệu không).
- **Tab Console** — log JavaScript phía trình duyệt (ví dụ nếu `public/login.js` có `console.log`).
- **Tab Source** — dòng code test tương ứng với action đang xem.

Cấu hình mức ghi trace

```
// playwright.config.ts (đã có từ Chương 9)
use: {
  trace: "on-first-retry", // chỉ ghi trace khi 1 test fail và được retry lần đầu
},
```

Giá trị	Khi nào ghi trace	Phù hợp cho
"off"	Không bao giờ	Không khuyến nghị
"on-first-retry"	Chỉ khi test fail và retry (mặc định của sách)	Chạy hàng ngày/CI — tiết kiệm dung lượng, vẫn có trace khi cần
"retain-on-failure"	Mọi test fail (không cần retry)	Debug cục bộ khi chưa cấu hình retry
"on"	MỌI test, pass hay fail	Chỉ dùng tạm để học/debug sâu — tốn dung lượng đáng kể

17.2. Ứng dụng thực tế: đọc lại 2 lần fail thật đã gặp

Nếu bạn đã làm theo bài tập ở Chương 11 (tạm bỏ `workers: 1`) hoặc Chương 14 (tạm bỏ `try/catch` trong `login.js`), bạn đã tự tạo ra 2 lần fail thật. Với trace của một lần fail như vậy, quy trình debug chuẩn là:

1. Mở tab **Network** trước — với lỗi ở Chương 14 (500 error), bạn sẽ thấy ngay request `/api/login` trả về status `500`, body không phải JSON — xác nhận đúng giả thuyết ban đầu.
2. Xem **action ngay trước khi fail** trong timeline — với lỗi ở Chương 11 (TC-07 flaky do race condition), bạn sẽ thấy bước `expect().toBeVisible()` timeout vì phần tử không tồn tại — nhưng trace của MỘT test đơn lẻ sẽ không tự lộ ra nguyên nhân (test khác chạy song song can thiệp) — đây là lúc kỹ năng suy luận từ Chương 6 (hiểu hệ thống) quan trọng hơn công cụ.
3. Đọc **Console tab** — nếu code frontend có lỗi JS không bắt được (như lỗi gốc ở Chương 14), thường sẽ thấy "Uncaught (in promise) SyntaxError..." ở đây — dấu hiệu trực tiếp dẫn tới nguyên nhân.

17.3. UI Mode — chạy và debug tương tác trong lúc viết

```
npm run test:ui
```

UI Mode mở một giao diện riêng: danh sách test bên trái, chạy từng test hoặc cả file, xem trace **ngay lập tức** không cần chờ tạo file `.zip` riêng, và có nút "pick locator" để click vào một phần tử trên trang và lấy ngay locator Playwright khuyến nghị cho nó — đặc biệt hữu ích khi đang viết test mới, không phải khi debug test cũ đã fail.

17.4. `--debug` — dừng lại từng bước

```
npm run test:debug -- ch11-login-test-cases
```

Chạy với `--debug` mở **Playwright Inspector**: test dừng lại **trước mỗi action**, cho phép bạn bấm "Step over" để chạy tiếp từng bước một, hoặc gõ lệnh Playwright trực tiếp vào ô lệnh để thử locator ngay trên trang đang mở. Khác UI Mode (thiết kế để chạy nhanh nhiều test), `--debug` phù hợp khi cần hiểu **chính xác** vì sao một bước cụ thể không hoạt động như mong đợi.

17.5. `codegen` — sinh code bằng cách thao tác tay

```
npx playwright codegen http://localhost:3000/login
```

Lệnh này mở một trình duyệt thật, kèm cửa sổ Playwright Inspector — mọi cú click, gõ chữ của bạn trên trang được **tự động chuyển thành code Playwright**, hiện ra ở cửa sổ inspector. Ví dụ, click vào email, gõ `demo@example.com`, click vào password, gõ mật khẩu, click nút đăng nhập, `codegen` sẽ sinh ra gần đúng:

```
await page.getByLabel('Email').click();
await page.getByLabel('Email').fill('demo@example.com');
await page.getByLabel('Mật khẩu').click();
await page.getByLabel('Mật khẩu').fill('Password123!');
await page.getByRole('button', { name: 'Đăng nhập' }).click();
```

Quan trọng: code sinh ra bởi `codegen` là **điểm khởi đầu tốt, không phải sản phẩm cuối**. So với code thật trong `ch11-login-test-cases.spec.ts`, code trên thiếu: cấu trúc test có tên rõ nghĩa (TC-01...), thiếu assertion (`codegen` không tự biết bạn muốn kiểm tra gì), và có thể có những dòng thừa (như `.click()` trước `.fill()` — không cần thiết vì `fill()` đã tự focus vào input). Luôn **đọc lại và tinh gọn** code từ `codegen`, áp dụng các nguyên tắc đã học ở Chương 6 và 11, trước khi coi nó là test hoàn chỉnh.

Bài tập

1. Chạy `npm run test:trace` trong `code/playwright-tests/`, sau đó `npm run show-trace` với một file `trace.zip` bất kỳ trong `test-results/` — xem qua Timeline, Network, Console.
2. Chạy `npm run test:ui`, thử dùng nút "pick locator" trên trang `/profile` — so sánh locator nó gợi ý với locator đã viết tay trong `pages/ProfilePage.ts`.

3. Chạy `npm run test:debug -- ch13-form-input-upload`, dùng "Step over" đi qua từng bước của test "upload ảnh đại diện" — quan sát ảnh trong `#avatar-filename` thay đổi ra sao qua từng bước.
4. Chạy `npx playwright codegen http://localhost:3000/login` (yêu cầu demo app đang chạy — `cd code/demo-app && npm start` ở terminal khác), thử đăng nhập tay, so sánh code sinh ra với `pages/LoginPage.ts`.

Lỗi thường gặp

- **Đặt `trace: "on"` cho chạy CI hàng ngày:** sinh ra dung lượng trace khổng lồ cho hàng trăm test pass — chỉ nên dùng tạm thời khi cần debug sâu, không để mặc định lâu dài.
- **Coi code từ `codegen` là sản phẩm cuối, không tinh gọn lại:** dẫn đến test khó đọc, thiếu assertion có ý nghĩa, hoặc dùng locator không tối ưu (xem mục 17.5).
- **Debug bằng cách đọc code trước, xem trace sau:** nên làm ngược lại — xem trace (đặc biệt tab Network và Console) để có giả thuyết trước, rồi mới quay lại đọc code để xác nhận.

Tóm tắt & tiếp theo

- Trace Viewer ghi lại đầy đủ diễn biến một lần chạy test, xem lại được sau khi test đã chạy xong.
- UI Mode phù hợp khi đang viết/chạy nhiều test; `--debug` phù hợp khi cần dừng lại từng bước.
- `codegen` sinh code nhanh bằng thao tác tay, nhưng luôn cần tinh gọn lại theo các nguyên tắc đã học (Chương 6, 11) trước khi dùng thật.

Chương 18 học cách trình bày kết quả test — reporter HTML đã dùng ngầm định từ đầu, và cách tích hợp Allure để có báo cáo chi tiết hơn, phù hợp chia sẻ với cả người không đọc code.

Chương 18: Reporting — HTML reporter, tích hợp Allure

Mục tiêu học

- Hiểu cấu trúc báo cáo HTML mặc định của Playwright, cách chia sẻ nó cho người khác.
- Cấu hình nhiều reporter cùng lúc.
- Tích hợp Allure — reporter phổ biến trong doanh nghiệp, phù hợp chia sẻ với người không đọc code (PM, PQA — nhắc lại Chương 4).

Code mẫu: `playwright.config.ts` (cấu hình 2 reporter cùng lúc) — `code/playwright-tests/`.

18.1. HTML reporter — đã dùng từ Chương 9

Từ đầu sách, `playwright.config.ts` đã có `reporter: "html"`. Sau khi chạy `npm test`, mở báo cáo:

```
npm run report
```

Báo cáo HTML hiển thị: danh sách test theo file, pass/fail/skip, thời gian chạy từng test, và với test fail — ảnh chụp màn hình, trace đính kèm trực tiếp (không cần mở riêng bằng `show-trace`). Đây là báo cáo **tốt cho người viết test**, nhưng khá "kỹ thuật" — không phải lựa chọn tốt nhất khi cần gửi báo cáo cho PM hoặc PQA không quen đọc log test.

18.2. Nhiều reporter cùng lúc

```
// playwright.config.ts
reporter: [["html"], ["allure-playwright"]],
```

Playwright cho phép khai báo **một danh sách reporter**, chạy song song — không phải chọn một trong hai. Ở đây, `html` vẫn sinh báo cáo như cũ, đồng thời `allure-playwright` ghi thêm dữ liệu thô vào thư mục `allure-results/` để xử lý ở bước tiếp theo.

18.3. Tích hợp Allure

Allure là công cụ báo cáo test phổ biến trong doanh nghiệp, không riêng gì Playwright (cũng dùng được với JUnit, TestNG, Cypress...) — tạo báo cáo HTML đẹp, có thể nhóm test theo "feature", gắn severity, lịch sử pass/fail qua nhiều lần chạy.

Bước 1 — cài reporter (đã làm, chạy trong Node.js, không cần gì thêm):

```
npm install -D allure-playwright
```

Bước 2 — chạy test như thường:

```
npm test
```

Sau bước này, `allure-results/` chứa các file `.json` thô (mỗi test 1 file `*-result.json`) — đây là dữ liệu Allure cần, **đã sinh ra thành công chỉ với Node.js**, không cần công cụ nào khác.

Bước 3 — sinh báo cáo HTML từ dữ liệu thô (cần cài thêm Allure CLI):

```
npm run allure:generate # allure generate allure-results --clean -o allure-report
npm run allure:open     # allure open allure-report
```

Lưu ý quan trọng: hai lệnh ở Bước 3 cần **Allure CLI** đã cài sẵn trên máy (qua `npm install -g allure-commandline`, Homebrew/Scoop, hoặc tải trực tiếp) — và Allure CLI (bản phổ biến hiện tại) cần **Java Runtime (JRE)** để chạy. Đây là bước cài đặt **tách biệt** khỏi Node.js/npm, không tự động có sẵn dù bạn đã cài Playwright đầy đủ.

18.4. Vì sao tách riêng "sinh dữ liệu" và "sinh báo cáo"?

Đây là kiến trúc phổ biến trong hệ sinh thái Allure: `allure-playwright` (chạy trong Node.js) chỉ làm nhiệm vụ **ghi lại dữ liệu thô** khi test chạy — nhẹ, không phụ thuộc Java. Việc **dựng** dữ liệu đó thành trang HTML tương tác là công việc của Allure CLI (viết bằng Java) — tách riêng để `allure-playwright` không phải kéo theo cả một JVM runtime vào mọi project Node.js chỉ để ghi log.

Trong CI/CD (Chương 19), bước cài Java + Allure CLI thường được thêm vào workflow như một bước riêng, trước khi chạy `allure generate`.

Bài tập

1. Chạy `npm test` trong `code/playwright-tests/`, xác nhận thư mục `allure-results/` được tạo ra với nhiều file `*-result.json` (một cho mỗi test).
2. Mở một file `.json` trong `allure-results/` bằng editor — tìm trường `status` và `name`, xác nhận nó khớp với một test bạn biết trong `tests/`.
3. Nếu máy bạn có Java: cài Allure CLI (`npm install -g allure-commandline` hoặc qua Homebrew/Scoop), chạy `npm run allure:generate && npm run allure:open`, xem báo cáo HTML thực sự sinh ra.
4. So sánh báo cáo HTML mặc định của Playwright (`npm run report`) với những gì bạn đọc được về Allure ở mục 18.3 — với đối tượng "PM không đọc code", báo cáo nào bạn sẽ chọn gửi, và vì sao?

Lỗi thường gặp

- **Tưởng chỉ cần** `npm install -D allure-playwright` **là có báo cáo HTML Allure ngay**: package này chỉ sinh dữ liệu thô — cần thêm Allure CLI (và Java) mới ra được báo cáo HTML cuối cùng, xem mục 18.3-18.4.
- **Commit** `allure-results/` / `allure-report/` **vào git**: đây là dữ liệu sinh ra từ mỗi lần chạy test, không nên commit — đã được thêm vào `.gitignore` cùng `test-results/`, `playwright-report/`.
- **Chạy** `allure generate` **nhiều lần mà không có** `--clean`: dữ liệu cũ và mới trộn lẫn trong `allure-report/`, gây hiểu sai lịch sử — script `allure:generate` trong `package.json` đã có sẵn cờ này.

Tóm tắt & tiếp theo

- Playwright hỗ trợ khai báo nhiều reporter cùng lúc (`reporter: [[ ... ], [ ... ]]`).
- `allure-playwright` sinh dữ liệu thô chỉ cần Node.js; sinh báo cáo HTML cuối cùng cần Allure CLI (và Java) — hai bước tách biệt.
- Chọn reporter theo đối tượng nhận báo cáo: HTML mặc định cho người viết test, Allure cho việc chia sẻ rộng hơn (PM, PQA).

Chương 19 — chương cuối của Phần IV — tích hợp toàn bộ 31 test đã có vào một pipeline CI/CD thật bằng GitHub Actions, chạy tự động mỗi khi có commit mới.

Chương 19: CI/CD — chạy Playwright trên GitHub Actions

Mục tiêu học

- Hiểu vì sao automation test cần chạy tự động trên CI, không chỉ trên máy cá nhân.
- Viết được một workflow GitHub Actions hoàn chỉnh, chạy đúng 31 test đã có.
- Biết cách lưu lại báo cáo HTML làm artifact để xem sau khi CI chạy xong.

Code mẫu: `.github/workflows/playwright.yml` — workflow thật, đã kiểm chứng từng bước chạy đúng cục bộ (`npm ci` sạch từ lockfile, `npm test` pass 31/31) trước khi đưa vào CI.

19.1. Vì sao cần CI, không chỉ chạy trên máy cá nhân?

Nhắc lại Chương 1 (SDLC) và Chương 2 (Regression testing): giá trị lớn nhất của automation test là **chạy lại liên tục, tự động**, mỗi khi có code mới — không phải chạy một lần rồi thôi. Nếu test chỉ chạy khi ai đó nhớ chạy tay trên máy mình, giá trị đó gần như mất hết: một PR có thể merge dù làm hỏng tính năng khác, chỉ vì không ai chạy lại test trước khi merge.

CI (Continuous Integration) giải quyết đúng vấn đề này: mỗi khi có commit/PR mới, một máy chủ (không phải máy cá nhân) tự động clone code, cài đặt, và chạy toàn bộ test suite — kết quả hiển thị ngay trên PR, không ai có thể "quên chạy test" nữa.

19.2. Workflow GitHub Actions hoàn chỉnh

```
# .github/workflows/playwright.yml
name: Playwright Tests

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22

      - name: Install demo-app dependencies
        working-directory: code/demo-app
        run: npm ci

      - name: Install playwright-tests dependencies
        working-directory: code/playwright-tests
        run: npm ci

      - name: Install Playwright browsers (Chromium only)
        working-directory: code/playwright-tests
        run: npx playwright install --with-deps chromium

      - name: Type-check
        working-directory: code/playwright-tests
        run: npm run typecheck

      - name: Run Playwright tests
        working-directory: code/playwright-tests
        run: npm test

      - name: Upload HTML report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: playwright-report
```

```
path: code/playwright-tests/playwright-report/  
retention-days: 7
```

Đi qua từng phần:

- **on:** — chạy khi có push hoặc pull request vào nhánh `main`. Đây là cấu hình phổ biến nhất: test chạy trên mọi PR **trước khi** merge, không phải sau.
- **Hai bước `npm ci` riêng biệt** — vì project có 2 package.json độc lập (`demo-app` và `playwright-tests`), nhắc lại quyết định kiến trúc ở Chương 9). `npm ci` (khác `npm install`) cài **chính xác** phiên bản trong `package-lock.json`, không tự nâng cấp gì — đảm bảo CI dùng đúng phiên bản đã test trên máy dev.
- `npx playwright install --with-deps chromium` — cờ `--with-deps` tự cài thêm các thư viện hệ thống Linux mà Chromium cần (không có sẵn trên máy ảo CI trần) — cờ này chỉ cần trên Linux, không cần trên máy Windows/macOS cá nhân.
- **`if: always()`** ở bước upload — chạy dù bước trước đó pass hay fail, để **luôn** có báo cáo tải về xem, đặc biệt quan trọng khi test fail.
- `npm run typecheck` (bổ sung sau khi rà soát lại sách ở Chương 30, dùng `tsconfig.json` — xem lại Chương 8) — chạy `tsc --noEmit` trước khi chạy test thật, bắt lỗi type (ví dụ gọi sai tham số một hàm) **nhANH hơn** nhiều so với để lỗi đó gây fail một test cụ thể rồi mới đi tìm nguyên nhân.

19.3. `webServer` hoạt động thế nào trên CI?

Nhắc lại Chương 9: `playwright.config.ts` có cấu hình `webServer` tự khởi động `demo-app`. Trên CI, `reuseExistingServer: !process.env.CI` sẽ là `false` (vì GitHub Actions tự đặt biến môi trường `CI=true`) — nghĩa là Playwright **luôn tự khởi động** demo app mới trên CI, không bao giờ giả định đã có sẵn server chạy từ trước (khác lúc dev cục bộ, có thể đang chạy `npm start` sẵn ở terminal khác).

19.4. `forbidOnly` và `retries` — hai cấu hình chỉ có ý nghĩa trên CI

```
// playwright.config.ts - đã có từ Chương 9  
forbidOnly: !!process.env.CI, // chặn merge nếu code còn sót test.only()  
retries: process.env.CI ? 2 : 0, // tự chạy lại 2 lần nếu fail - giảm ảnh hưởng flaky test
```

- `forbidOnly`: nếu ai lỡ để `test.only( ... )` trong code (chỉ chạy 1 test, bỏ qua test khác) và commit nhầm, CI sẽ **fail ngay** thay vì âm thầm bỏ qua toàn bộ test khác — bắt lỗi này trước khi merge.
- `retries`: chỉ bật trên CI, không bật khi dev cục bộ — vì lúc dev, bạn **muốn** thấy fail ngay để sửa, không muốn chờ retry 2 lần mới báo lỗi.

Bài tập

1. Đọc lại `.github/workflows/playwright.yml`, xác định: nếu bước "Run Playwright tests" fail, các bước sau nó có chạy tiếp không? Vì sao bước "Upload HTML report" vẫn cần chạy trong trường hợp đó?
2. Thử tạo một repository GitHub thật (hoặc dùng chính repo của sách này nếu đã đẩy lên GitHub), push code lên, mở tab "Actions" — xác nhận workflow chạy và cho kết quả 31 test pass.
3. Cố tình gây fail một test (ví dụ sửa sai một assertion), push lên, quan sát: workflow có báo fail đúng không, và artifact `playwright-report` có tải về xem được không?
4. Thêm một step mới vào workflow, chạy `npm run allure:generate` (Chương 18) — cần thêm bước cài Java trước đó (gợi ý: tìm action `actions/setup-java` trên GitHub Marketplace).

Lỗi thường gặp

- **Quên `npm ci` riêng cho cả `demo-app` VÀ `playwright-tests`**: chỉ cài một trong hai sẽ khiến `webServer` không khởi động được (thiếu `node_modules` của `demo-app`) — lỗi timeout khó hiểu nếu không biết nguyên nhân.
- **Dùng `npm install` thay vì `npm ci` trên CI**: có thể cài phiên bản khác với máy dev do `package-lock.json` không được tôn trọng chặt như `npm ci`, gây lỗi "chạy được trên máy tôi nhưng fail trên CI".
- **Không đặt `timeout-minutes`**: nếu `webServer` treo vì lý do nào đó, job có thể chạy vô thời hạn (tới giới hạn mặc định của GitHub, thường 6 giờ) — luôn đặt timeout hợp lý.

Tóm tắt & tiếp theo

- CI/CD đảm bảo automation test chạy tự động mỗi khi có code mới, không phụ thuộc việc ai đó nhớ chạy tay.
- Workflow GitHub Actions cho project 2 package.json cần `npm ci` riêng cho từng thư mục.
- `forbidOnly` và `retries` là hai cấu hình đặc biệt cho môi trường CI, khác với chạy cục bộ.

Phần IV (Playwright cơ bản) đến đây đã hoàn thành — 19 chương, 31 test, một demo app đầy đủ tính năng, một pipeline CI thật. Phần V bắt đầu các kỹ thuật nâng cao: API testing, visual testing, authentication/storage state, chạy song song, cross-browser, và component testing.

Phần V — Playwright nâng cao

Chương 20: API testing với `request` context (kết hợp UI + API test)

Mục tiêu học

- Viết API test thuần bằng fixture `request`, không cần mở trình duyệt.
- Phân biệt chính xác `request` (fixture độc lập) và `page.request` (dùng chung context với `page`).
- Dùng API để chuẩn bị trạng thái (đăng nhập) nhanh hơn, chỉ dùng UI cho phần thực sự cần kiểm tra qua trình duyệt.

Code mẫu: `code/playwright-tests/tests/ch20-api-testing.spec.ts` — 4 test, gồm một cặp đối chứng trực tiếp chứng minh sự khác biệt giữa `request` và `page.request`.

20.1. API test thuần — không cần trình duyệt

```
test("POST /api/login trả về ok:true với thông tin đúng", async ({ request }) => {
  const res = await request.post("/api/login", {
    data: { email: "demo@example.com", password: "Password123!" },
  });

  expect(res.status()).toBe(200);
  const body = await res.json();
  expect(body.ok).toBe(true);
});
```

Fixture `request` (khác `page`) gọi trực tiếp HTTP request, không mở Chromium/Firefox/WebKit nào — nhanh hơn nhiều so với test UI cho cùng một hành vi. Đây là lý do Testing Pyramid (Chương 2) khuyến nghị nhiều integration/API test hơn E2E test: cùng kiểm tra logic đăng nhập, API test chạy trong vài chục milli-giây, còn test UI (Chương 11) cần vài giây để mở trình duyệt, điền form, chờ điều hướng.

20.2. `request` fixture độc lập KHÔNG chia sẻ cookie với `page`

```
test("request fixture độc lập KHÔNG chia sẻ cookie với page", async ({ page, request })
⇒ {
  const res = await request.post("/api/login", {
    data: { email: "demo@example.com", password: "Password123!" },
  });
  expect((await res.json()).ok).toBe(true);

  await page.goto("/dashboard");
  await expect(page).toHaveURL(/\/login/); // vẫn bị coi là CHƯA đăng nhập!
});
```

Đây là điểm dễ gây nhầm lẫn nhất khi mới học API testing với Playwright: `request` là một **APIRequestContext hoàn toàn riêng biệt**, không liên quan gì tới trình duyệt mà `page` đang điều khiển. Cookie session mà `/api/login` trả về (qua `Set-Cookie`) chỉ được `request` context đó lưu lại nội bộ — `page` không hề biết gì về nó.

20.3. `page.request` — dùng chung context với `page`

```
test("page.request CÓ chia sẻ cookie với page", async ({ page }) ⇒ {
  const res = await page.request.post("/api/login", {
    data: { email: "demo@example.com", password: "Password123!" },
  });
  expect((await res.json()).ok).toBe(true);

  await page.goto("/dashboard"); // Đã có cookie session, vào thẳng Dashboard
  await expect(page.locator("#username")).toHaveText("demo@example.com");
});
```

`page.request` (khác `request` độc lập) là APIRequestContext **gắn với cùng browser context** mà `page` đang dùng — cookie nhận được từ response sẽ được lưu vào browser context đó, nên `page.goto()` ngay sau sẽ mang theo cookie này như một request thật từ trình duyệt.

20.4. Vì sao kỹ thuật này quan trọng?

So sánh hai cách chuẩn bị "đã đăng nhập" cho một test chỉ cần test tính năng **khác** đăng nhập (ví dụ test trang Đơn hàng):

Cách	Thời gian	Rủi ro
Điền form UI (Chương 11-15)	Vài giây (mở form, điền, chờ điều hướng)	Nếu form đăng nhập có bug, MỌI test khác cũng fail theo, dù không liên quan
<code>page.request.post("/api/login")</code>	Vài chục milli-giây	Không phụ thuộc UI form đăng nhập — chỉ phụ thuộc API <code>/api/login</code>

Với hàng chục/hàng trăm test cần "đã đăng nhập" làm điều kiện tiền đề (không phải test **về** đăng nhập), dùng `page.request` để đăng nhập nhanh giúp suite chạy nhanh hơn đáng kể, và tách rời sự phụ thuộc vào UI form — đúng tinh thần Testing Pyramid: chỉ dùng UI khi thực sự cần kiểm tra UI.

Lưu ý: kỹ thuật này khác — và đơn giản hơn — kỹ thuật storage state đầy đủ sẽ học ở Chương 22 (lưu lại trạng thái đăng nhập ra file, dùng lại giữa nhiều lần chạy test khác nhau). Ở đây, mỗi test tự gọi API đăng nhập riêng, không chia sẻ giữa các test.

Bài tập

1. Chạy `npm test`, xác nhận toàn bộ 35 test pass.
2. Viết một test API xác nhận `POST /api/test/reset` trả về đúng `{ ok: true }` (hiện chưa có test nào kiểm tra trực tiếp endpoint này, dù nó được gọi ở `beforeEach` của gần như mọi file).
3. Dùng `page.request` để viết một test: đăng nhập qua API, sau đó vào `/profile` và xác nhận tên hiển thị mặc định đúng là `demo` (giá trị khởi tạo trong `makeDemoProfile()`).
4. Giải thích bằng lời của bạn: vì sao `context.request` (gọi qua `context` thay vì `page`) cũng sẽ chia sẻ cookie với mọi `page` được tạo từ `context` đó — thử viết một test xác nhận điều này với 2 `page` khác nhau trong cùng một `context`.

Lỗi thường gặp

- Dùng `request` (độc lập) khi ý định là `page.request`: dẫn tới lỗi khó hiểu — API trả về `ok: true` nhưng `page` vẫn bị coi là chưa đăng nhập, vì hai context cookie khác nhau (xem mục 20.2).
- **Coi API test thay thế hoàn toàn cho UI test:** API test xác nhận backend đúng, nhưng không xác nhận UI hiển thị đúng, xử lý lỗi đúng phía client (như bug Chương 14) — vẫn cần cả hai.
- **Không dùng `page.request` để tối ưu tốc độ:** với suite lớn, cứ điền form UI để "chuẩn bị điều kiện" cho mọi test làm suite chạy chậm không cần thiết.

Tóm tắt & tiếp theo

- Fixture `request` viết API test thuần, nhanh hơn nhiều so với test UI cho cùng hành vi backend.
- `request` (độc lập) và `page.request` (chung context với `page`) có phạm vi cookie khác nhau — dùng đúng loại tùy mục đích.
- Dùng API để chuẩn bị trạng thái, chỉ dùng UI cho phần thực sự cần kiểm tra qua trình duyệt.

Chương 21 chuyển sang một loại kiểm thử khác hẳn: visual testing — so sánh ảnh chụp màn hình để phát hiện thay đổi UI ngoài ý muốn, thứ mà assertion văn bản (đã dùng suốt từ Chương 11) không phát hiện được.

Chương 21: Visual testing / so sánh screenshot

Mục tiêu học

- Viết được visual test bằng `toHaveScreenshot()`, hiểu cơ chế so sánh ảnh baseline.
- Hiểu và xử lý đúng vấn đề cố hữu: ảnh chụp màn hình phụ thuộc hệ điều hành.
- Biết khi nào visual testing đáng giá, khi nào assertion văn bản (Chương 11) đã đủ.

Code mẫu: `code/playwright-tests/tests/ch21-visual-testing.spec.ts` — kèm minh chứng THẬT: một lần cố tình đổi màu nút bấm, visual test bắt được ngay, phát hiện đúng "5% pixel khác biệt".

21.1. Vì sao cần visual testing, khi đã có assertion văn bản?

Toàn bộ test từ Chương 11 tới giờ kiểm tra **nội dung/hành vi** (text, URL, thuộc tính) — không kiểm tra **hình thức hiển thị**. Một CSS lỗi làm nút bấm bị đổi màu, lệch vị trí, hoặc chữ bị chồng lên nhau vẫn có thể **pass toàn bộ** test hiện có, vì nội dung text/URL không đổi. Visual testing lấp đúng lỗ hổng này: so sánh **ảnh chụp màn hình thật** với một ảnh "baseline" đã lưu trước.

21.2. `toHaveScreenshot()` — cơ bản

```
test("giao diện trang đăng nhập không đổi ngoài ý muốn", async ({ page }) => {  
  await page.goto("/login");  
  await expect(page).toHaveScreenshot("login-page.png");  
});
```

Lần chạy **đầu tiên**, Playwright chưa có ảnh để so sánh — cần tạo baseline:

```
npx playwright test ch21-visual-testing --update-snapshots
```

Ảnh được lưu vào `tests/ch21-visual-testing.spec.ts-snapshots/login-page-chromium-win32.png` — chú ý tên file có hậu tố `-chromium-win32`: Playwright tự đặt tên theo browser + hệ điều hành, vì (mục 21.4) ảnh render khác nhau giữa các hệ điều hành.

Có thể so sánh **cả trang** (`expect(page)`) hoặc **một phần tử cụ thể** (`expect(locator)`) — nên ưu tiên so sánh phần tử cụ thể khi có thể, để giảm nhiễu từ những phần không liên quan đang thay đổi (ví dụ đồng hồ, banner quảng cáo động).

21.3. Minh chứng thật: visual test bắt được thay đổi CSS

Để kiểm chứng cơ chế này thực sự hoạt động (không chỉ tin theo tài liệu), khi viết chương này, một dòng CSS được cố tình thêm vào tạm thời:

```
/* code/demo-app/public/style.css - thêm tạm để kiểm chứng, đã revert ngay sau đó */
button[type="submit"] {
  /* ... */
  background: #ff00ff;
}
```

Chạy lại test, kết quả:

```
3852 pixels (ratio 0.05 of all image pixels) are different.
```

Playwright báo **chính xác tỉ lệ pixel khác biệt** (5%) và đính kèm 2 ảnh: `*-actual.png` (ảnh thật vừa chụp) và `*-diff.png` (ảnh highlight vùng khác biệt màu đỏ) — đủ để xác định ngay đây là thay đổi cố ý hay bug thật, mà không cần đoán.

21.4. Vấn đề cố hữu: ảnh chụp màn hình phụ thuộc hệ điều hành

Đây là điều **quan trọng nhất** cần hiểu trước khi dùng visual testing trong CI thật: cùng một trang, Chrome trên Windows và Chrome trên Linux (như CI ở Chương 19, chạy `ubuntu-latest`) sẽ cho ra ảnh chụp **khác nhau ở mức pixel** — do font rendering, anti-aliasing khác nhau giữa hệ điều hành, dù giao diện "nhìn giống nhau bằng mắt người".

Baseline `login-page-chromium-win32.png` tạo trên Windows (máy viết sách này) **sẽ không khớp** nếu chạy trên `ubuntu-latest`. Đây không phải bug của Playwright hay của sách — đây là hạn chế cố hữu của kỹ thuật visual testing khi dev và CI dùng hệ điều hành khác nhau.

Giải pháp đã áp dụng trong sách (tạm thời, minh bạch):

```
test.describe("Visual testing (chỉ chạy local – xem lý do trong Chương 21)", () => {
  test.skip(
    !!process.env.CI,
    "Baseline ảnh được tạo trên Windows, không khớp ubuntu-latest trên CI (khác OS)"
  );
  // ...
});
```

Giải pháp đúng trong thực tế (không áp dụng trong sách vì cần Docker, ngoài phạm vi môi trường viết sách): tạo baseline bằng **Docker image chính thức của Playwright**

(mcr.microsoft.com/playwright) — image này dùng đúng phiên bản OS/browser mà CI Linux sẽ dùng, đảm bảo baseline luôn khớp bất kể máy dev chạy Windows/macOS/Linux gì.

Bài tập

1. Chạy `npx playwright test ch21-visual-testing --update-snapshots` để tự tạo baseline trên máy bạn, sau đó chạy lại không có cờ đó — xác nhận pass.
2. Tự tái hiện thí nghiệm ở mục 21.3: đổi một giá trị CSS trong `code/demo-app/public/style.css`, chạy lại test, xem ảnh `*-diff.png` trong `test-results/`. Nhớ revert lại sau khi xem xong.
3. Đổi `maxDiffPixelRatio: 0.1` vào tham số thứ hai của `toHaveScreenshot()` (cho phép sai khác tới 10%) — thử lại thí nghiệm ở bài 2, xem ngưỡng nào bắt được, ngưỡng nào bỏ qua thay đổi đó.
4. Đọc tài liệu Playwright về Docker image chính thức — viết ra (không cần chạy) các bước bạn sẽ làm để tạo baseline nhất quán với CI Linux nếu máy bạn có Docker.

Lỗi thường gặp

- **Tạo baseline trên máy dev (Windows/macOS), dùng thẳng cho CI Linux:** gây fail giả 100% các lần chạy CI — đây là lỗi rất phổ biến khi mới áp dụng visual testing, xem mục 21.4.
- **So sánh toàn trang cho mọi test:** dễ bị nhiễu bởi phần không liên quan đang thay đổi (ngày giờ, dữ liệu động) — ưu tiên so sánh phần tử cụ thể.
- **Không đặt ngưỡng sai khác hợp lý:** mặc định rất chặt (gần như 0% sai khác), có thể fail chỉ vì khác biệt sub-pixel không đáng kể — cân nhắc `maxDiffPixelRatio` cho các trường hợp phù hợp, nhưng không đặt quá cao khiến test mất tác dụng phát hiện lỗi thật.

Tóm tắt & tiếp theo

- `toHaveScreenshot()` phát hiện thay đổi UI mà assertion văn bản không thấy được — đã kiểm chứng thật bằng cách cố tình đổi CSS.

- Ảnh chụp màn hình phụ thuộc hệ điều hành — baseline tạo trên Windows không dùng được cho CI Linux; giải pháp thực tế là dùng Docker image chính thức của Playwright.
- SÁCH TẠM `test.skip` nhóm test này trên CI, minh bạch ghi rõ lý do — một quyết định kỹ thuật thật, không phải giấu đi hạn chế.

Chương 22 học kỹ thuật storage state — lưu trạng thái đăng nhập ra file để tái sử dụng giữa nhiều lần chạy test, đi xa hơn kỹ thuật "đăng nhập qua API" đã học ở Chương 20.

Chương 22: Authentication & storage state (test app có đăng nhập)

Mục tiêu học

- Lưu trạng thái đăng nhập ra file (`storageState.json`) bằng global setup, chạy một lần cho cả test suite.
- Dùng `test.use({ storageState: ... })` để test "đã đăng nhập sẵn" mà không cần đăng nhập lại.
- Khép lại kỹ thuật **seed dữ liệu test** đã hứa từ Chương 1: seed trực tiếp trạng thái tài khoản cho TC-07, không cần lặp UI/API nhiều lần.

Code mẫu: `code/playwright-tests/global-setup.ts` + `tests/ch22-auth-storage-state.spec.ts`.

22.1. Storage state là gì, khác `page.request` (Chương 20) thế nào?

Chương 20 dùng `page.request.post("/api/login")` để đăng nhập nhanh — nhưng vẫn phải gọi **trong từng test** cần đăng nhập. **Storage state** đi xa hơn: đăng nhập **một lần duy nhất** (thường ở bước chuẩn bị trước khi test suite chạy), lưu cookie ra file, rồi **mọi test** chỉ cần " nạp " file đó vào là đã ở trạng thái đăng nhập — không gọi API hay điền form lần nào nữa.

22.2. Global setup — chạy một lần cho cả test suite

```
// global-setup.ts
import { request as playwrightRequest } from "@playwright/test";

async function globalSetup() {
  const requestContext = await playwrightRequest.newContext({ baseURL:
"http://localhost:3000" });

  await requestContext.post("/api/test/reset");
  const res = await requestContext.post("/api/login", {
    data: { email: "demo@example.com", password: "Password123!" },
  });
  if (!(await res.json()).ok) throw new Error("Global setup: đăng nhập thất bại");

  await requestContext.storageState({ path: "storageState.json" }); // lưu cookie ra
file
  await requestContext.dispose();
}

export default globalSetup;
```

```
// playwright.config.ts
export default defineConfig({
  globalSetup: "./global-setup.ts",
  // ...
});
```

Khác `beforeAll` (Chương 12, chạy một lần **cho mỗi** `describe`), `globalSetup` chạy **đúng một lần cho toàn bộ lần chạy** `npm test`, trước cả file test đầu tiên. Vì `webServer` (Chương 9) khởi động trước khi test run bắt đầu, tới lúc `globalSetup` chạy, demo app đã sẵn sàng nhận request.

22.3. Dùng storage state trong test

```
test.use({ storageState: "storageState.json" });

test("đã đăng nhập sẵn – mở thẳng /dashboard không cần qua /login", async ({ page }) => {
  await page.goto("/dashboard"); // KHÔNG có bước điền form đăng nhập nào ở đây
  await expect(page.locator("#username")).toHaveText("demo@example.com");
});
```

`test.use({ storageState: "..." })` đặt ở đầu file (hoặc trong `describe`) áp dụng cho **mọi test** trong phạm vi đó — Playwright tự nạp cookie từ file vào browser context trước khi mỗi test bắt đầu, không cần gọi gì thêm.

Khi nào KHÔNG nên dùng storage state?

Test **về chính hành vi đăng nhập** (như TC-01 → TC-09, Chương 11) phải **không** dùng storage state — vì mục đích của chúng chính là kiểm tra luồng đăng nhập, dùng session có sẵn sẽ bỏ qua đúng phần cần test. Trong `ch22-auth-storage-state.spec.ts`, test cho TC-07 ghi đè lại bằng `test.use({ storageState: { cookies: [], origins: [] } })` — nạp một storage state **rỗng**, đảm bảo test đó bắt đầu từ trạng thái chưa đăng nhập, độc lập với `test.use` ở đầu file.

22.4. Khép lại lời hứa từ Chương 1: seed dữ liệu cho TC-07

Nhắc lại ghi chú ở `code/ch01-sdlc/test-case-list-login.md`: "TC-07 (khoá tài khoản) là ví dụ về case cần dữ liệu/trạng thái đặc biệt — sẽ được dùng làm ví dụ cho kỹ thuật seed dữ liệu test ở Phần V." Chương 11 tự động hoá TC-07 bằng cách **lặp 5 lần** đăng nhập sai qua UI — đúng nhưng chậm, và không rõ ràng "5 lần sai" hay "kiểm tra hành vi ở lần thứ 5" mới là điều test thực sự muốn xác minh.

Chương này thêm endpoint test-only để **seed trực tiếp** trạng thái "đã sai 4 lần":

```
// server.js
app.post("/api/test/seed-failed-attempts", (req, res) => {
  const { email, count } = req.body || {};
  const user = setFailedAttempts(email || "demo@example.com", count ?? 0);
  // ...
  res.json({ ok: true });
});
```

```
// test chỉ cần đăng nhập sai ĐÚNG 1 LẦN để kiểm tra hành vi khoá – nhanh và rõ ý hơn
nhiều
await request.post("/api/test/seed-failed-attempts", {
  data: { email: "demo@example.com", count: 4 },
});
// ... đăng nhập sai 1 lần, xác nhận bị khoá
```

Đây chính xác là kỹ thuật **seed dữ liệu test**: thay vì tái tạo toàn bộ chuỗi hành động dẫn tới một trạng thái, tạo trực tiếp trạng thái đó — nhanh hơn, và test đọc rõ ý định hơn ("test hành vi ở lần sai thứ 5", không phải "test cả 5 lần sai").

Bài tập

1. Chạy `npm test`, xác nhận toàn bộ 40 test pass, và `storageState.json` được tạo ra ở thư mục `code/playwright-tests/` sau khi chạy (đã có trong `.gitignore`, không commit).
 2. Xoá `storageState.json`, chạy `npx playwright test ch22-auth-storage-state` — quan sát `globalSetup` tự tạo lại file đó trước khi test chạy.
 3. So sánh 2 cách tự động hoá TC-07: vòng lặp 5 lần (Chương 11, `ch11-login-test-cases.spec.ts`) và seed trực tiếp (chương này) — viết ra 1 tình huống mà cách lặp vẫn cần thiết (không thể seed thay được).
 4. Thêm một endpoint test-only mới `/api/test/seed-profile` để seed sẵn dữ liệu hồ sơ (Chương 13), dùng nó viết một test xác nhận trang `/profile` hiển thị đúng dữ liệu đã seed mà không cần điền form trước.
-

Lỗi thường gặp

- **Dùng storage state cho test đang kiểm tra chính luồng đăng nhập:** bỏ qua đúng phần cần test — xem cảnh báo ở mục 22.3.
 - **Commit `storageState.json` vào git:** đây là dữ liệu sinh ra (chứa cookie session), không nên commit — đã thêm vào `.gitignore`.
 - **Seed dữ liệu qua endpoint test-only nhưng để lộ ra production:** các route như `/api/test/reset`, `/api/test/seed-failed-attempts` chỉ nên tồn tại trong môi trường test/dev — một dự án thật cần đảm bảo các route này **không** được build/deploy lên production.
-

Tóm tắt & tiếp theo

- `globalSetup` chạy một lần cho cả test suite — phù hợp để tạo storage state dùng chung.
- `test.use({ storageState: ... })` cho test "đã đăng nhập sẵn"; ghi đè bằng storage state rỗng cho test cần bắt đầu từ trạng thái chưa đăng nhập.
- Seed dữ liệu trực tiếp qua endpoint test-only nhanh và rõ ý hơn nhiều so với tái tạo toàn bộ chuỗi hành động dẫn tới trạng thái đó.

Chương 23 quay lại giải quyết đúng cách vấn đề đã tạm né từ Chương 11 (`workers: 1`) — chạy test song song an toàn, dùng chính kỹ thuật seed dữ liệu vừa học ở chương này.

Chương 23: Chạy song song, sharding, tối ưu tốc độ test suite

Mục tiêu học

- Phân biệt **workers** (chạy song song trong một máy) và **sharding** (chia test suite ra nhiều máy/CI job khác nhau).
- Áp dụng kỹ thuật đúng để chạy song song an toàn: mỗi worker seed dữ liệu riêng.
- Thực sự bật lại chạy song song (đã tạm né từ Chương 11) — kiểm chứng bằng `--workers=4` chạy ổn định nhiều lần liên tiếp.

Code mẫu: `code/playwright-tests/tests/ch23-song-song-sharding.spec.ts` — đã kiểm chứng thật: chạy `--workers=4` liên tiếp 4 lần, luôn "4 passed", không một lần flaky.

23.1. Workers vs Sharding — hai khái niệm khác nhau

Nhiều máy — Sharding

Máy 1: shard 1/3

Máy 2: shard 2/3

Máy 3: shard 3/3

Một máy (ví dụ máy CI, hoặc máy dev)

Worker 1

Worker 2

Worker 3

	Workers	Sharding
Chạy song song ở đâu	Nhiều tiến trình, CÙNG một máy	Chia test ra NHIỀU máy/job CI khác nhau
Giới hạn bởi	Số CPU core của máy đó	Số máy/job có thể chạy đồng thời (thường không giới hạn nhiều như CPU)
Cấu hình	<code>workers</code> trong config, hoặc <code>--workers=N</code>	<code>--shard=1/3</code> , <code>--shard=2/3</code> , <code>--shard=3/3</code> (chạy trên 3 job riêng)

Hai kỹ thuật này **kết hợp được với nhau**: mỗi shard (mỗi máy CI) lại tự chạy nhiều worker song song bên trong nó — tổng số test chạy đồng thời = số shard × số worker mỗi shard.

23.2. Sharding trong GitHub Actions

```
# .github/workflows/playwright.yml - mở rộng để chạy 3 shard song song
strategy:
  matrix:
    shard: [1, 2, 3]
steps:
  # ... các bước cài đặt như Chương 19 ...
  - name: Run Playwright tests (shard ${matrix.shard}/3)
    working-directory: code/playwright-tests
    run: npx playwright test --shard=${matrix.shard}/3
```

`strategy.matrix` của GitHub Actions tạo ra **3 job chạy song song**, mỗi job nhận một phần `--shard`. Với suite hiện tại (44 test, chạy vài chục giây), sharding chưa cần thiết — nhưng với suite hàng nghìn test (nhiều giờ nếu chạy tuần tự), sharding là kỹ thuật bắt buộc để giữ thời gian CI hợp lý.

23.3. Vấn đề đã né từ Chương 11: chạy song song trên dữ liệu dùng chung

Nhắc lại: từ Chương 11, `playwright.config.ts` đặt `workers: 1` vì mọi test dùng chung tài khoản `demo@example.com` — chạy song song gây đung độ trạng thái (TC-07 fail ngẫu nhiên). Đây **không** phải hạn chế của Playwright — đây là hạn chế của **cách thiết kế dữ liệu test** dùng chung một tài khoản cho mọi test.

23.4. Giải pháp đúng: mỗi worker seed tài khoản riêng

```
// server.js – endpoint mới (Chương 23)
app.post("/api/test/create-user", (req, res) => {
  const { email, password } = req.body || {};
  createUser(email, password); // tạo tài khoản MỚI, độc lập với demo@example.com
  res.json({ ok: true });
});
```

```
// ch23-song-song-sharding.spec.ts
test.describe("Chạy song song an toàn – mỗi worker một tài khoản riêng", () => {
  test.describe.configure({ mode: "parallel" }); // ghi đè fullyParallel:false của
  config gốc

  test.beforeEach(async ({ request }, testInfo) => {
    const email = `worker${testInfo.parallelIndex}@example.com`; // KHÁC nhau giữa các
    worker
    await request.post("/api/test/create-user", { data: { email, password:
    "Password123!" } });
  });

  test("...", async ({ page }, testInfo) => {
    const email = `worker${testInfo.parallelIndex}@example.com`;
    // đăng nhập, thao tác... – KHÔNG BAO GIỜ đụng tài khoản của worker khác
  });
});
```

`testInfo.parallelIndex` là chỉ số (0, 1, 2...) **đảm bảo khác nhau** giữa các worker đang chạy đồng thời — dùng nó để tạo email riêng cho từng worker, mỗi worker có "vùng dữ liệu" hoàn toàn tách biệt. Kiểm chứng thật:

```
npx playwright test ch23-song-song-sharding --workers=4
# Running 4 tests using 4 workers
# 4 passed (3.4s)
```

Chạy lại lệnh này **4 lần liên tiếp** khi viết chương này — luôn `4 passed`, không một lần flaky (khác hẳn TC-07 ở Chương 11 khi chạy song song trên tài khoản chung).

23.5. Vì sao chỉ áp dụng cho chương này, chưa áp dụng lại cho TOÀN BỘ suite?

Đây là điểm cần trung thực: 22 chương trước (ch11 đến ch22) đều hard-code `demo@example.com` — để bật song song an toàn cho **toàn bộ** suite, mỗi file đó cần refactor để dùng tài khoản riêng theo worker, giống kỹ thuật ở mục 23.4. Đây là công việc có thật, nhưng **nằm ngoài phạm vi một chương** — sách chọn cách trung thực hơn: giữ `workers: 1` là mặc định (an toàn cho toàn bộ suite hiện tại), và dùng `test.describe.configure({ mode: "parallel" })` như một ngoại lệ có chủ đích cho nhóm test đã được thiết kế đúng để chạy song song.

Bài học quan trọng hơn kỹ thuật cụ thể: chạy song song an toàn là vấn đề **thiết kế dữ liệu test** trước khi là vấn đề cấu hình Playwright — nếu thiết kế mọi test độc lập dữ liệu từ đầu (giống Chương 23), không cần "sửa lại" gì cả.

Bài tập

1. Chạy `npx playwright test ch23-song-song-sharding --workers=4` vài lần liên tiếp, xác nhận luôn `4 passed`.
2. Thử áp dụng kỹ thuật ở mục 23.4 để refactor `ch16-fixtures-co-ban.spec.ts` (dùng fixture `LoggedInPage`) sang dùng tài khoản riêng theo worker — đây chính là bước đầu của việc "mở rộng ra toàn bộ suite" nói ở mục 23.5.
3. Đọc kỹ cấu hình `strategy.matrix` ở mục 23.2 — nếu suite có 300 test, chia làm 3 shard, mỗi shard chạy trên máy 4 core (`workers=4`), tối đa bao nhiêu test chạy đồng thời tại một thời điểm?
4. Giải thích: vì sao sharding không giải quyết được vấn đề "đụng độ dữ liệu chung" như ở Chương 11 — dù chia ra nhiều máy, các test trong CÙNG một shard vẫn có thể đụng độ nếu vẫn dùng chung một tài khoản.

Lỗi thường gặp

- **Tương tăng `workers` là luôn tăng tốc, không rủi ro gì:** nếu dữ liệu test không được thiết kế độc lập (như phần lớn suite hiện tại), tăng `workers` chỉ tái tạo lại chính xác vấn đề đã gặp ở Chương 11.
- **Nhầm sharding với workers:** sharding chia theo MÁY, workers chia theo TIẾN TRÌNH trong một máy — cấu hình sai chỗ (ví dụ tưởng `--shard` thay thế được `workers`) dẫn đến hiểu sai về mức độ song song thực tế.
- **Dùng `testInfo.workerIndex` thay vì `testInfo.parallelIndex`:** `workerIndex` có thể đổi giá trị khi Playwright khởi động lại worker (ví dụ sau khi retry), không đảm bảo tính duy nhất giữa các worker đang chạy đồng thời như `parallelIndex`.

Tóm tắt & tiếp theo

- Workers chạy song song trong một máy; sharding chia suite ra nhiều máy — hai kỹ thuật độc lập, kết hợp được với nhau.
- Chạy song song an toàn đòi hỏi mỗi worker có dữ liệu test độc lập — `testInfo.parallelIndex` là cách chuẩn để tạo định danh duy nhất theo worker.
- Kiểm chứng thật bằng `--workers=4` chạy lặp lại nhiều lần là cách xác nhận đáng tin cậy hơn đọc lý thuyết suông.

Chương 24 chuyển sang một trục mở rộng khác: chạy CÙNG một bộ test trên nhiều trình duyệt (Chromium, Firefox, WebKit) và giả lập thiết bị di động.

Chương 24: Cross-browser testing & mobile emulation

Mục tiêu học

- Cấu hình nhiều `projects` để chạy cùng bộ test trên Chromium, Firefox, WebKit.
- Giả lập thiết bị di động (viewport, `isMobile`) mà không cần thiết bị thật.
- Quyết định hợp lý: chạy toàn bộ suite hay chỉ một phần trên nhiều browser.

Code mẫu: `code/playwright-tests/tests/ch24-cross-browser-mobile.spec.ts` — 2 test, chạy thật trên cả 4 project (chromium, firefox, webkit, mobile-chrome) = 8 lần chạy, đã xác nhận pass.

24.1. Cài thêm Firefox và WebKit

Từ Chương 8, project chỉ cài Chromium. Để test cross-browser thật (không phải nói suông), cần cài thêm 2 engine còn lại:

```
npx playwright install firefox webkit
```

Lưu ý quan trọng: "WebKit" ở đây là engine dùng trong Safari — Playwright cho phép test hành vi giống Safari **trên Windows/Linux**, không cần máy Mac thật. Đây rất hữu ích để phát hiện lỗi đặc thù Safari sớm, dù đội ngũ không có sẵn máy Mac để test tay.

24.2. Khai báo nhiều `projects`

```
// playwright.config.ts
projects: [
  { name: "chromium", use: { ...devices["Desktop Chrome"] } },
  {
    name: "firefox",
    use: { ...devices["Desktop Firefox"] },
    testMatch: /ch24-cross-browser-mobile\.spec\.ts/,
  },
  {
    name: "webkit",
    use: { ...devices["Desktop Safari"] },
    testMatch: /ch24-cross-browser-mobile\.spec\.ts/,
  },
  {
    name: "mobile-chrome",
    use: { ...devices["Pixel 7"] }, // giả lập thiết bị di động thật (viewport, user-agent ...)
    testMatch: /ch24-cross-browser-mobile\.spec\.ts/,
  },
],
```

`devices["Pixel 7"]` (và hàng trăm thiết bị khác Playwright hỗ trợ sẵn) cung cấp đúng viewport, user-agent, tỉ lệ pixel của thiết bị thật — **giả lập**, không phải chạy trên thiết bị/emulator Android thật, nhưng đủ chính xác cho phần lớn mục đích kiểm thử responsive.

24.3. `testMatch` — quyết định quan trọng: KHÔNG chạy mọi test trên mọi browser

Chú ý `testMatch` ở 3 project firefox/webkit/mobile-chrome: chỉ file `ch24-cross-browser-mobile.spec.ts` chạy trên các project này — **43 test khác** (Chương 11-23) chỉ chạy trên `chromium`. Đây là quyết định có chủ đích, không phải thiếu sót:

- Chạy **toàn bộ** suite trên cả 4 project sẽ tốn gần **4 lần thời gian CI**.
- Phần lớn logic (validation, API, storage state...) không có lý do đặc biệt để khác nhau giữa các engine trình duyệt — nếu Chromium đúng, khả năng cao Firefox/WebKit cũng đúng.
- Thực tế phổ biến: chỉ chạy **một bộ smoke test đại diện** (các luồng quan trọng nhất) trên đa browser, còn lại chạy trên một browser chính (thường Chromium, engine phổ biến nhất).

Chạy `npm test` sau khi thêm cấu hình này: 52 test tổng (44 test cũ + 2 test × 4 project mới) — xác nhận đúng số lượng, không có project nào chạy nhầm file khác.

24.4. Kiểm tra riêng cho mobile: fixture `isMobile`

```
test("form đăng nhập hiển thị đầy đủ, không bị cắt trên viewport di động", async ({
  page,
  isMobile,
}) => {
  await page.goto("/login");
  const viewport = page.viewportSize();
  const formBox = await page.locator("#login-form").boundingBox();

  expect(formBox!.width).toBeLessThanOrEqual(viewport!.width); // không tràn ngang

  if (isMobile) {
    expect(viewport!.width).toBeLessThan(500); // nhánh riêng, chỉ chạy trên mobile-
chrome
  }
});
```

`isMobile` là fixture có sẵn của Playwright, tự động `true / false` tùy project đang chạy có khai báo `isMobile: true` trong device descriptor hay không (`devices["Pixel 7"]` có, `devices["Desktop Chrome"]` không) — cho phép viết logic khác nhau cho mobile ngay trong cùng một test, không cần file riêng.

Bài tập

1. Chạy `npm test`, xác nhận 52 test pass — trong đó đúng 8 test (2 × 4 project) thuộc `ch24-cross-browser-mobile.spec.ts`.
2. Chạy riêng `npx playwright test ch24-cross-browser-mobile --project=webkit` — chỉ chạy trên WebKit, thời gian nhanh hơn chạy cả 4 project.
3. Thử đổi `devices["Pixel 7"]` thành `devices["iPhone 15"]` — chạy lại, xác nhận vẫn pass (viewport khác nhưng cùng logic `isMobile`).
4. Thêm một project mới `tablet` dùng `devices["iPad Pro 11"]`, viết test riêng kiểm tra viewport tablet không kích hoạt nhánh `isMobile` (vì iPad thường không đặt `isMobile: true`).

Lỗi thường gặp

- **Chạy toàn bộ suite trên mọi browser mà không cân nhắc chi phí thời gian:** với suite lớn, đây là nguyên nhân phổ biến khiến CI chạy quá lâu — luôn cân nhắc `testMatch` để giới hạn phạm vi.
- **Quên cài Firefox/WebKit trên CI** (chỉ cài Chromium như Chương 19): job CI sẽ fail ngay khi chạy tới project firefox/webkit vì thiếu executable — đã cập nhật `.github/workflows/playwright.yml` để cài đủ cả 3.

- **Nhằm "giả lập thiết bị di động" với "chạy trên thiết bị thật":** `devices["Pixel 7"]` giả lập viewport/user-agent trên engine Chromium desktop — không phát hiện được lỗi đặc thù của trình duyệt di động thật (Chrome for Android) hay hệ điều hành di động thật.
-

Tóm tắt & tiếp theo

- `projects` trong `playwright.config.ts` cho phép chạy cùng bộ test trên nhiều browser/thiết bị.
- `testMatch` giới hạn project nào chạy file nào — tránh chạy toàn bộ suite trên mọi browser một cách không cần thiết.
- `isMobile` cho phép viết logic riêng cho mobile trong cùng một test, dùng chung cho mọi thiết bị.

Chương 25 học component testing — kiểm thử một component UI độc lập, không cần chạy cả trang, phù hợp với các dự án dùng framework component (React/Vue/Svelte).

Chương 25: Component testing với Playwright

Mục tiêu học

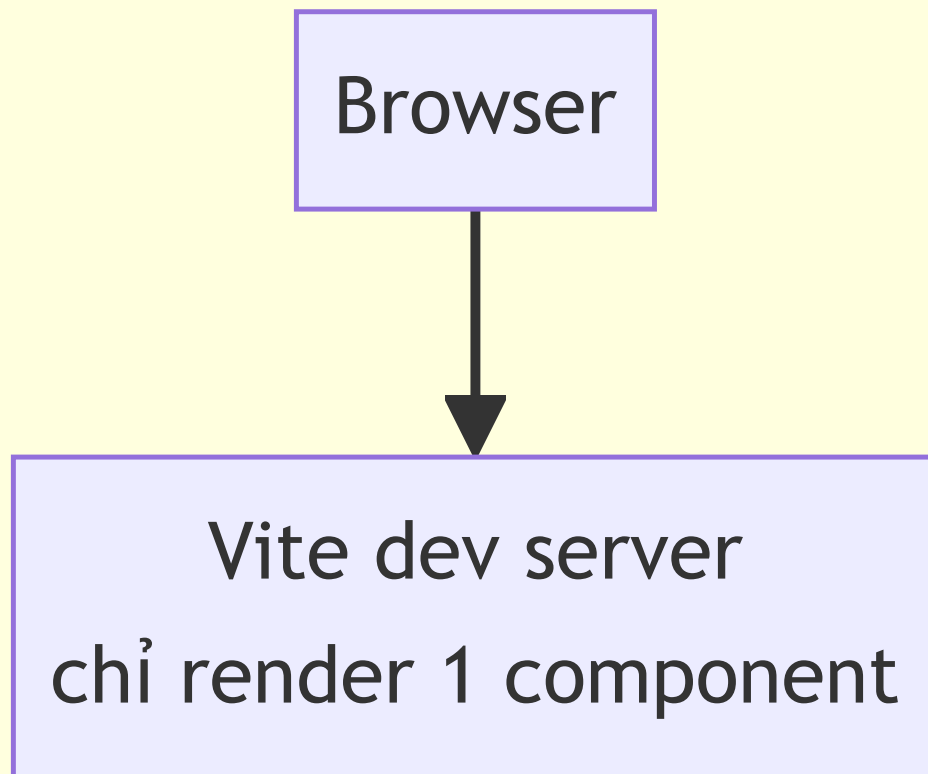
- Hiểu component testing khác E2E testing (Chương 11-24) ở điểm cốt lõi nào.
- Cài đặt và chạy được `@playwright/experimental-ct-react` từ đầu.
- Viết test render một component React độc lập, không cần server hay trang nào.

Code mẫu: `code/component-testing/` — project **riêng biệt**, độc lập với `demo-app` và `playwright-tests`. Đã kiểm chứng thật: `npm test` build bằng Vite, mount component trong Chromium thật, 2/2 test pass.

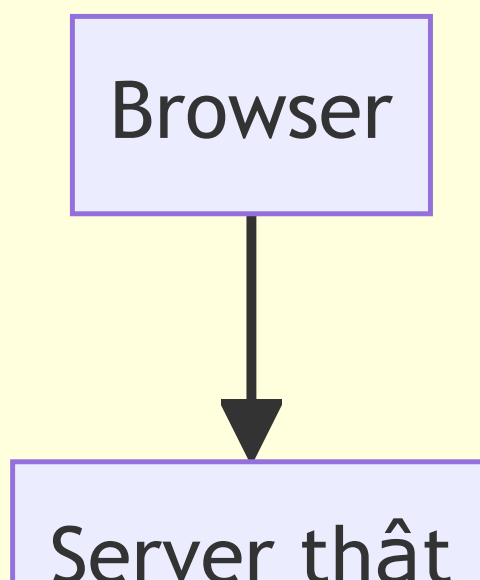
25.1. Component testing khác E2E testing ở đâu?

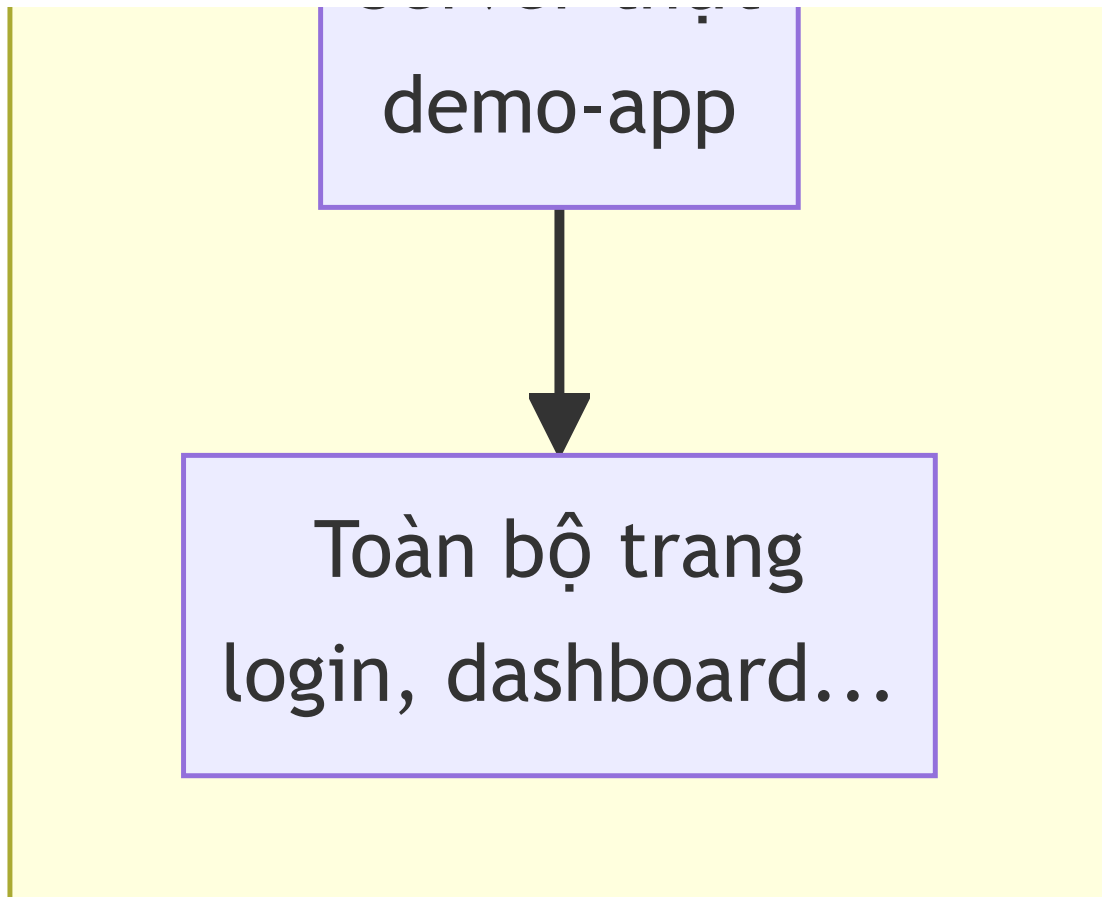
Toàn bộ Phần IV-V tới giờ là **E2E testing**: mở trình duyệt, điều hướng tới một **URL thật** của một **ứng dụng đang chạy** (`demo-app`), thao tác như người dùng thật. **Component testing** khác hẳn: render **một component UI đơn lẻ** (không phải cả trang, không cần server), kiểm tra hành vi của riêng nó.

Component Testing (Chương 25)



E2E Testing (Chương 11-24)





Component testing chỉ áp dụng được cho dự án dùng **framework component** (React, Vue, Svelte) — `demo-app` của sách (HTML thuần + vanilla JS) **không** có khái niệm "component" theo nghĩa này, nên chương này dùng một project React nhỏ, riêng biệt, để minh họa đúng kỹ thuật.

25.2. Cài đặt từ đầu

```
mkdir component-testing && cd component-testing
npm init -y
npm install -D @playwright/experimental-ct-react react react-dom @types/react
@types/react-dom
npx playwright install chromium
```

Cấu hình riêng biệt với `playwright.config.ts` thông thường — dùng file `playwright-ct.config.ts`:

```
// playwright-ct.config.ts
import { defineConfig, devices } from "@playwright/experimental-ct-react";

export default defineConfig({
  testDir: "./tests",
  use: { ctPort: 3100 },
  projects: [{ name: "chromium", use: { ...devices["Desktop Chrome"] } }],
});
```

Và 2 file "bootstrap" bắt buộc (Playwright dùng để dựng trang HTML tạm render component):

```
playwright/index.html - HTML rỗng, chỉ có <div id="root">
playwright/index.tsx - nơi import CSS toàn cục nếu cần (để trống nếu không cần)
```

25.3. Viết component và test

```
// src/PasswordField.tsx - tái hiện hành vi hiện/ẩn mật khẩu của demo-app (Chương 9)
import { useState } from "react";

type PasswordFieldProps = {
  label: string;
  value: string;
  onChange: (value: string) => void;
};

export function PasswordField({ label, value, onChange }: PasswordFieldProps) {
  const [visible, setVisible] = useState(false);
  return (
    <div>
      <label htmlFor="password-field">{label}</label>
      <input
        id="password-field"
        type={visible ? "text" : "password"}
        value={value}
        onChange={e => onChange(e.target.value)}
      />
      <button aria-label={visible ? "Ẩn ký tự" : "Hiện ký tự"} onClick={() =>
setVisible((v) => !v)}>
        ⦿
      </button>
    </div>
  );
}
```

```
// tests/PasswordField.spec.tsx
import { test, expect } from "@playwright/experimental-ct-react";
import { PasswordField } from "../src/PasswordField";

test("mặc định ẩn mật khẩu, bấm nút để hiện ký tự", async ({ mount }) => {
  const component = await mount(<PasswordField label="Mật khẩu" value="" onChange={() => {} } />);

  const input = component.getByLabel("Mật khẩu");
  await expect(input).toHaveAttribute("type", "password");

  await component.getByRole("button", { name: "Hiện ký tự" }).click();
  await expect(input).toHaveAttribute("type", "text");
});
```

`mount()` — điểm khác biệt cốt lõi so với `page.goto()` — **render trực tiếp** component React này vào một trang tạm (thông qua Vite), trả về một handle giống `Locator`, dùng được `getByLabel` / `getByRole` như bình thường.

Chạy:

```
npm test
```

Output thật khi chạy chương này: Vite build component (✓ built in 633ms), rồi Playwright mount và chạy test trong Chromium — 2 passed.

25.4. Khi nào dùng component testing, khi nào dùng E2E?

	Component Testing	E2E Testing
Tốc độ	Rất nhanh (không cần server, không điều hướng)	Chậm hơn (cần server, trang đầy đủ)
Phạm vi	Một component, cô lập	Toàn bộ luồng người dùng thật
Phát hiện lỗi tích hợp (routing, API thật, session...)	Không	Có
Phù hợp với	Component tái sử dụng nhiều nơi (button, form field, modal...)	Luồng nghiệp vụ hoàn chỉnh (đăng nhập, thanh toán...)

Nhắc lại Testing Pyramid (Chương 2): component testing nằm **giữa** unit test và E2E test về mặt tốc độ/phạm vi — hợp lý để viết nhiều component test cho các component dùng lại nhiều nơi, thay vì chỉ dựa vào E2E test để gián tiếp kiểm tra chúng.

Bài tập

1. Chạy `npm install && npx playwright install chromium && npm test` trong `code/component-testing/`, xác nhận 2 test pass.
2. Thêm một prop `disabled?: boolean` vào `PasswordField`, khi `true` thì input và nút đều bị vô hiệu hoá — viết test xác nhận bằng `toBeDisabled()`.
3. So sánh thời gian chạy `npm test` ở đây với thời gian chạy toàn bộ `code/playwright-tests/` — giải thích vì sao component testing nhanh hơn đáng kể dù đều "mở trình duyệt thật".
4. Nếu dự án của bạn dùng Vue hoặc Svelte thay vì React, tìm package tương ứng (`@playwright/experimental-ct-vue`, `@playwright/experimental-ct-svelte`) — cấu hình có gì khác so với bản React ở chương này?

Lỗi thường gặp

- **Tương component testing thay thế được E2E testing:** hai kỹ thuật bổ trợ nhau — component testing không phát hiện được lỗi tích hợp giữa các phần (routing, API thật, session...), những thứ chỉ E2E test (Chương 11-24) mới kiểm tra được.
- **Thiếu file `playwright/index.html / index.tsx`:** đây là 2 file bootstrap bắt buộc, quên tạo sẽ khiến Playwright báo lỗi không tìm được entry point khi build bằng Vite.
- **Áp dụng component testing cho dự án không dùng framework component:** như `demo-app` của sách (HTML thuần) — kỹ thuật này chỉ áp dụng được khi UI được viết bằng React/Vue/Svelte thật.

Tóm tắt & tiếp theo

- Component testing render một component UI độc lập bằng Vite, không cần server backend.
- `mount()` thay cho `page.goto()`; locator (`getByLabel`, `getByRole`) dùng giống hệt E2E testing.
- Chỉ áp dụng được cho dự án dùng framework component; bổ trợ (không thay thế) E2E testing.

Chương 26 — chương cuối Phần V — học cách viết custom fixture nâng cao và reporter riêng, hoàn thiện các kỹ thuật tổ chức test đã học từ Chương 15-16.

Chương 26: Custom fixtures nâng cao, viết plugin/reporter riêng

Mục tiêu học

- Viết fixture **worker-scoped** — chạy một lần cho cả worker, khác fixture test-scoped (Chương 16) chạy lại cho mỗi test.
- Kết hợp fixture worker-scoped với kỹ thuật "mỗi worker một tài khoản riêng" (Chương 23).
- Viết một custom reporter đơn giản, không cần cài package ngoài.

Code mẫu: `fixtures.ts` (fixture `workerAccount`, `isolatedLoggedInPage`) + `tests/ch26-custom-fixtures-nang-cao.spec.ts` + `tools/simple-reporter.js`.

26.1. Fixture worker-scoped — chạy một lần cho cả worker

Mọi fixture ở Chương 16 (`LoggedInPage`) là **test-scoped**: chạy lại từ đầu cho **mỗi** test. Với việc tạo tài khoản (Chương 23) — vốn không cần tạo lại nếu đã có — chạy lại mỗi test là lãng phí. **Fixture worker-scoped** giải quyết đúng việc này: chạy **một lần cho cả worker**, mọi test trong worker đó dùng lại cùng kết quả.

```
// fixtures.ts
type WorkerFixtures = {
  workerAccount: { email: string; password: string };
};

export const test = base.extend<TestFixtures, WorkerFixtures>({
  // ...
  workerAccount: [
    async ({}, use, workerInfo) => {
      const email = `worker-account-${workerInfo.workerIndex}@example.com`;
      const password = "Password123!";

      // Fixture worker-scoped KHÔNG dùng được fixture `request` (test-scoped) – phải tự
      // tạo APIRequestContext riêng, giống global-setup.ts (Chương 22).
      const requestContext = await playwrightRequest.newContext({ baseURL:
"http://localhost:3000" });
      await requestContext.post("/api/test/create-user", { data: { email, password } });
      await requestContext.dispose();

      await use({ email, password }); // chạy 1 lần, dùng lại cho MỌI test trong worker
    },
    { scope: "worker" }, // ← khai báo scope, khác mặc định (test-scoped)
  ],
});
```

Ba điểm khác biệt so với fixture test-scoped (Chương 16):

1. **Khai báo dạng tuple** [hàm, { scope: "worker" }], không phải hàm trực tiếp.
2. **Không truy cập được fixture test-scoped** (page, request) — chỉ dùng fixture worker-scoped khác, hoặc tạo tài nguyên độc lập (như playwrightRequest.newContext() ở trên).
3. Tham số thứ ba của hàm fixture là **workerInfo** (khác **testInfo** của fixture test-scoped) — chứa **workerIndex**, ổn định trong suốt vòng đời worker đó.

26.2. Kết hợp fixture worker-scoped với fixture test-scoped

```
isolatedLoggedInPage: async ({ page, workerAccount }, use) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login(workerAccount.email, workerAccount.password);
  await use(page);
},
```

`isolatedLoggedInPage` (test-scoped) **phụ thuộc vào** `workerAccount` (worker-scoped) — Playwright tự hiểu và chỉ tạo `workerAccount` một lần, tái sử dụng cho mọi test gọi `isolatedLoggedInPage` trong cùng worker. Đây chính là cách kết hợp: **setup đặt đồ, không đổi giữa các test** → worker scope; **setup rẻ, cần làm lại mỗi test** (đăng nhập lại, vì mỗi test có `page` /browser context riêng — nhắc lại Chương 12) → test scope.

Kiểm chứng thật:

```
npx playwright test ch26-custom-fixtures-nang-cao --workers=3
```

```
[SimpleReporter] Bắt đầu chạy 3 test ...
[SimpleReporter] ✓ test #0: ... (231ms)
[SimpleReporter] ✓ test #1: ... (235ms)
[SimpleReporter] ✓ test #2: ... (226ms)
[SimpleReporter] Xong: passed - chạy trong 2.1s
```

26.3. Viết custom reporter

Một custom reporter chỉ cần là **một class** với (một phần) các phương thức Playwright gọi tới ở từng giai đoạn — không cần cài package, không cần `extends` class nào:

```
// tools/simple-reporter.js
class SimpleReporter {
  onBegin(config, suite) {
    this.startTime = Date.now();
    this.total = suite.allTests().length;
    console.log(`\n[SimpleReporter] Bắt đầu chạy ${this.total} test ...`);
  }

  onTestEnd(test, result) {
    const icon = result.status === "passed" ? "✓" : result.status === "skipped" ? "o" : "X";
    console.log(`[SimpleReporter] ${icon} ${test.title} (${result.duration}ms)`);
  }

  onEnd(result) {
    const seconds = ((Date.now() - this.startTime) / 1000).toFixed(1);
    console.log(`[SimpleReporter] Xong: ${result.status} - chạy trong ${seconds}s\n`);
  }
}

module.exports = SimpleReporter;
```

Đăng ký trong `playwright.config.ts`, cùng với `html` và `allure-playwright` (Chương 18):

```
reporter: [["html"], ["allure-playwright"], [".tools/simple-reporter.js"]],
```

Ba phương thức phổ biến nhất (còn nhiều phương thức khác như `onStdOut`, `onError`):

Phương thức	Gọi khi nào
<code>onBegin(config, suite)</code>	Một lần, trước khi test đầu tiên chạy — biết trước tổng số test
<code>onTestEnd(test, result)</code>	Sau mỗi test — biết pass/fail/skip và thời gian chạy
<code>onEnd(result)</code>	Một lần, sau khi toàn bộ test run xong

26.4. Khi nào viết custom reporter, khi nào dùng có sẵn?

Với nhu cầu thông thường, `html` (Chương 9) và `allure-playwright` (Chương 18) đã đủ. Viết reporter riêng đáng làm khi cần: gửi kết quả tới hệ thống nội bộ (Slack, dashboard riêng của công ty), định dạng output đặc thù cho công cụ khác đọc, hoặc log gọn hơn cho terminal khi chạy dev hàng ngày (như `SimpleReporter` ở đây).

Bài tập

1. Chạy `npx playwright test ch26-custom-fixtures-nang-cao --workers=3`, xác nhận log của `SimpleReporter` hiện đúng 3 dòng ✓.
2. Thêm phương thức `onStdOut` vào `SimpleReporter`, in ra mọi `console.log` từ code test/app — thử thêm một `console.log` tạm trong `demo-app/server.js` để quan sát.
3. Đổi `{ scope: "worker" }` thành bỏ hẵn (về test-scoped mặc định) trong `workerAccount`, chạy lại `npx playwright test ch26-custom-fixtures-nang-cao --workers=3` — dùng `console.log` trong fixture để đếm số lần `POST /api/test/create-user` được gọi, so sánh trước/sau.
4. Viết thêm phương thức `onError` vào `SimpleReporter`, in ra một dòng riêng khi có lỗi ở cấp độ framework (không phải lỗi test thường) xảy ra.

Lỗi thường gặp

- **Cố truy cập fixture test-scoped (`page`, `request`) từ fixture worker-scoped:** Playwright báo lỗi ngay khi khai báo — fixture worker-scoped chỉ phụ thuộc được fixture worker-scoped khác.
- **Dùng worker-scoped cho dữ liệu CẦN sạch ở mỗi test:** nếu test làm thay đổi trạng thái (như TC-07 khoá tài khoản), dùng chung 1 tài khoản worker-scoped cho mọi test sẽ gây đúng vấn đề đã gặp ở Chương 11 — chỉ dùng worker scope cho setup **không đổi** giữa các test.

- **Quên** `module.exports` trong file reporter tùy biến: Playwright không nhận diện được class, báo lỗi khi load config.
-

Tóm tắt & tiếp theo

- Fixture worker-scoped chạy một lần cho cả worker — phù hợp cho setup đắt đỏ, không đổi giữa các test (như tạo tài khoản).
- Custom reporter chỉ cần một class với các phương thức `onBegin` / `onTestEnd` / `onEnd` — không cần cài gì thêm.
- Kết hợp fixture worker-scoped + test-scoped cho phép tối ưu tốc độ mà vẫn giữ mỗi test độc lập.

Phần V (Playwright nâng cao) đến đây đã hoàn thành — 26 chương, 55 test, một demo app đầy đủ, một project component testing riêng, và một pipeline CI. Phần VI chuyển hướng hoàn toàn: AI + MCP — dùng AI để điều khiển trình duyệt và hỗ trợ viết test.

Phần VI — Nâng cao: AI + MCP / Browser Automation

Chương 27: Model Context Protocol (MCP) là gì, vì sao liên quan đến automation test

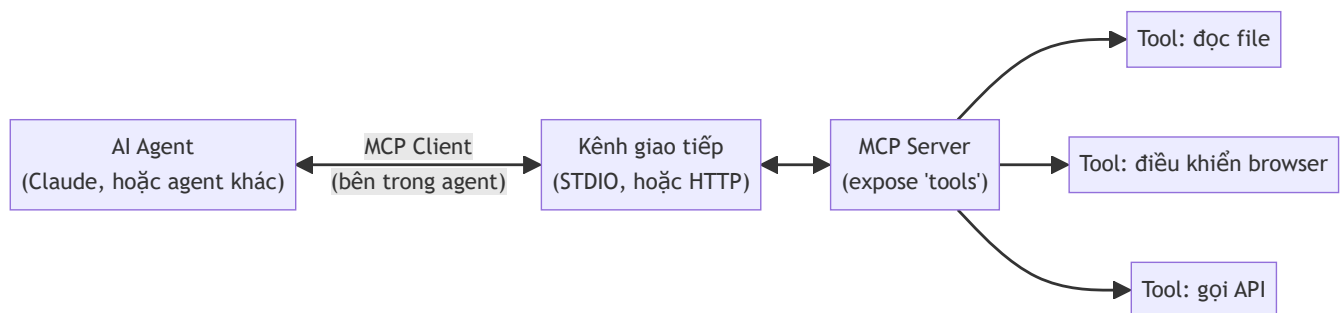
Mục tiêu học

- Giải thích được MCP là gì, ở mức kiến trúc client-server.
- Phân biệt "tool" trong MCP với hàm thông thường — vì sao AI agent gọi được nó.
- Tự viết và chạy được một MCP server + client tối giản, thấy toàn bộ luồng giao tiếp thật.

Code mẫu: `code/mcp-basics/` — MCP server/client chạy được thật, đã kiểm chứng: liệt kê tool, gọi tool, nhận kết quả — không phải mô tả lý thuyết suông.

27.1. MCP là gì?

Model Context Protocol (MCP) là một giao thức mở (do Anthropic khởi xướng, hiện được nhiều AI agent hỗ trợ) định nghĩa cách một **AI agent** (như Claude) giao tiếp với các **công cụ bên ngoài** — đọc file, gọi API, điều khiển trình duyệt, truy vấn database — theo một chuẩn thống nhất, thay vì mỗi công cụ phải viết tích hợp riêng cho mỗi AI khác nhau.



Ba khái niệm cốt lõi:

- **MCP Server** — chương trình expose một tập **tool** (và có thể cả "resource", "prompt") mà agent có thể gọi tới. Đây là phần **bạn** (hoặc người khác) viết ra.
- **MCP Client** — phần nằm **bên trong** AI agent, biết cách kết nối tới MCP server, liệt kê tool có sẵn, và gọi chúng khi AI quyết định cần dùng.
- **Tool** — một hàm có **tên, mô tả, schema tham số rõ ràng** — AI đọc mô tả này để quyết định "tool nào phù hợp với việc cần làm", rồi gọi với tham số đúng định dạng.

27.2. Vì sao liên quan đến automation test?

Chương 28 sẽ dùng **Playwright MCP server** — một MCP server chính thức expose các tool như "mở trang", "click", "điền form" — cho phép AI agent **tự điều khiển trình duyệt** để thực hiện tác vụ, hoặc hỗ trợ sinh code test (Chương 29). Hiểu MCP ở chương này là nền tảng bắt buộc trước khi dùng được Playwright MCP server một cách có chủ đích, không chỉ làm theo hướng dẫn mà không hiểu cơ chế bên dưới.

27.3. Viết một MCP server tối giản

```
// server.js
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { readFileSync } from "fs";

const server = new McpServer({ name: "book-demo-mcp-server", version: "1.0.0" });

server.registerTool(
  "count_login_test_cases",
  {
    title: "Đếm test case đăng nhập",
    description: "Đếm số test case (TC-xx) trong bộ test case đăng nhập ở Chương 1.",
    inputSchema: {}, // tool này không cần tham số
  },
  async () => {
    const content = readFileSync("../ch01-sdlc/test-case-list-login.md", "utf8");
    const ids = [...new Set(content.match(/\\bTC-\\d+\\b/g) || [])];
    return { content: [{ type: "text", text: `Có ${ids.length} test case: ${ids.join(", ")}.` }] };
  }
);

const transport = new StdioServerTransport();
await server.connect(transport); // lắng nghe qua STDIO - cách phổ biến nhất cho MCP
server local
```

Ba phần bắt buộc của một tool: **tên** (để agent gọi), **description** (để agent — thực chất là LLM — hiểu tool này dùng để làm gì và quyết định có nên gọi không), và **inputSchema** (định nghĩa tham số bằng Zod — agent biết chính xác cần truyền gì).

27.4. Viết một MCP client tối giản (đóng vai AI agent)

```
// client.js
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StdioClientTransport } from "@modelcontextprotocol/sdk/client/stdio.js";

const transport = new StdioClientTransport({ command: "node", args: ["server.js"] });
const client = new Client({ name: "book-demo-mcp-client", version: "1.0.0" });
await client.connect(transport);

const { tools } = await client.listTools(); // agent tự "khám phá" tool có sẵn
console.log(tools.map((t) => t.name));

const result = await client.callTool({ name: "count_login_test_cases", arguments: {} });
console.log(result.content[0].text);
```

Đây **chính xác** là cách một AI agent thật (Claude hay bất kỳ AI hỗ trợ MCP nào) giao tiếp với MCP server — không có bước "ma thuật" nào khác. Chạy thử:

```
cd code/mcp-basics
npm install
npm run demo
```

Output thật (đã kiểm chứng khi viết chương):

```
≡ 1. Danh sách tool mà server này cung cấp ≡
- count_login_test_cases: ...
- get_test_case_detail: ...

≡ 2. Gọi tool count_login_test_cases ≡
Bộ test case đăng nhập (Chương 1) có 10 test case: TC-01, TC-02, ... , TC-10.
```

Bài tập

1. Chạy `npm install && npm run demo` trong `code/mcp-basics/`, xác nhận output khớp như trên.
2. Đọc `server.js`, thêm một tool mới `list_all_bugs` — đọc `code/ch03-test-case-plan-bug/bug-report-example.md`, trả về ID và tiêu đề bug tìm được trong file đó.
3. Chạy `client.js` sau khi thêm tool mới, gọi thử tool đó — xác nhận `listTools()` hiện đủ 3 tool.
4. Giải thích bằng lời của bạn: nếu `description` của một tool viết mơ hồ (ví dụ chỉ ghi "xử lý dữ liệu"), điều gì có thể xảy ra khi một AI agent thật cố quyết định có nên gọi tool đó không?

Lỗi thường gặp

- **Viết `description` tool quá ngắn/mơ hồ**: AI agent dựa hoàn toàn vào text mô tả để quyết định gọi tool nào — mô tả mơ hồ dẫn đến agent gọi sai tool, hoặc không gọi tool đúng khi cần.
- **Nhầm MCP server với một REST API thông thường**: MCP có giao thức riêng (JSON-RPC qua STDIO hoặc HTTP), không gọi trực tiếp như `fetch()` một REST endpoint — cần dùng SDK/Client đúng cách.
- **Quên `await server.connect(transport)`**: server sẽ không lắng nghe được request nào, client kết nối tới sẽ treo hoặc lỗi timeout.

Tóm tắt & tiếp theo

- MCP là giao thức chuẩn cho AI agent gọi tool bên ngoài — Server expose tool, Client (bên trong agent) kết nối và gọi.
- Một tool cần tên, description rõ ràng, và schema tham số — AI dựa vào đó để quyết định gọi tool nào, với tham số gì.
- Đã tự viết và chạy được MCP server/client thật, không chỉ đọc lý thuyết.

Chương 28 dùng **Playwright MCP server** chính thức — một MCP server có sẵn, expose các tool điều khiển trình duyệt thật, để AI agent có thể tự thực hiện tác vụ trên web.

Chương 28: Playwright MCP server — để AI agent điều khiển trình duyệt

Mục tiêu học

- Cài đặt và khám phá tool của Playwright MCP server chính thức (`@playwright/mcp`).
- Hiểu cách một AI agent điều khiển trình duyệt qua MCP — khác gọi Playwright API trực tiếp (Chương 11-26) ở điểm nào.
- Tự viết một script "đóng vai agent", điều khiển demo-app hoàn toàn qua MCP tool calls.

Code mẫu: `code/playwright-mcp-demo/` — đã kiểm chứng thật: đăng nhập thành công vào demo-app chỉ bằng cách gọi tool MCP, không có dòng Playwright API nào được gọi trực tiếp.

28.1. Playwright MCP server là gì?

`@playwright/mcp` (do Microsoft phát triển, cùng đội ngũ Playwright) là một MCP server (nhắc lại kiến trúc ở Chương 27) expose các **tool điều khiển trình duyệt** — thay vì AI agent phải tự viết code Playwright, nó chỉ cần **gọi tool** như "mở URL", "click vào phần tử", "gõ text".

```
npm install @playwright/mcp @modelcontextprotocol/sdk
```

Chạy `list-tools.js` (`code/playwright-mcp-demo/`) để tự khám phá — kết quả thật khi viết chương này có **hơn 20 tool**, một số quan trọng nhất:

Tool	Vai trò
<code>browser_navigate</code>	Điều hướng tới một URL
<code>browser_snapshot</code>	Chụp accessibility snapshot (không phải ảnh — cây cấu trúc phần tử)
<code>browser_click</code>	Click vào phần tử
<code>browser_type</code>	Gõ text vào phần tử
<code>browser_take_screenshot</code>	Chụp ảnh màn hình (chỉ để xem, không dùng để xác định phần tử)
<code>browser_network_requests</code>	Xem danh sách network request đã xảy ra

28.2. Vì sao `browser_snapshot`, không phải ảnh chụp màn hình?

Đây là điểm thiết kế quan trọng: AI agent **không "nhìn" màn hình như con người** để tìm phần tử — nó đọc **accessibility snapshot**, một cây cấu trúc text mô tả từng phần tử (vai trò, tên, giá trị) — chính là dữ liệu mà `getByRole` / `getByLabel` (Chương 11) cũng dựa vào. Đây là lý do tư duy chọn locator tốt (Chương 6, 11) — dùng role/label rõ ràng — cũng trực tiếp giúp AI agent "thấy" đúng phần tử cần thao tác.

28.3. Tự viết một "agent" điều khiển demo-app qua MCP

```
// demo.js
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StdioClientTransport } from "@modelcontextprotocol/sdk/client/stdio.js";

const transport = new StdioClientTransport({ command: "npx", args: ["@playwright/mcp", "--headless"] });
const client = new Client({ name: "book-demo-agent", version: "1.0.0" });
await client.connect(transport);

async function callTool(name, args) {
  const result = await client.callTool({ name, arguments: args });
  return result.content.map((c) => c.text).join("\n");
}

await callTool("browser_navigate", { url: "http://localhost:3000/login" });
await callTool("browser_type", { target: "#email", text: "demo@example.com" });
await callTool("browser_type", { target: "#password", text: "Password123!" });
await callTool("browser_click", { target: "#submit-btn" });

const snapshot = await callTool("browser_snapshot", {});
console.log(snapshot.includes("demo@example.com") ? "Đăng nhập thành công!" : "Thất bại.");
```

Chạy thật (yêu cầu `demo-app` đang chạy ở terminal khác):

```
cd code/demo-app && npm start & # terminal 1
cd code/playwright-mcp-demo && npm run demo # terminal 2
```

Output thật:

```
✓ Đăng nhập THÀNH CÔNG qua MCP - thấy 'demo@example.com' trong snapshot.
```

Không có một dòng `page.goto()`, `page.fill()`, hay `page.click()` **nào trong** `demo.js` — mọi thao tác đi qua tool MCP (`browser_navigate`, `browser_type`, `browser_click`). Tham số `target` của các tool này nhận **CSS selector trực tiếp** (như `"#email"`) hoặc một tham chiếu lấy từ `browser_snapshot` — linh hoạt hơn locator Playwright thông thường vì AI agent thường không biết trước cấu trúc trang, cần tự khám phá qua snapshot trước.

28.4. So sánh: viết test bằng Playwright API trực tiếp vs điều khiển qua MCP

	Playwright API trực tiếp (Chương 11-26)	Điều khiển qua MCP (chương này)
Ai/cái gì gọi	Code test do người viết	AI agent (hoặc script mô phỏng agent)
Biết trước cấu trúc trang?	Có — người viết code đã xem trang, chọn locator cụ thể	Không nhất thiết — agent tự khám phá qua <code>browser_snapshot</code>
Phù hợp cho	Automation test cố định, chạy lại nhiều lần (regression)	Tác vụ một lần, thăm dò, hoặc để AI tự quyết định hành động tiếp theo
Ổn định/lặp lại	Rất cao (đã học kỹ locator ổn định — Chương 11)	Thấp hơn — phụ thuộc khả năng "hiểu" trang của AI ở mỗi lần chạy

Kết luận quan trọng: MCP không thay thế automation test truyền thống (Chương 11-26) — nó phục vụ mục đích khác: để AI agent **tự thực hiện tác vụ chưa biết trước**, hoặc **hỗ trợ sinh test case** (Chương 29), không phải để chạy regression suite ổn định hàng ngày.

Bài tập

- Chạy `npm run list-tools` trong `code/playwright-mcp-demo/`, đọc mô tả của `browser_fill_form` — nó khác `browser_type` gọi nhiều lần ở điểm nào?
- Chạy `npm run demo`, xác nhận output khớp như mục 28.3.
- Sửa `demo.js` để thử đăng nhập với mật khẩu sai — dùng `browser_snapshot` để tìm và in ra đúng thông báo lỗi "Email hoặc mật khẩu không đúng" (TC-02, Chương 1).
- Thử gọi `browser_navigate` tới `http://localhost:3000/orders` (chưa đăng nhập) — dùng `browser_snapshot` xác nhận agent "nhìn thấy" đúng việc đã bị chuyển về `/login` (TC-10).

Lỗi thường gặp

- Quên khởi động** `demo-app` **trước khi chạy** `demo.js`: `browser_navigate` sẽ thất bại vì không kết nối được tới `localhost:3000`.
- Nhầm** `browser_take_screenshot` **dùng để xác định phần tử**: tool này chỉ để **xem**, không dùng để lấy `target` cho `browser_click` / `browser_type` — phải dùng `browser_snapshot`.

- **Kỳ vọng kết quả 100% ổn định như automation test truyền thống:** MCP phù hợp cho tác vụ AI tự quyết định, không nên dùng để thay thế regression suite đã xây từ Chương 11 — xem mục 28.4.
-

Tóm tắt & tiếp theo

- Playwright MCP server expose tool điều khiển trình duyệt qua accessibility snapshot, không qua ảnh chụp màn hình.
- Đã tự viết và chạy được một "agent" điều khiển demo-app thật, hoàn toàn qua MCP tool calls.
- MCP phục vụ mục đích khác automation test truyền thống — bổ trợ, không thay thế Chương 11-26.

Chương 29 dùng chính khả năng "AI hiểu trang qua snapshot" vừa thấy ở chương này để hỗ trợ **sinh test case** và tìm hiểu khái niệm self-healing test.

Chương 29: Kết hợp AI sinh test case, self-healing test khi UI thay đổi

Mục tiêu học

- Hiểu cơ chế bên dưới việc AI hỗ trợ sinh test case — không phải "ma thuật", mà dựa trên accessibility snapshot (Chương 28).
- Tự viết một công cụ sinh khung test bằng heuristic, thấy rõ ranh giới giữa heuristic và AI thật.
- Viết và kiểm chứng thật kỹ thuật self-healing locator — tự phục hồi khi một chiến lược locator không còn đúng.

Code mẫu: `code/playwright-tests/utils/resilient-locate.ts` + `tests/ch29-self-healing.spec.ts` (self-healing, đã kiểm chứng 3/3 pass) và `code/playwright-mcp-demo/generate-test-skeleton.js` (sinh test từ snapshot, đã chạy thật).

29.1. AI sinh test case — cơ chế bên dưới là gì?

Các công cụ "AI sinh test case" (nhiều công cụ thương mại quảng cáo tính năng này) về bản chất làm 2 việc: (1) **quan sát** trang web — qua accessibility snapshot, giống `browser_snapshot` đã học ở Chương 28 — để biết trang có những phần tử nào, và (2) **suy luận** bằng LLM để quyết định nên điền giá trị gì, click gì, assert gì, dựa trên ngữ cảnh (tên field, nội dung trang...).

Chương này **không gọi LLM thật** (ngoài phạm vi một chương của sách, cần API key và chi phí riêng) — nhưng tách riêng được **phần (1)**, hoàn toàn có thể tự làm bằng code thường (heuristic), để thấy rõ phần nào là "cơ chế lấy dữ liệu" và phần nào mới thực sự cần AI.

29.2. Tự viết một công cụ sinh khung test bằng heuristic

```
// code/playwright-mcp-demo/generate-test-skeleton.js
const elementPattern = /(textbox|button|checkbox|combobox)\s+"([^\"]+)"/g;
// ... quét accessibility snapshot, tìm mọi phần tử thao tác được theo role + tên ...
```

Chạy thật trên trang `/login`:

```
cd code/playwright-mcp-demo
npm run generate -- http://localhost:3000/login
```

Kết quả thật (đã lưu ở `example-generated-login.spec.ts`):

```
import { test, expect } from "@playwright/test";

test("khung test tự sinh cho http://localhost:3000/login", async ({ page }) => {
  await page.goto("http://localhost:3000/login");

  await page.getByRole("textbox", { name: "Email" }).fill("TODO: điền giá trị");
  await page.getByRole("textbox", { name: "Mật khẩu" }).fill("TODO: điền giá trị");
  // await page.getByRole("button", { name: "Hiện ký tự" }).click();
  // await page.getByRole("button", { name: "Đăng nhập" }).click();

  // TODO: thêm assertion kiểm tra kết quả mong đợi
});
```

Đây chính xác là phần (1) ở mục 29.1 — tự động tìm ra "trang này có field Email, Mật khẩu, nút Hiện ký tự, nút Đăng nhập". **Phần (2) — AI thật sẽ làm tiếp:** biết "Email" nên điền `demo@example.com` (không phải chuỗi ngẫu nhiên), biết nên click "Đăng nhập" sau khi điền xong (bỏ comment dòng đó), và biết nên assert `toHaveURL("/dashboard")` dựa trên hiểu ngữ cảnh nghiệp vụ "đăng nhập" — những việc heuristic thuần **không** làm được.

Bài học quan trọng nhất của mục này: dù dùng công cụ AI sinh test case thật, kết quả sinh ra vẫn cần được **review kỹ** — đúng tinh thần đã học ở Chương 17 với `codegen`: điểm khởi đầu tốt, không phải sản phẩm cuối.

29.3. Self-healing locator — không cần AI

Self-healing test là kỹ thuật giúp test **tự phục hồi** khi UI thay đổi nhẹ, không cần con người sửa code test ngay. Nhiều công cụ thương mại quảng cáo "self-healing bằng AI", nhưng nguyên lý cốt lõi — thử nhiều chiến lược locator theo thứ tự ưu tiên — **không cần AI**:

```
// utils/resilient-locate.ts
export async function resilientLocate(page, strategies, timeoutMs = 2000) {
  for (const strategy of strategies) {
    const locator = strategy.locate();
    try {
      await locator.waitFor({ state: "visible", timeout: timeoutMs });
      return { locator, strategyUsed: strategy.name };
    } catch {
      continue; // chiến lược này không tìm được - thử chiến lược tiếp theo
    }
  }
  throw new Error(`resilientLocate: không chiến lược nào tìm được phần tử`);
}
```

Kiểm chứng thật — giả lập tình huống "dev đổi ID input email":

```
const { locator: emailInput, strategyUsed } = await resilientLocate(page, [
  { name: "css cũ (#email-old) - giả lập ID đã đổi", locate: () => page.locator("#email-old") },
  { name: "getByLabel Email - chiến lược dự phòng", locate: () =>
    page.getByLabel("Email") },
]);

expect(strategyUsed).toBe("getByLabel Email - chiến lược dự phòng");
```

Chiến lược đầu (`#email-old`) cố tình sai (mô phỏng UI đã đổi) — `resilientLocate` tự chuyển sang `getByLabel("Email")` (chiến lược dựa trên role/label, ổn định hơn — nhắc lại Chương 11) mà **không cần sửa code test nào**. Đã kiểm chứng: 3/3 test pass, gồm cả trường hợp không chiến lược nào đúng — báo lỗi rõ ràng thay vì treo im lặng.

29.4. Ranh giới giữa self-healing "thật" (AI) và kỹ thuật ở mục 29.3

`resilientLocate` chỉ hoạt động khi **bạn đã lường trước** các chiến lược dự phòng hợp lý (viết sẵn trong code). Self-healing dựa trên AI thật đi xa hơn: khi TẤT CẢ chiến lược viết sẵn đều thất bại, AI có thể **tự phân tích accessibility snapshot mới**, suy luận phần tử nào "giống" phần tử cũ nhất (dựa trên vị trí, text xung quanh, vai trò...) mà không cần con người viết trước chiến lược dự phòng cụ thể đó. Đây là điểm khác biệt cốt lõi — không phải "có AI hay không" một cách mơ hồ, mà là "cần con người lường trước tình huống, hay để hệ thống tự suy luận tình huống mới".

Bài tập

1. Chạy `cd code/playwright-tests && npm test -- ch29-self-healing`, xác nhận 3/3 pass.

2. Chạy `cd code/playwright-mcp-demo && npm run generate -- http://localhost:3000/profile` (yêu cầu demo-app đang chạy) — so sánh khung test sinh ra với `pages/ProfilePage.ts` (Chương 15) đã viết tay, thiếu những gì so với phiên bản hoàn chỉnh?
3. Thêm một chiến lược thứ ba vào bài test self-healing — dùng `getByPlaceholder` hoặc `getByTestId` (giả định demo-app có `data-testid`) làm chiến lược dự phòng thứ hai, trước `getByLabel`.
4. Viết ra (không cần code) một tình huống mà `resilientLocate` **không** đủ để xử lý — tình huống đó cần loại "self-healing" nào (dựa AI thật) để giải quyết?

Lỗi thường gặp

- **Tin tưởng tuyệt đối vào công cụ "AI sinh test case", không review:** kết quả sinh ra (dù bằng heuristic hay AI thật) đều cần con người kiểm tra lại giá trị điền, assertion — đây là nguyên tắc xuyên suốt từ Chương 17 (`codegen`).
- **Nhằm self-healing với "test không bao giờ fail":** self-healing chỉ xử lý được tình huống đã lường trước (chiến lược dự phòng có sẵn) — nó không biến một test sai logic thành đúng.
- **Viết quá nhiều chiến lược dự phòng "phòng hờ":** mỗi chiến lược thêm là thêm một khả năng che giấu lỗi thật (test "tự sửa" qua một chiến lược không phù hợp, dẫn tới test không còn kiểm tra đúng ý định ban đầu) — chỉ thêm chiến lược dự phòng có cơ sở thực tế.

Tóm tắt & tiếp theo

- AI sinh test case = quan sát (accessibility snapshot, đã tự làm được bằng heuristic) + suy luận (cần AI thật) — tách được hai phần này giúp hiểu rõ AI thực sự đóng góp gì.
- Self-healing locator (thử nhiều chiến lược theo thứ tự) không cần AI, hoạt động tốt cho tình huống đã lường trước — đã kiểm chứng thật với 3/3 test pass.
- Self-healing dựa trên AI thật xử lý được tình huống MỚI, chưa từng lường trước — khác biệt cốt lõi với kỹ thuật ở chương này.

Chương 30 — chương cuối cùng của sách — tổng hợp toàn bộ kiến thức qua một case study thực tế: xây dựng một pipeline test bán tự động kết hợp Playwright, CI/CD, và AI/MCP.

Chương 30: Case study thực tế — xây pipeline test bán tự động với AI + Playwright

Mục tiêu học

- Áp dụng lại toàn bộ quy trình đã học — từ khái niệm QA (Phần I) đến AI/MCP (Phần VI) — cho một tính năng **hoàn toàn mới**, viết ra ngay trong chương này.
- Thấy rõ visual testing (Chương 21) tự phát hiện một thay đổi UI thật khi tính năng mới được thêm.
- Tổng kết: một pipeline test bán tự động trông như thế nào khi kết hợp đủ các kỹ thuật.

Code mẫu: `code/playwright-tests/pages/ForgotPasswordPage.ts` + `tests/ch30-case-study.spec.ts` (5 test case, CS-01 → CS-05) — tự động hoá đầy đủ một tính năng mới ("Quên mật khẩu") vừa được thêm vào `demo-app` cho chính chương này.

30.1. Bài toán: thêm tính năng "Quên mật khẩu"

Demo app (từ Chương 9) chưa có tính năng quên mật khẩu. Chương này đóng vai một task thực tế: viết requirement, test case, tự động hoá, và xác nhận qua CI — đúng chu trình SDLC đã học ở Chương 1, nhưng lần này áp dụng đủ mọi kỹ thuật đã học trong suốt 29 chương trước.

Yêu cầu (rút gọn): Trang `/forgot-password` cho phép nhập email, gửi yêu cầu, luôn hiển thị một thông báo chung (không tiết lộ email có tồn tại trong hệ thống hay không — đúng nguyên tắc bảo mật đã học ở TC-03, Chương 1 và 3).

30.2. Bước 1: Viết test case trước (shift-left, Chương 1)

ID	Tiêu đề	Kết quả mong đợi
CS-01	Gửi yêu cầu với email hợp lệ	Hiển thị: "Nếu email tồn tại trong hệ thống, hướng dẫn đặt lại mật khẩu đã được gửi."
CS-02	Gửi yêu cầu với email KHÔNG tồn tại	Cùng thông báo với CS-01 — không tiết lộ khác biệt
CS-03	Bỏ trống email	Không submit được, lỗi "Email là bắt buộc"
CS-04	Sai định dạng email	Không submit được, lỗi "Email không đúng định dạng"
CS-05	Từ trang đăng nhập, bấm "Quên mật khẩu?"	Chuyển đúng sang <code>/forgot-password</code>

CS-02 là test case **quan trọng nhất** — nó kiểm tra một thuộc tính bảo mật (không rò rỉ thông tin tài khoản), không chỉ "tính năng hoạt động".

30.3. Bước 2: Cài đặt tính năng, dùng MCP sinh khung test nháp (Chương 28-29)

Sau khi thêm route `/forgot-password` vào `demo-app` (xem `src/views.js`, `server.js`), dùng lại công cụ đã viết ở Chương 29 để có điểm khởi đầu:

```
cd code/playwright-mcp-demo
npm run generate -- http://localhost:3000/forgot-password
```

Kết quả (khung nháp, chưa hoàn chỉnh):

```
test("khung test tự sinh cho .../forgot-password", async ({ page }) => {
  await page.goto("http://localhost:3000/forgot-password");
  await page.getByRole("textbox", { name: "Email" }).fill("TODO: điền giá trị");
  // await page.getByRole("button", { name: "Gửi hướng dẫn" }).click();
  // TODO: thêm assertion kiểm tra kết quả mong đợi
});
```

30.4. Bước 3: Hoàn thiện thành Page Object + test thật (Chương 15, 11)

```
// pages/ForgotPasswordPage.ts
export class ForgotPasswordPage {
  constructor(private page: Page) {}
  async goto() { await this.page.goto("/forgot-password"); }
  async requestReset(email: string) {
    await this.page.getByLabel("Email").fill(email);
    await this.page.getByRole("button", { name: "Gửi hướng dẫn" }).click();
  }
  get emailError() { return this.page.locator("#email-error"); }
  get serverSuccess() { return this.page.locator("#server-success"); }
}
```

```
// tests/ch30-case-study.spec.ts
test("CS-02: email không tồn tại vẫn nhận CÙNG thông báo", async ({ page }) => {
  const forgotPasswordPage = new ForgotPasswordPage(page);
  await forgotPasswordPage.goto();
  await forgotPasswordPage.requestReset("khong-ton-tai@example.com");

  await expect(forgotPasswordPage.serverSuccess).toHaveText(
    "Nếu email tồn tại trong hệ thống, hướng dẫn đặt lại mật khẩu đã được gửi."
  );
});
```

Chạy `npm test -- ch30-case-study`: **5/5 pass** — cả 5 test case đã được tự động hoá đầy đủ.

30.5. Một phát hiện thật: visual test (Chương 21) tự bắt được thay đổi

Sau khi thêm link "Quên mật khẩu?" vào trang đăng nhập (`loginPage()` trong `views.js`) và chạy lại **toàn bộ** suite, `ch21-visual-testing.spec.ts` **fail**:

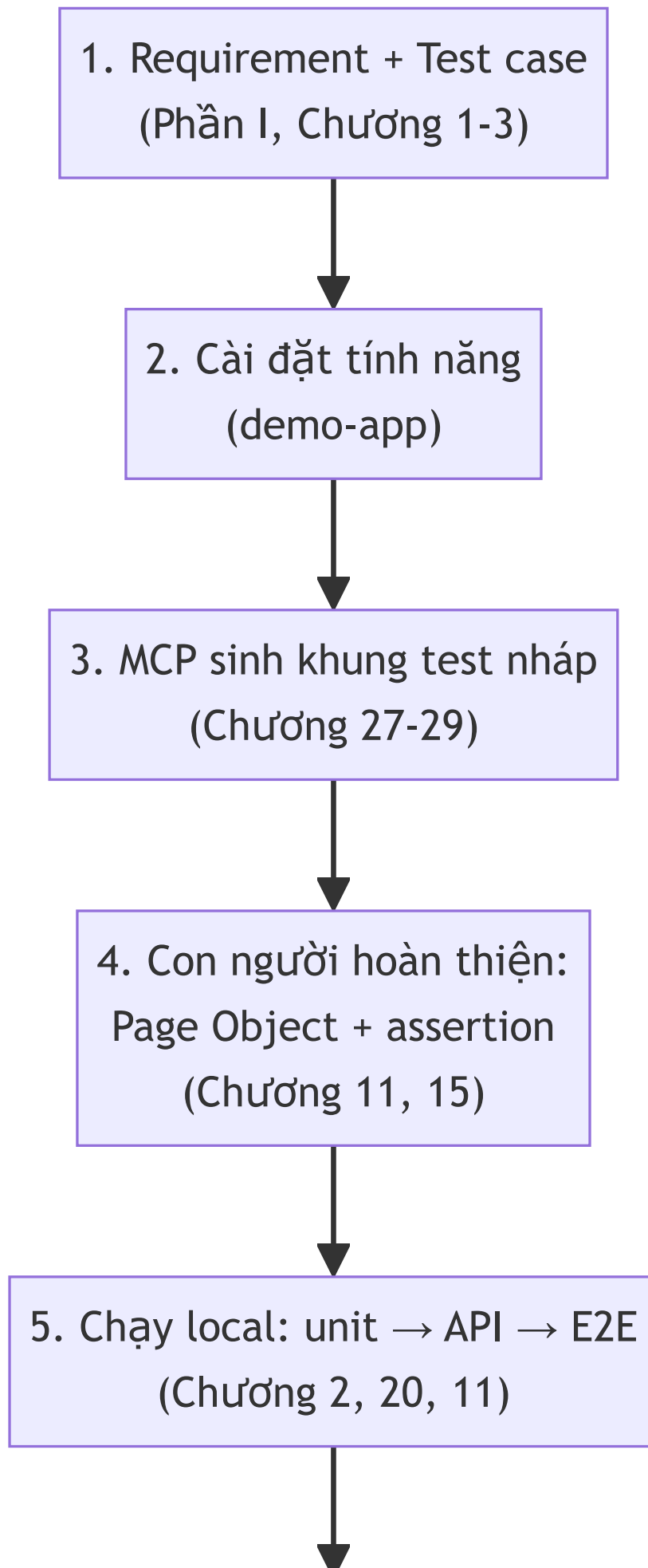
```
Expected: tests\ch21-visual-testing.spec.ts-snapshots\login-page-chromium-win32.png
Received: ... \login-page-actual.png
```

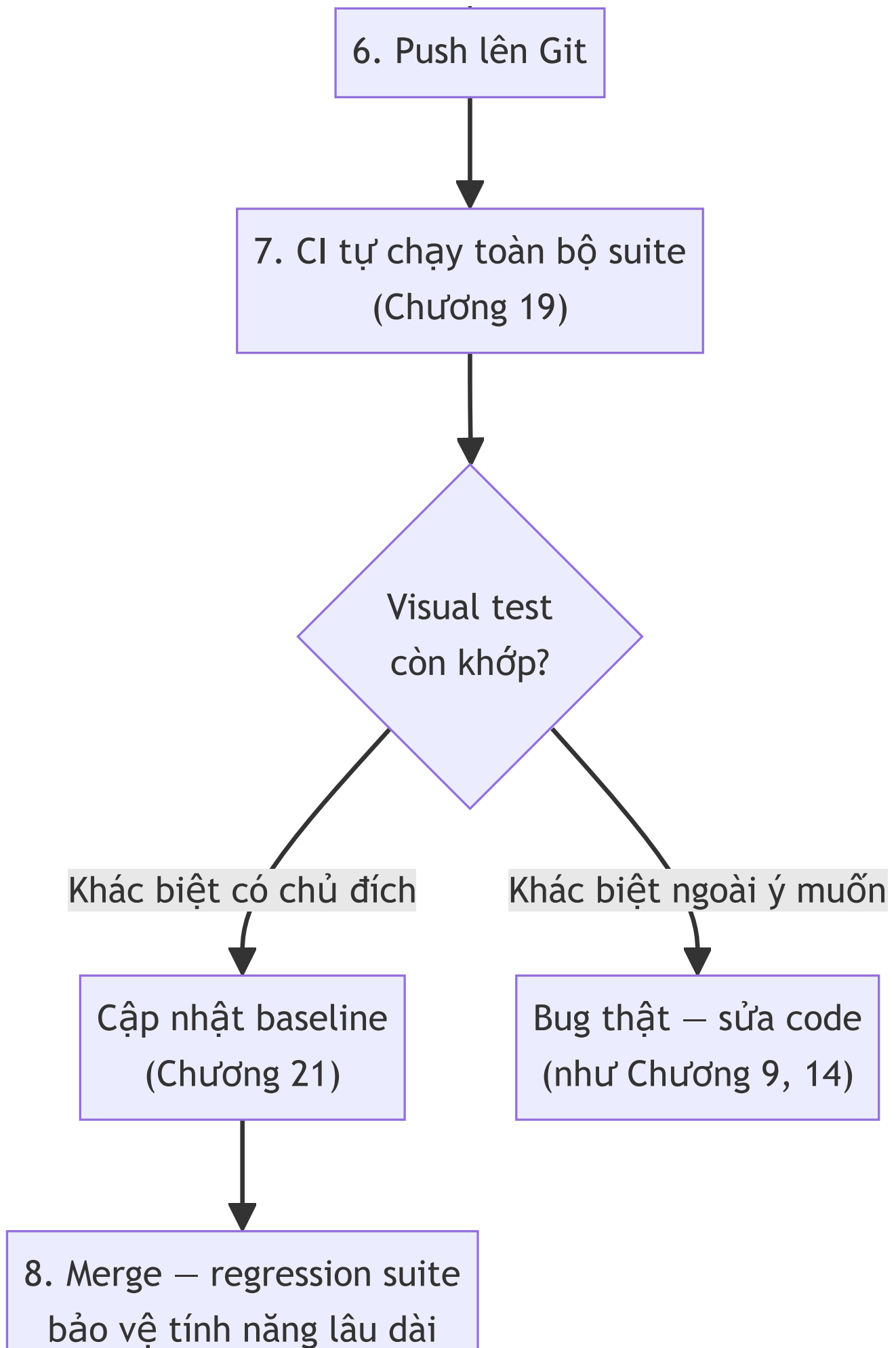
Đây không phải lỗi — đây là visual testing đang làm đúng việc của nó: phát hiện trang `/login` đã thay đổi layout (thêm 1 dòng link mới) so với baseline cũ. Vì thay đổi này là **có chủ đích** (tính năng mới), bước xử lý đúng là cập nhật lại baseline:

```
npx playwright test ch21-visual-testing --update-snapshots
```

Sau đó chạy lại toàn bộ suite: **63/63 pass**. Đây là minh chứng thật, xảy ra ngay khi viết chương này — không phải ví dụ dựng sẵn — cho quy trình thật mà một team sẽ làm mỗi khi visual test báo "khác biệt": xác nhận thay đổi có chủ đích hay không, rồi mới quyết định cập nhật baseline hay sửa lại code.

30.6. Toàn cảnh: một pipeline test bán tự động





"Bán tự động" (không phải "hoàn toàn tự động") vì con người vẫn giữ vai trò quan trọng ở bước 1 (quyết định test case nào cần), bước 4 (review và hoàn thiện code do MCP/heuristic sinh ra), và bước 8 (quyết định một khác biệt visual là bug hay thay đổi có chủ đích). AI/MCP hỗ trợ tăng tốc các bước lặp lại (bước 3), không thay thế phán đoán của con người ở các bước quyết định.

Bài tập tổng hợp (áp dụng lại từ Chương 1 đến 30)

1. Chạy toàn bộ `npm test` trong `code/playwright-tests/`, xác nhận 63/63 pass.
2. Chọn một tính năng khác bạn muốn thêm vào demo-app (ví dụ: đổi mật khẩu khi đã đăng nhập). Áp dụng lại đúng quy trình ở mục 30.2-30.4: viết test case trước, cài đặt, dùng MCP sinh khung nháp, hoàn thiện thành Page Object + test thật.
3. Sau khi thêm tính năng ở bài 2, chạy lại toàn bộ suite — nếu `ch21-visual-testing` fail, làm theo đúng quy trình ở mục 30.5 để xử lý.
4. Nhìn lại sơ đồ ở mục 30.6 — với dự án thật bạn đang làm (nếu có), vẽ lại sơ đồ tương tự, đánh dấu bước nào bạn đã có, bước nào chưa có.

Tóm tắt toàn sách

- **Phần I-II:** QA không phải một giai đoạn cuối cùng — testing xuyên suốt SDLC, và automation QA là sự kết hợp giữa tư duy QA và năng lực kỹ thuật.
- **Phần III-IV:** Playwright cung cấp API vững chắc (locators, actions, assertions, auto-waiting) cùng cấu trúc tổ chức test (POM, fixtures, CI/CD) để xây một suite bền vững.
- **Phần V:** Các kỹ thuật nâng cao (API testing, visual testing, storage state, parallel, cross-browser, component testing) mở rộng phạm vi và tốc độ của suite đó.
- **Phần VI:** MCP mở ra một trục hoàn toàn khác — AI agent tự điều khiển trình duyệt, hỗ trợ sinh test case — bổ trợ, không thay thế, automation test truyền thống.

Xuyên suốt sách, mọi ví dụ đều **chạy được thật**, và ít nhất 2 lần automation test tự phát hiện bug thật (Chương 9, 14) — đây chính là giá trị cốt lõi mà cuốn sách muốn truyền tải: automation testing không phải để "có test cho có", mà để phát hiện vấn đề thật, sớm nhất có thể, lặp lại được mãi mãi.

Phụ lục

Phụ lục A: Best practices & checklist automation test

Bảng tổng hợp các nguyên tắc đã học xuyên suốt sách, tổ chức theo checklist thực dụng — dùng để tự rà soát trước khi coi một automation test (hoặc cả suite) là "xong".

Checklist khi viết một test case mới

- [] **Test case có "Kết quả mong đợi" cụ thể, đo lường được** trước khi viết code — không viết code test khi chưa rõ ràng cái gì là "đúng" (Chương 3).
- [] **Locator ưu tiên theo** `getByRole` / `getByLabel` / `getByText`, chỉ dùng CSS/XPath khi không còn lựa chọn nào tốt hơn (Chương 6, 11).
- [] **Không có** `waitForTimeout` — dựa vào auto-waiting của action/assertion; nếu cần chờ một network request cụ thể, dùng `waitForResponse` (Chương 14).
- [] **Assertion khớp chính xác với "Kết quả mong đợi"** trong test case gốc — không chỉ kiểm tra "có gì đó xảy ra" (Chương 6, 11).
- [] **Test độc lập hoàn toàn** — không phụ thuộc thứ tự chạy, không để lại trạng thái ảnh hưởng test khác (Chương 6, 11).
- [] **Dữ liệu test được seed/reset rõ ràng** (qua API test-only, hoặc `beforeEach`) — không dựa vào trạng thái "may rủi" còn lại từ lần chạy trước (Chương 1, 22).

Checklist khi tổ chức test lớn hơn

- [] **Logic dùng chung được tách vào Page Object** khi nhiều test cùng dùng một trang (Chương 15).
- [] **Setup/teardown lặp lại được tách vào fixture**, không lặp lại trong từng `beforeEach` (Chương 16).
- [] **Dữ liệu mutable (tài khoản, trạng thái) không dùng chung giữa test chạy song song** — mỗi worker seed dữ liệu riêng nếu cần chạy song song an toàn (Chương 23).
- [] `test.step()` cho test dài, nhiều bước — dễ đọc trace/report khi fail (Chương 12).

Checklist trước khi đưa vào CI/CD

- [] `npm ci` **chạy sạch** (không chỉ `npm install`) từ lockfile, xác nhận đúng phiên bản dependency (Chương 19).
- [] `forbidOnly: true` **trên CI** — chặn `test.only()` sót lại làm CI bỏ qua test khác (Chương 19).
- [] `retries` **chỉ bật trên CI**, không bật khi dev cục bộ (Chương 19).
- [] **Artifact (report, trace) được upload** dù test pass hay fail (`if: always()`) — luôn có dữ liệu để debug khi CI fail (Chương 19).
- [] **Visual test snapshot tạo trên đúng môi trường CI** (Docker, hoặc trực tiếp trên CI), không tạo trên máy dev rồi commit thẳng (Chương 21).

Checklist về tư duy (không phải cấu hình)

- [] **Không phải mọi test case đều nên tự động hoá** — cân nhắc tần suất chạy lại, độ ổn định (Chương 5).
- [] **Code sinh ra từ `codegen` /AI/heuristic luôn cần review**, không phải sản phẩm cuối (Chương 17, 29).
- [] **Khi test fail, đọc kỹ thông báo lỗi trước khi kết luận** — nhiều khi test đang "làm đúng việc của nó": phát hiện vấn đề thật (Chương 9, 14).
- [] **Automation test không thay thế Manual/Exploratory testing** — bổ trợ cho nhau, không chọn một bỏ một (Chương 2).

Tham khảo nhanh: nguyên tắc "5 điều kiện auto-waiting"

Trước khi thực hiện action, Playwright chờ phần tử: (1) attached vào DOM, (2) visible, (3) stable (không đang animation), (4) receives events (không bị đê), (5) không disabled. Hiểu 5 điều kiện này giúp debug nhanh hơn khi một action bị timeout (Chương 14).

Phụ lục B: Các lỗi thường gặp và cách xử lý (tra cứu nhanh theo triệu chứng)

Tổng hợp toàn bộ mục "Lỗi thường gặp" trong 30 chương, tổ chức theo **triệu chứng** để tra cứu nhanh khi gặp lỗi tương tự — mỗi dòng trở lại chương gốc có giải thích chi tiết.

Test fail ngẫu nhiên (flaky), không nhất quán

Triệu chứng	Nguyên nhân thường gặp	Xem lại
Test đôi khi pass đôi khi fail, không đổi gì trong code	Dùng <code>waitForTimeout</code> thay vì auto-waiting; máy chạy nhanh/chậm khác nhau	Chương 6, 14
Test pass riêng lẻ nhưng fail khi chạy cùng lúc với test khác	Nhiều test dùng chung dữ liệu/tài khoản, chạy song song gây đụng độ	Chương 11, 23
Test pass trên máy dev, fail trên CI	Thiếu <code>npm ci</code> / cài đặt đầy đủ trên CI; visual test baseline tạo trên OS khác	Chương 19, 21

Locator không tìm được phần tử, hoặc tìm nhầm

Triệu chứng	Nguyên nhân thường gặp	Xem lại
<code>strict mode violation: resolved to N elements</code>	Locator khớp nhiều phần tử — thường do <code>aria-label</code> /text trùng lặp một phần	Chương 9, 11
Locator hoạt động lúc viết, vỡ sau khi dev đổi UI	Dùng CSS selector/cấu trúc DOM thay vì role/label	Chương 6, 11, 29
<code>getByLabel</code> không tìm được input	Input thiếu <code><label for></code> liên kết, hoặc dùng <code>aria-label</code> không khớp	Chương 11

Network / async không như mong đợi

Triệu chứng	Nguyên nhân thường gặp	Xem lại
UI "treo im lặng" khi server lỗi	Code frontend gọi <code>res.json()</code> mà không kiểm tra <code>res.ok</code> /bọc try-catch	Chương 14
<code>waitForResponse</code> bị treo mãi	Gọi <code>waitForResponse</code> SAU khi đã trigger request, bị lỡ response	Chương 14, 20
Test gọi API nhưng <code>page</code> vẫn coi là chưa đăng nhập	Dùng <code>request</code> (độc lập) thay vì <code>page.request</code> (chung context)	Chương 20

Cấu hình / môi trường

Triệu chứng	Nguyên nhân thường gặp	Xem lại
<code>Executable doesn't exist</code> khi chạy test	Quên <code>npx playwright install</code> sau khi <code>npm install</code>	Chương 8
<code>webServer</code> timeout chờ URL sẵn sàng	Thiếu <code>npm install</code> ở project server (demo-app), hoặc chưa chạy đúng thư mục	Chương 9, 19
CI fail ở project firefox/webkit nhưng chromium pass	Chỉ cài Chromium (<code>playwright install chromium</code>), thiếu 2 engine còn lại	Chương 24
MCP server treo, không phản hồi	Quên <code>await server.connect(transport)</code>	Chương 27

Reporting / debug

Triệu chứng	Nguyên nhân thường gặp	Xem lại
Không có báo cáo HTML Allure dù đã cài <code>allure-playwright</code>	Thiếu bước sinh báo cáo cuối bằng Allure CLI (cần Java) — package Node.js chỉ ghi dữ liệu thô	Chương 18
Trace/report quá nặng, chiếm nhiều dung lượng	Đặt <code>trace: "on"</code> cho chạy hàng ngày thay vì <code>"on-first-retry"</code>	Chương 17

Tư duy / quy trình (không phải lỗi kỹ thuật)

Triệu chứng	Nguyên nhân thường gặp	Xem lại
Test pass nhưng không phát hiện được bug thật	Assertion mơ hồ, không khớp đúng "Kết quả mong đợi" gốc	Chương 3, 6, 13
Automation suite bị bỏ qua khi báo fail	Tích tụ flaky test lâu ngày, đội ngũ mất niềm tin vào suite	Chương 6
Code sinh bởi <code>codegen</code> /AI dùng thẳng, không sửa	Thiếu bước review/tinh gọn trước khi coi là test hoàn chỉnh	Chương 17, 29

Cách dùng phụ lục này

Khi gặp lỗi mới không có trong bảng: (1) đọc kỹ thông báo lỗi đầy đủ trước khi tra cứu, (2) tìm chương liên quan tới **loại thao tác** đang thực hiện (locator, network, config...), (3) áp dụng quy trình debug đã học ở Chương 17 (Trace Viewer, Console tab) trước khi đoán mò.

Phụ lục C: Tài liệu tham khảo & cộng đồng

Tài liệu chính thức

Công cụ	Tài liệu
Playwright	playwright.dev — tài liệu chính, luôn cập nhật theo phiên bản mới nhất
Playwright Test API reference	playwright.dev/docs/api/class-test — tra cứu mọi fixture, matcher
@playwright/mcp	github.com/microsoft/playwright-mcp — Playwright MCP server chính thức (Chương 28)
Model Context Protocol	modelcontextprotocol.io — đặc tả giao thức MCP, SDK chính thức (Chương 27)
Allure Report	allurereport.org — tài liệu Allure CLI, tích hợp với nhiều framework (Chương 18)
Express	expressjs.com — framework dùng cho demo-app trong sách

Từ cuốn sách này

Toàn bộ code mẫu trong sách nằm ở code/ — mỗi thư mục có README.md riêng, giải thích cách chạy và vai trò từng file:

- code/ch01-sd1c/, ch02- ... / ... ch06- ... / — ví dụ Phần I-II (test case, ROI calculator, POM đối chiếu tốt/xấu).
- code/demo-app/ — web app mẫu dùng xuyên suốt Phần IV-VI.
- code/playwright-tests/ — project Playwright chính, 30 chương test tích lũy.
- code/component-testing/ — ví dụ component testing (Chương 25).
- code/mcp-basics/, code/playwright-mcp-demo/ — ví dụ MCP (Chương 27-29).

Cộng đồng & nơi hỏi đáp

- Playwright Discord** — cộng đồng chính thức, phản hồi nhanh cho câu hỏi kỹ thuật.
- GitHub Discussions** của microsoft/playwright — tìm kiếm vấn đề đã có người gặp trước.
- Stack Overflow**, tag playwright — lượng câu hỏi/đáp lớn, nhiều tình huống thực tế.

Gợi ý học tiếp sau khi đọc xong sách

- Đọc changelog Playwright** mỗi khi có phiên bản mới — framework cập nhật khá nhanh, API mới (như browser_snapshot-style tooling ở Chương 28) xuất hiện đều đặn.

2. **Thử áp dụng lên một dự án thật** của bạn — chuyển toàn bộ demo-app trong sách thành nền tảng để thực hành trên ứng dụng thực tế bạn đang làm việc.
 3. **Tìm hiểu sâu hơn về Docker cho visual testing** (đã nhắc ở Chương 21) — cần thiết khi triển khai visual testing thật trong CI đa hệ điều hành.
 4. **Theo dõi sự phát triển của MCP** — giao thức còn mới, nhiều tool/agent hỗ trợ đang được thêm liên tục; kiến thức nền tảng ở Phần VI sẽ giúp bạn thích nghi nhanh với thay đổi.
-

Lời kết

Cảm ơn bạn đã theo hết 30 chương. Cuốn sách này được viết với một nguyên tắc xuyên suốt: **mọi ví dụ đều chạy được thật**, không có đoạn code minh họa nào chỉ tồn tại trên giấy. Hy vọng cách tiếp cận đó giúp bạn tự tin áp dụng automation testing vào công việc thực tế — không chỉ hiểu lý thuyết, mà biết chính xác nó hoạt động (và đôi khi thất bại) ra sao trong thực tế.